

第 4 章

處理器

4.1 介紹

4.2 邏輯設計慣例

4.3 建構數據通道

4.4 簡單實作方案

4.5 管道化處理概觀

4.6 管道化數據通道及控制

4.7 數據危障：前饋或停滯

4.8 控制危障

4.9 例外

4.10 平行性與進階的指令階層平行性

4.11 實例：ARM Cortex-A8 與 Intel Core i7 的管道

4.12 走得更快：指令階層平行性與矩陣乘法

4.14 謬誤與陷阱

4.15 總結評論

4.1 介紹

- ▣ 第一章說明了計算機的效能是由下列三個主要的因素所決定：指令個數、時脈週期時間以及平均每個指令的時脈週期數(CPI)
越⼩越好
- ▣ 第二章說明了編譯器和指令集架構決定一個程式需要的指令個數
- ▣ 處理器的實作決定時脈週期時間和平均每個指令的時脈週期數兩者
 - 本章對MIPS 指令集的兩種不同實作方式建構其數據通道(datapath)和控制單元(control unit)

基本的MIPS 實作

- ▣ 探討MIPS 核心指令集中部分指令的實作：
 - 載入字組(lw)和儲存字組(sw)
 - add、sub、AND、OR 和slt
 - 若相等則分支(beq)以及跳躍指令(j)
- ▣ 可了解
 - 指令集架構如何決定實作上的方方面面
 - 不同實作策略如何影響計算機的時脈速率以及CPI

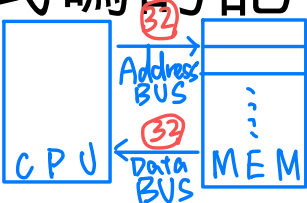
實作的概觀

在實作這些不同類別的指令時，每個指令開始的兩個步驟都一樣：

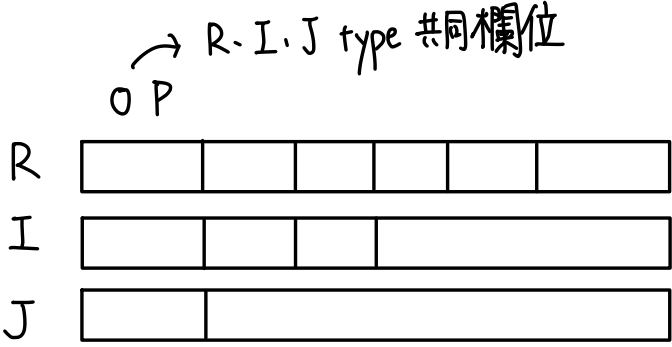
Program Counter

Read 1. 將程式計數器(PC)的內容送到含有程式碼的記憶體，並且從記憶體中擷取指令

從mem拿指令去CPU執行



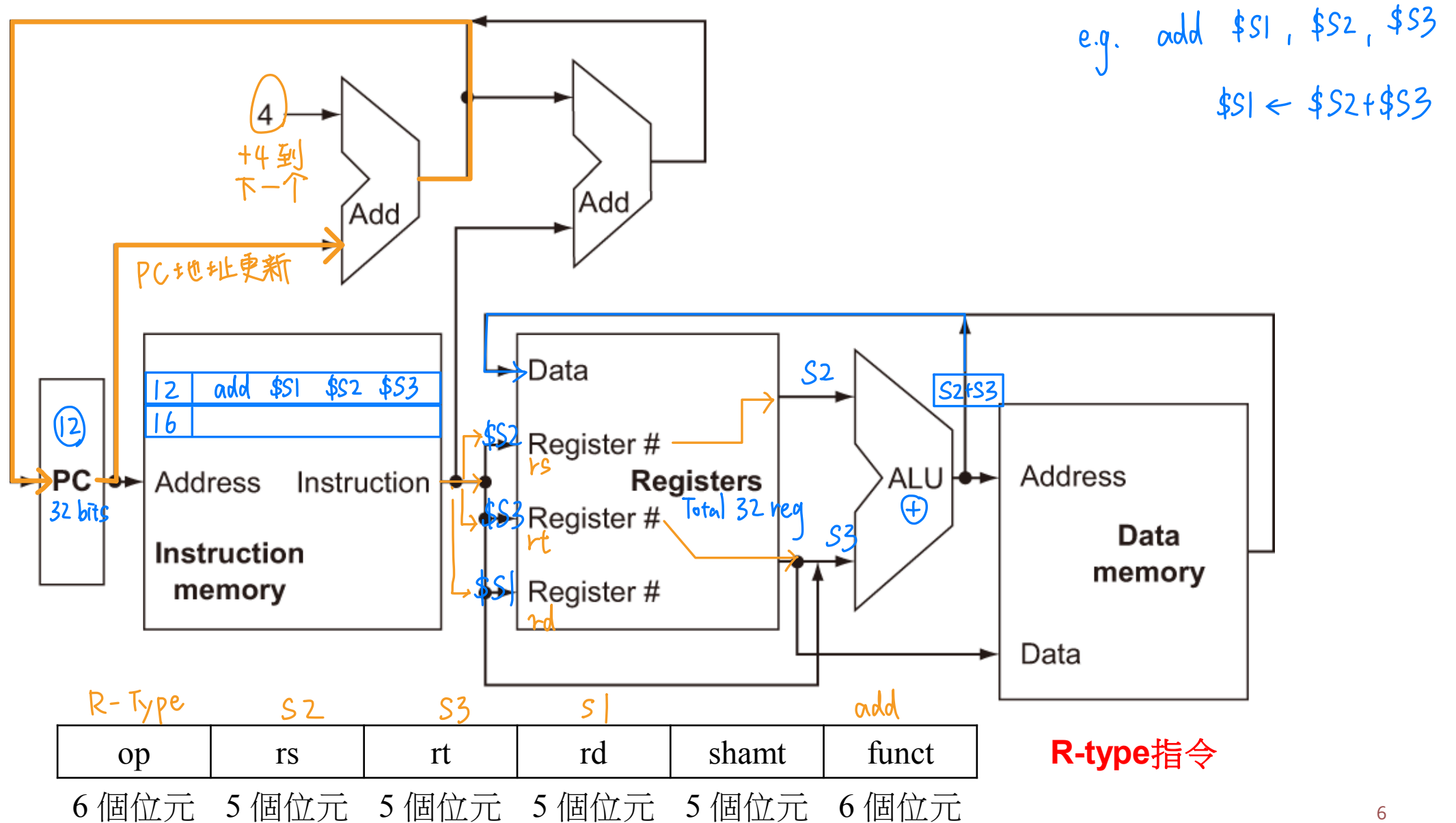
Decode 2. (op-code) 使用指令中的欄位來選擇並讀取一個或兩個暫存器的內容。我們只需對載入字組指令讀取一個暫存器，然而大部分其他指令則需讀取兩個暫存器

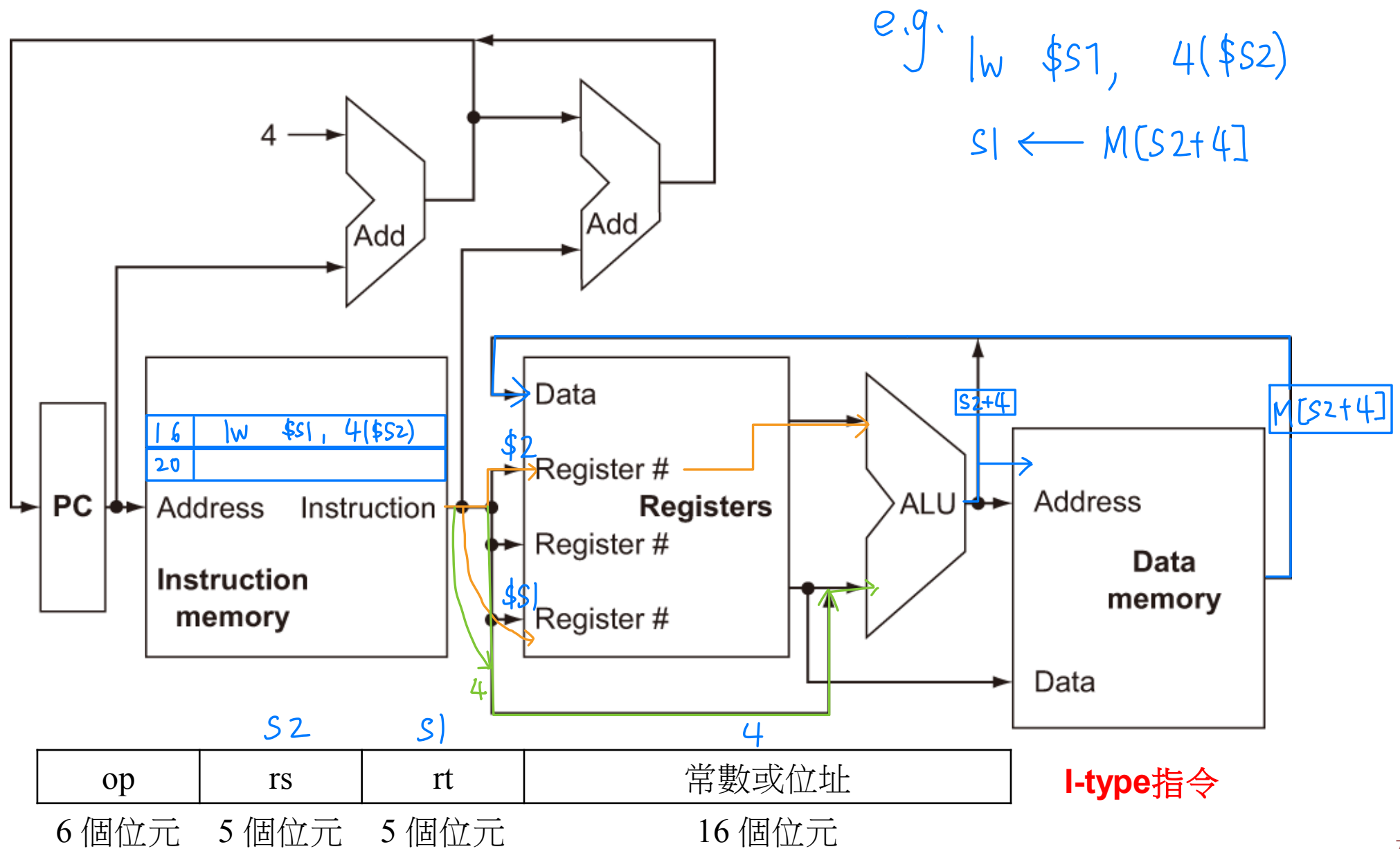


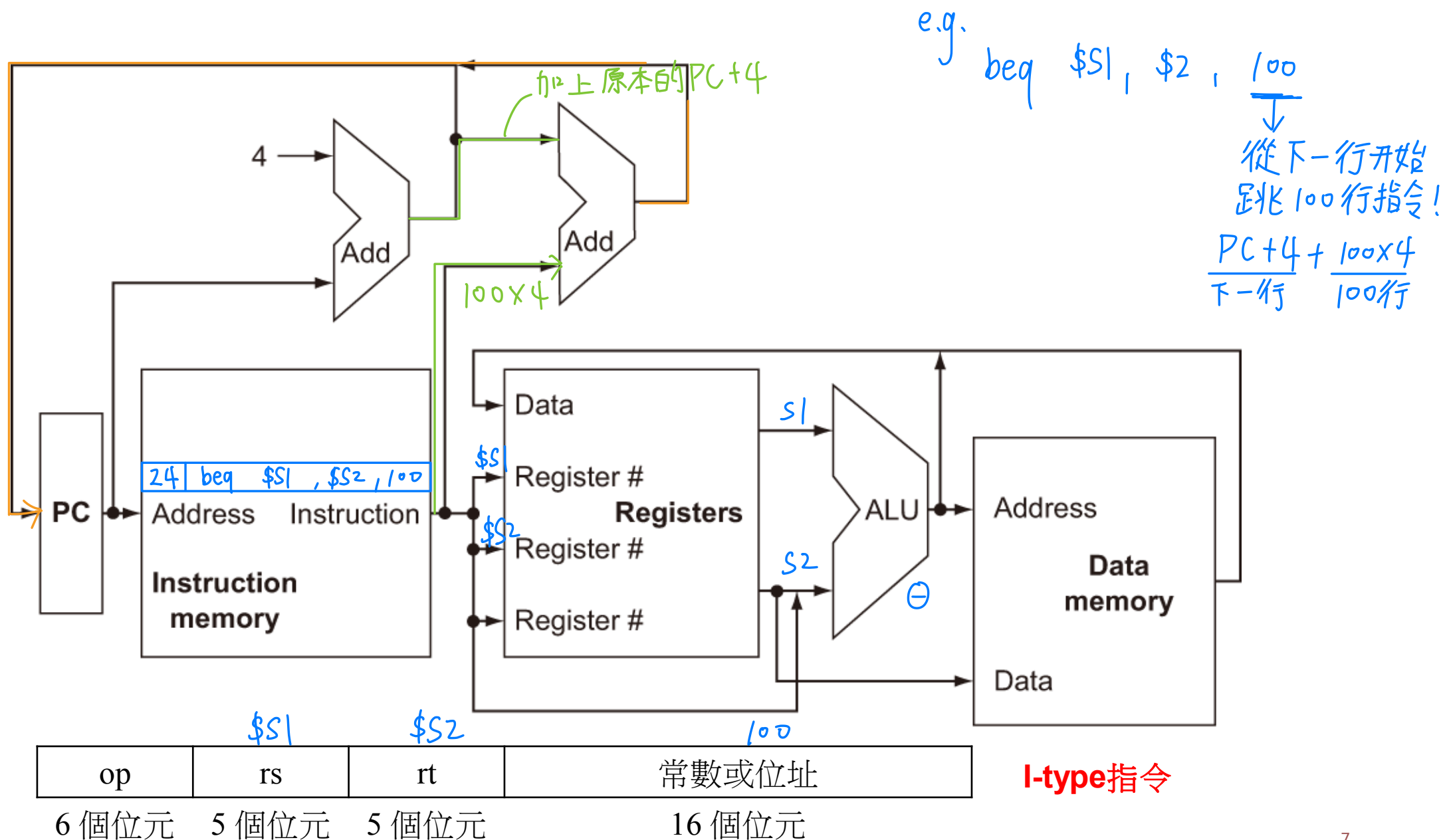
實作的概觀

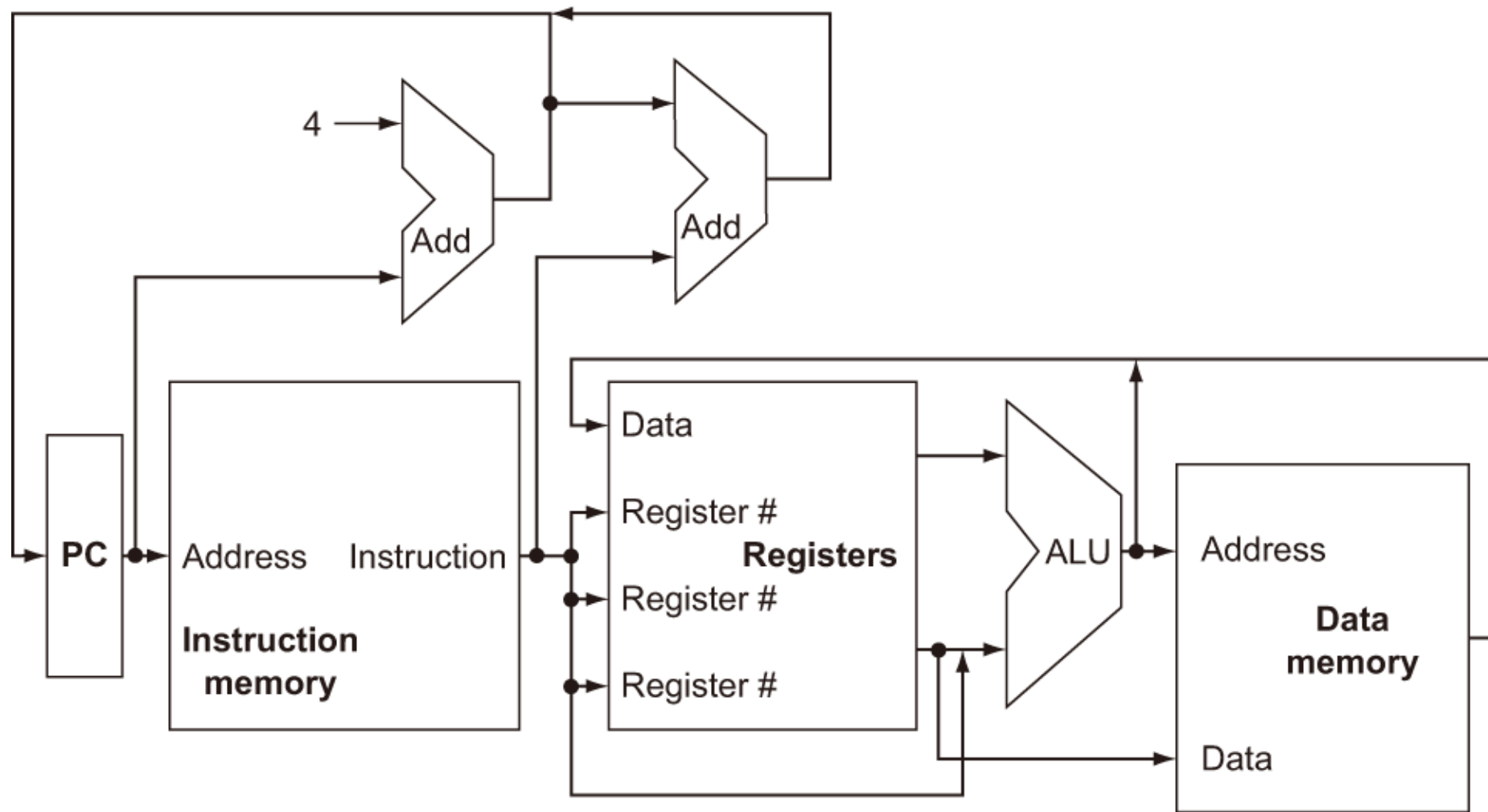
- 在這兩個步驟之後，用來完成指令的動作與指令類別有關
 - 三種指令類別(記憶體存取、算術邏輯及分支)
- ▣ 圖4.1 以高階的觀點顯示MIPS 中不同的功能單元和指令類別之間的聯結的實作圖

圖4.1 MIPS 指令集合實作的概觀圖，圖中表示了各主要功能單元和它們之間的主要連線關係









2	10000
---	-------

6 個位元

26 個位元

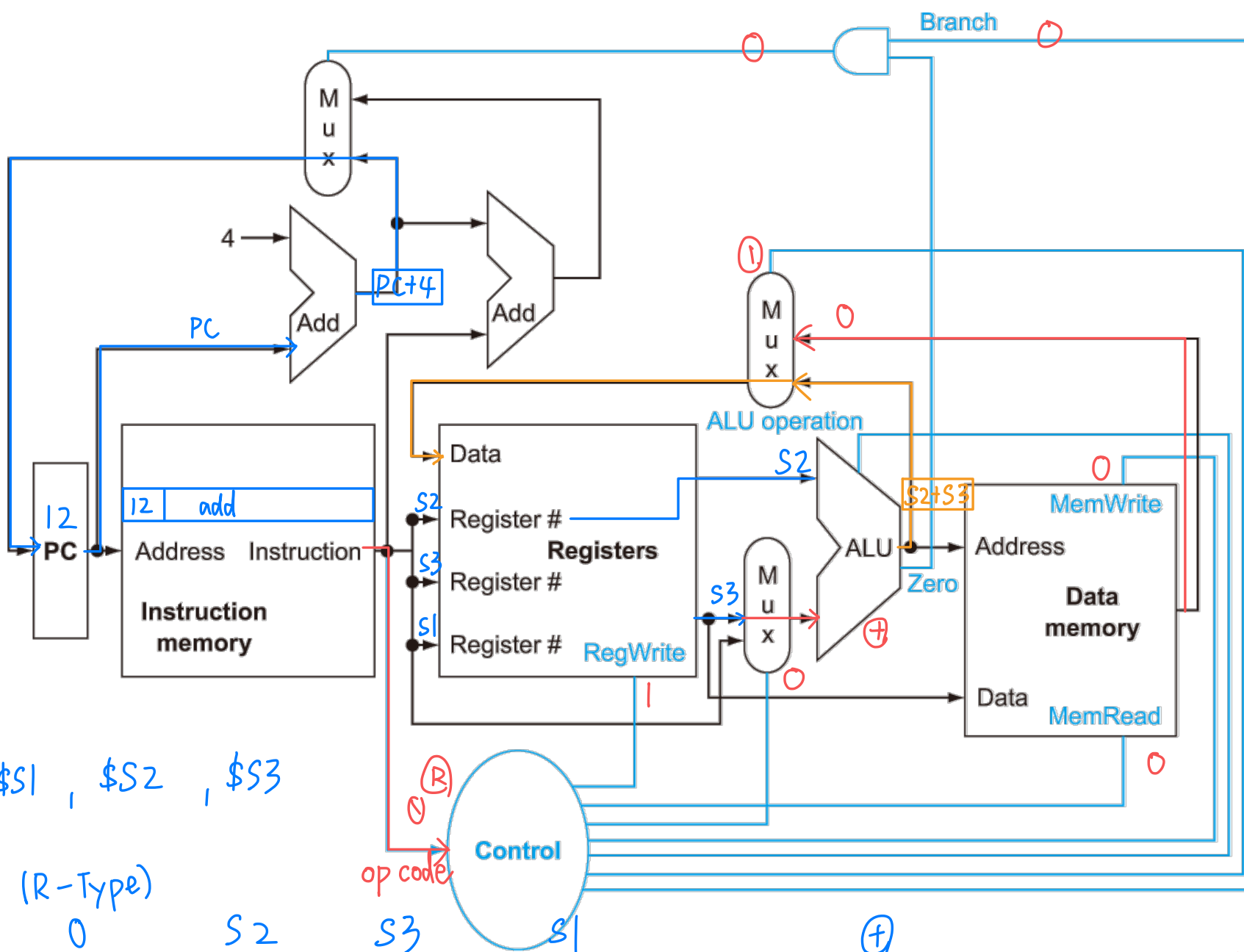
J-type指令

圖4.1 MIPS 指令集合實作的概觀圖，圖中表示了各主要功能單元和它們之間的主要連線關係

所有指令都是由使用程式計數器(PC)來提供指令位址給指令記憶體開始。在指令擷取進來之後，指令中的欄位指出指令所使用到的暫存器運算元。一旦暫存器運算元被擷取完成，則這些運算元可以被用來計算記憶體的位址(用於載入或儲存指令)、計算算術結果(用於整數的算術邏輯指令)或是比較(用於條件分支指令)。如果指令是算術邏輯指令，則必須將ALU 的運算結果寫到暫存器中。如果是載入或儲存指令，則ALU 的結果被當成記憶體位址來儲存一個暫存器的值到記憶體中或是從記憶體載入一個值到暫存器中。從ALU 或記憶體得到的結果會被寫回暫存器檔案。條件分支指令使用ALU 的輸出來決定下一道指令的位址，這個位址要不是來自一個加法器(將目前的PC 值+4和條件分支的偏移量相加)就是來自另一個加法器的結果(將目前的PC 值累加4)。將各功能單元相連的粗體線條代表匯流排(bus)，其包含多條訊號線。使用箭頭讓讀者瞭解資料的流向。因為訊號線有可能會相互交錯，所以我在訊號線交錯的地方使用黑點來表示交錯的訊號線是相連的。

實作的概觀

- 圖4.2 顯示圖4.1 加上三個所需的多工器以及主要功能單元的控制訊號後的數據通道
 - 每一道指令從一個時脈的邊緣開始執行並且於下一個時脈的邊緣完成
 - 這樣的設計方式並不實際 (有些指令很慢, 所以快的會跟著變慢)
 - 因為時脈必須被拉長以容納最慢的指令



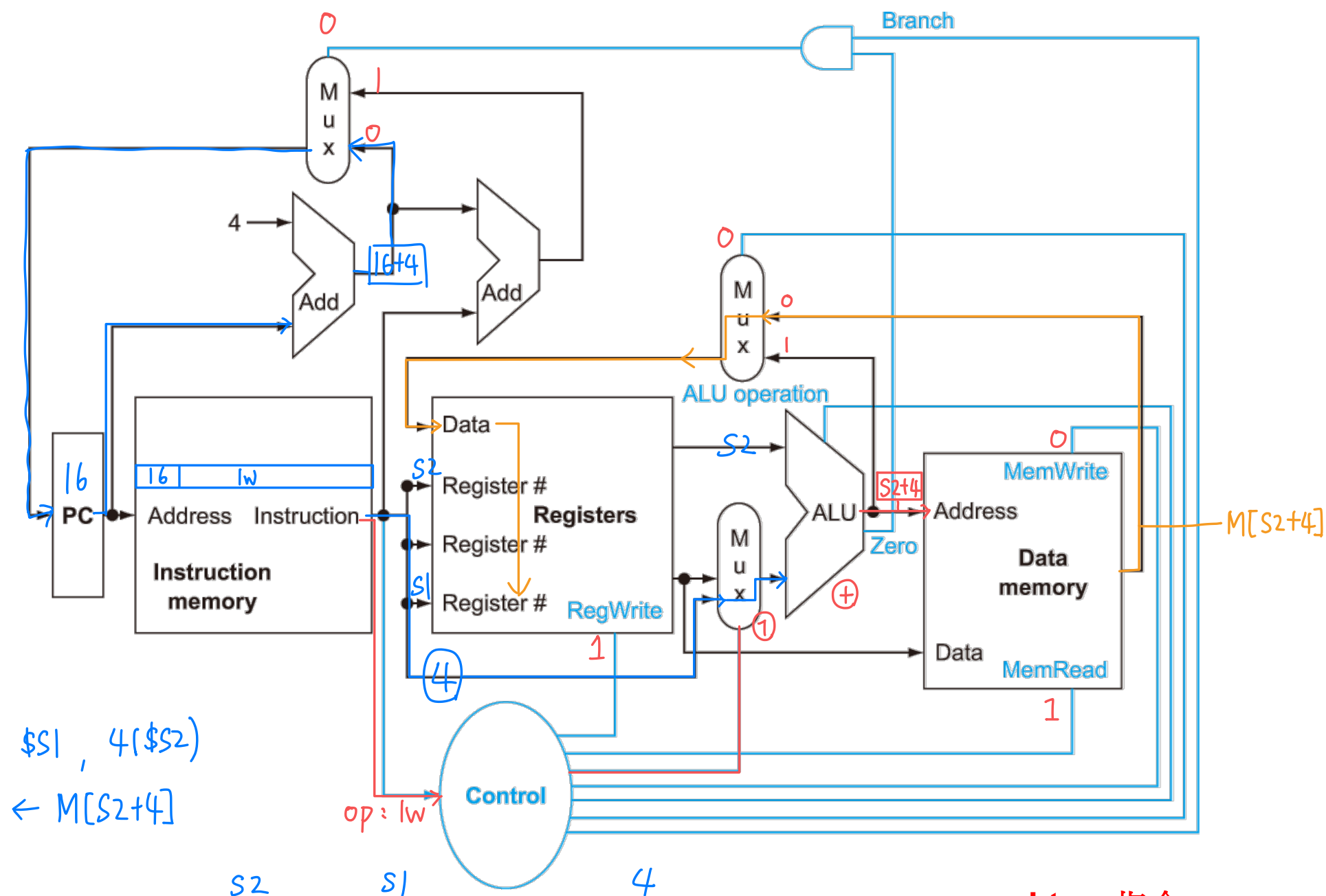
e.g. add \$s1, \$s2, \$s3

(R-Type)

0	s2	s3	s1		(+)
op	rs	rt	rd	Shamt	funct

6 個位元 5 個位元 5 個位元 5 個位元 5 個位元 6 個位元

R-type指令



e.g. $lw \$s1, 4(\$s2)$
 $s1 \leftarrow M[s2+4]$

op	rs	rt	常數或位址
----	----	----	-------

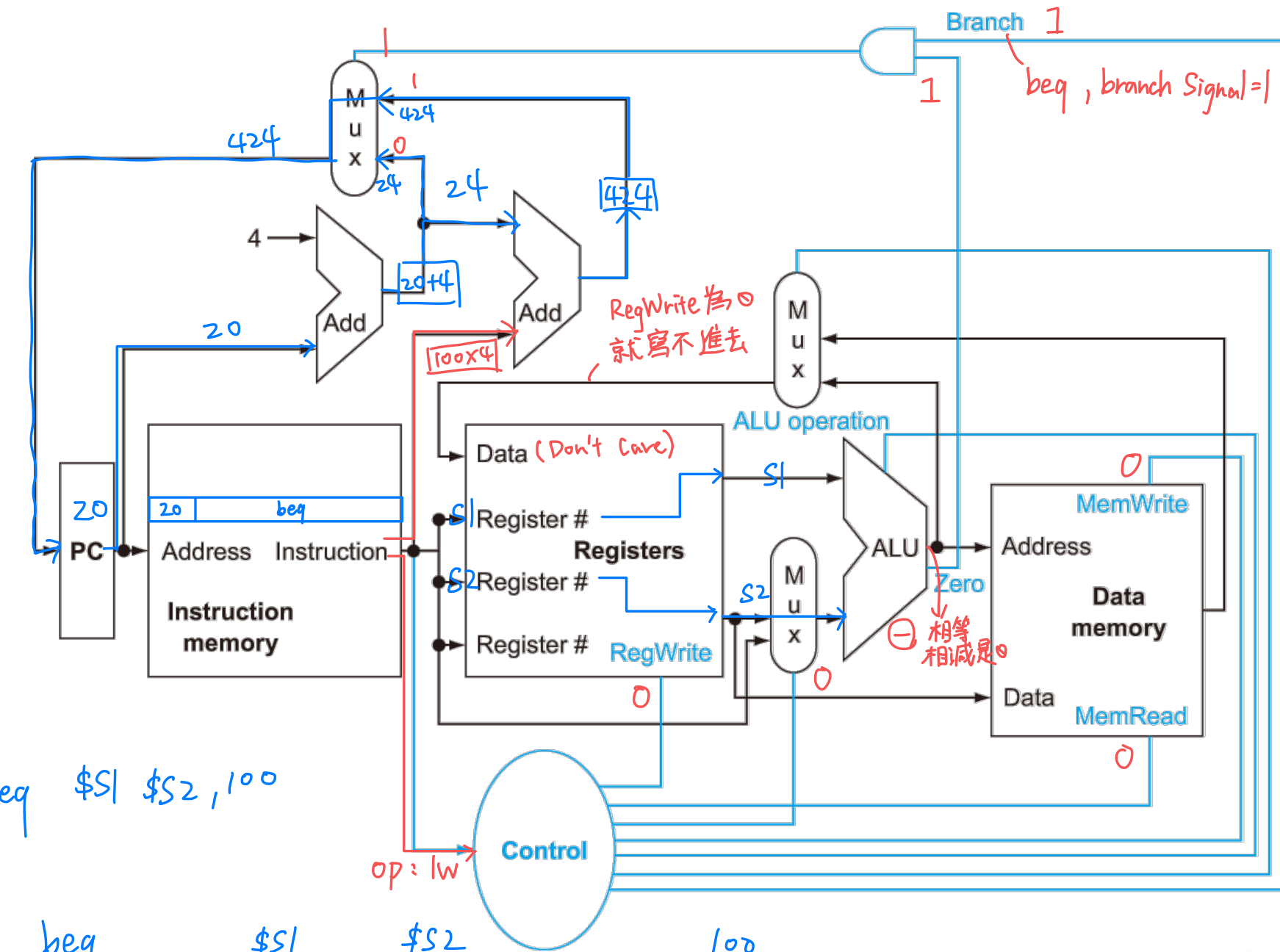
6 個位元

5 個位元

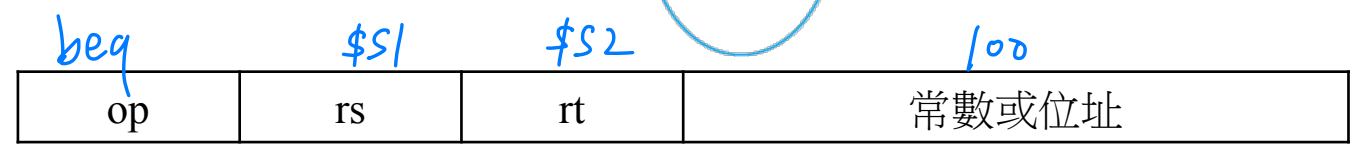
5 個位元

16 個位元

I-type指令

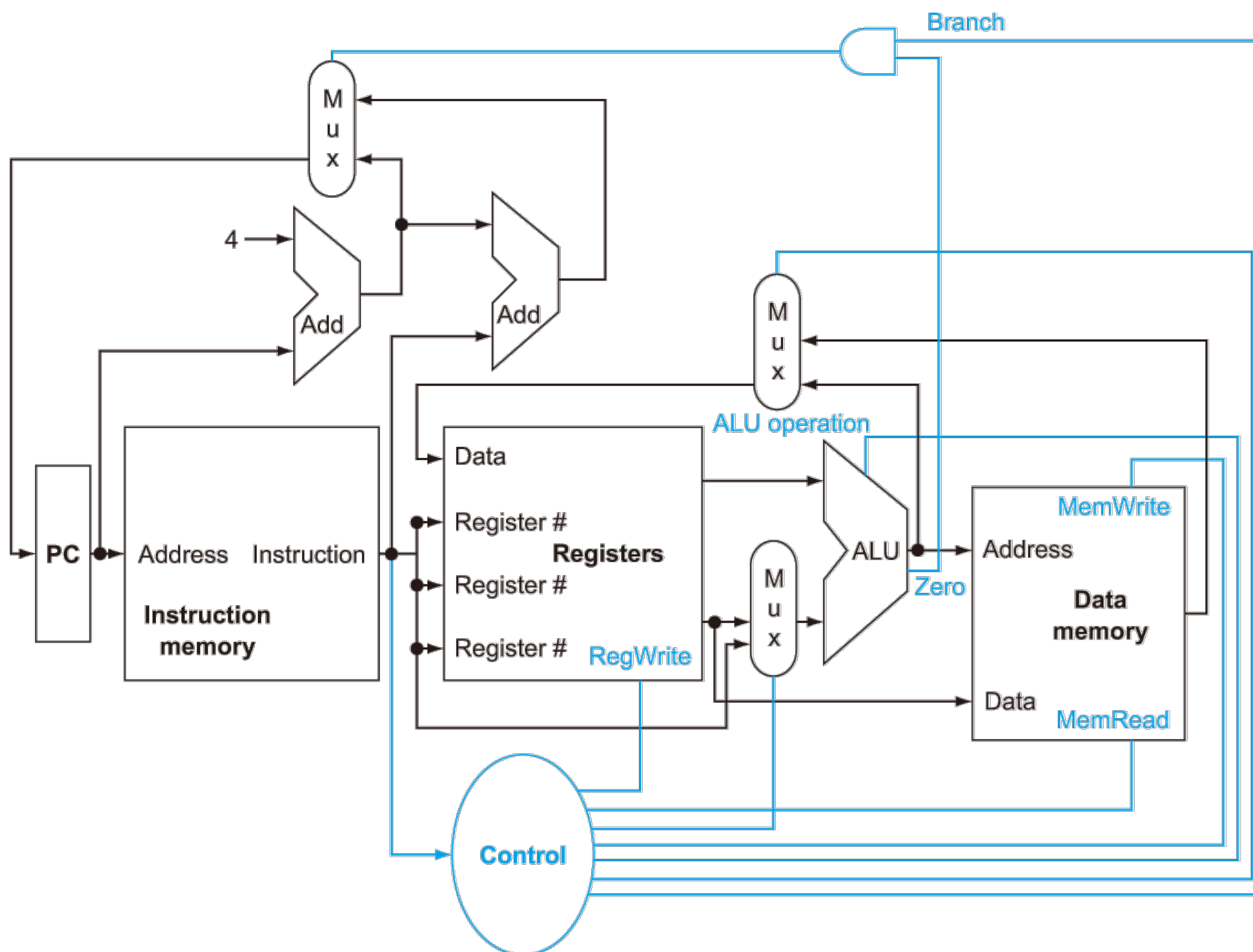


e.g. `beq $s1 $s2, 100`



I-type指令

6 個位元 5 個位元 5 個位元 16 個位元



2	10000
---	-------

6 個位元

26 個位元

J-type指令

圖4.2 MIPS 子集合的基本實作，包括所需的多工器和控制訊號線

最上面的多工器控制**PC**的內容由哪一個值取代(**PC+4** 或是條件分支目的位址)；這一個多工器是由兩條訊號線經過**AND** 閘後的結果來控制的，而這兩條訊號線是**ALU** 的**Zero** 輸出訊號線以及指出現在執行的指令是分支指令的控制訊號線。圖中間的多工器用來選擇是由**ALU** 的輸出(在算術邏輯指令的情況下)或是數據記憶體輸出(在載入指令的情況下)寫回暫存器檔案中。最後，最下面的多工器用來決定**ALU** 的第二個輸入是來自暫存器(在算術邏輯指令或分支的情況下)或是指令的偏移量欄位(在載入或儲存指令的情況下)。新增的控制訊號線淺而易懂，用來決定**ALU** 要執行的動作、數據記憶體應該被讀取或寫入、或是暫存器要不要執行寫入的動作。我們將控制訊號線的部分著灰色，讓它們更容易被辨識

4.2 邏輯設計慣例

組合 logic

序項 logic

■ 數據通道元件有兩種不同形式的邏輯元件：數值運算元件和狀態儲存元件

ALU、Adder

- 數值運算元件全然屬於組合(combination)邏輯
 - ALU 就是一個組合邏輯元件的例子
- 狀態元件(state elements)可以儲存狀態
 - 這些狀態元件完全地說明了計算機的現況
 - 指令記憶體、數據記憶體以及暫存器都是屬於狀態元件
 - 必要的輸入包括要寫入元件的數據內容以及決定何時將數據寫入的時脈，輸出則是在其之前被寫入的值
 - 因為它們的輸出與它們的輸入以及內部狀態均有關而稱之為序向的(sequential)

時脈致動方法

—— 用 clock 決定他完成一件事

- 時脈致動方法(clocking methodology)定義了訊號何時可以被讀取以及何時可以被寫入
 - 設計來保證事情的可預期性
 - 為了簡單起見，採用邊緣觸發(edge-triggered)的時脈致動方法
- 圖4.3 顯示了兩個狀態元件包圍著一個組合邏輯的區塊
 - 該組合邏輯區塊應在一個時脈週期內運作完成

每個 clock 來時才會把 data 抓進去

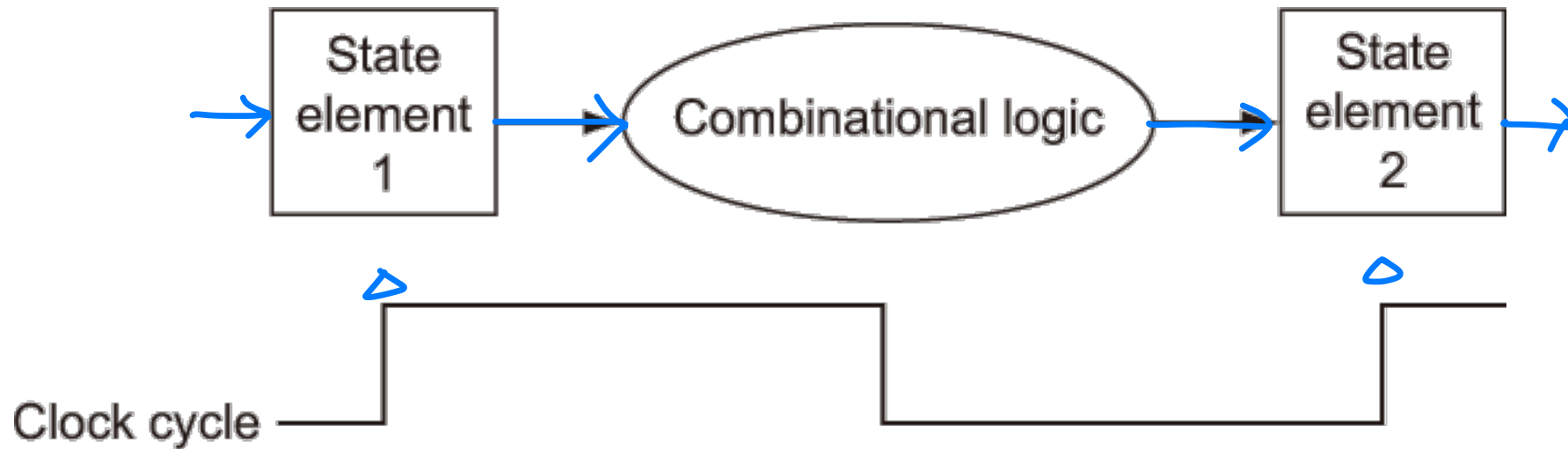


圖4.3 組合邏輯、狀態元件和時脈之間關係密切

在一個同步的數位系統中，時脈決定了狀態元件何時將值寫入內部的儲存體。任何狀態元件的輸入值在致動的時脈邊緣造成狀態被更新之前，都必須是穩定的(也就是說，到達的值必須一直維持穩定不變到時脈邊緣之後)。本章中所有的狀態元件(包括記憶體)都假設是邊緣觸發的

時脈致動方法

- 邊緣觸發的方法使得我們可以在同一個時脈週期中讀取暫存器的內容，之後對這個值做組合邏輯運算，再寫回原本的暫存器中
 - 圖4.4為一個通則性的例子
 - 採用邊緣觸發的時序方法時，在單一時脈週期內不會發生反饋(feedback)的情況

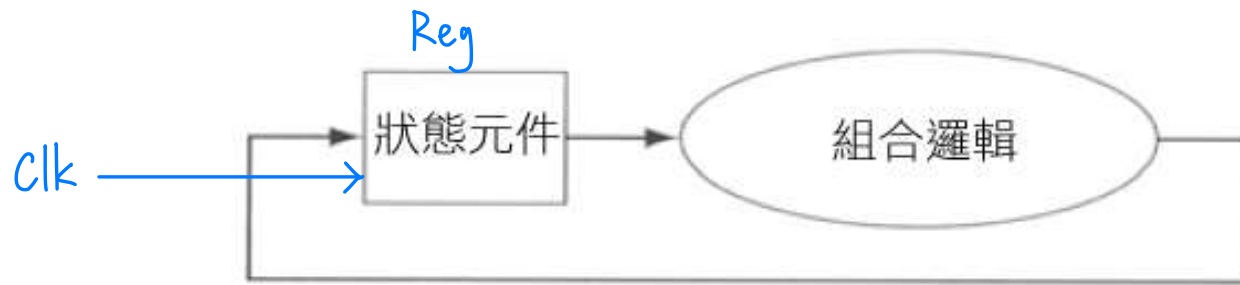


圖4.4 邊緣觸發的方法允許狀態元件同時在一個時脈週期內被讀取和寫入，而不會造成可能會產生不確定值的競賽(**race**)情況

當然，時脈週期仍然必須夠長到當致動時脈邊緣發生時，輸入的值已經穩定。因為狀態元件的更新是邊緣觸發的，所以時脈週期內並不會發生反饋的情況。如果反饋的情況可能發生的話，這樣的設計就無法正確地運作。在本章和下一章中的設計都是依靠如本圖所示的邊緣觸發方法及相同的結構

4.3 建構數據通道

- 圖 4.6 表示如何將圖 4.5 中的三個元件組合成一個可以擷取指令並且遞增PC 值以得知下一個連續指令位址的數據通道

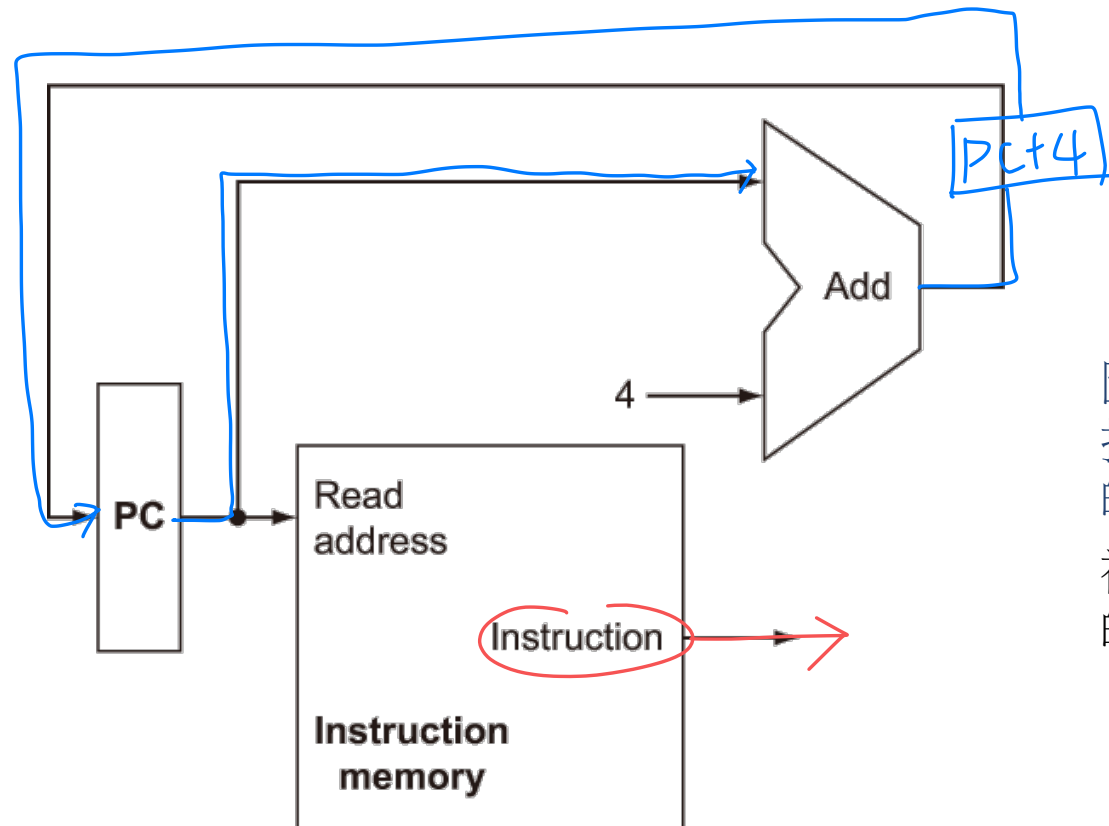


圖4.6 數據通道中用來擷取指令和遞增程式計數器(PC)的部分
被擷取的指令會被數據通道的其他部分使用

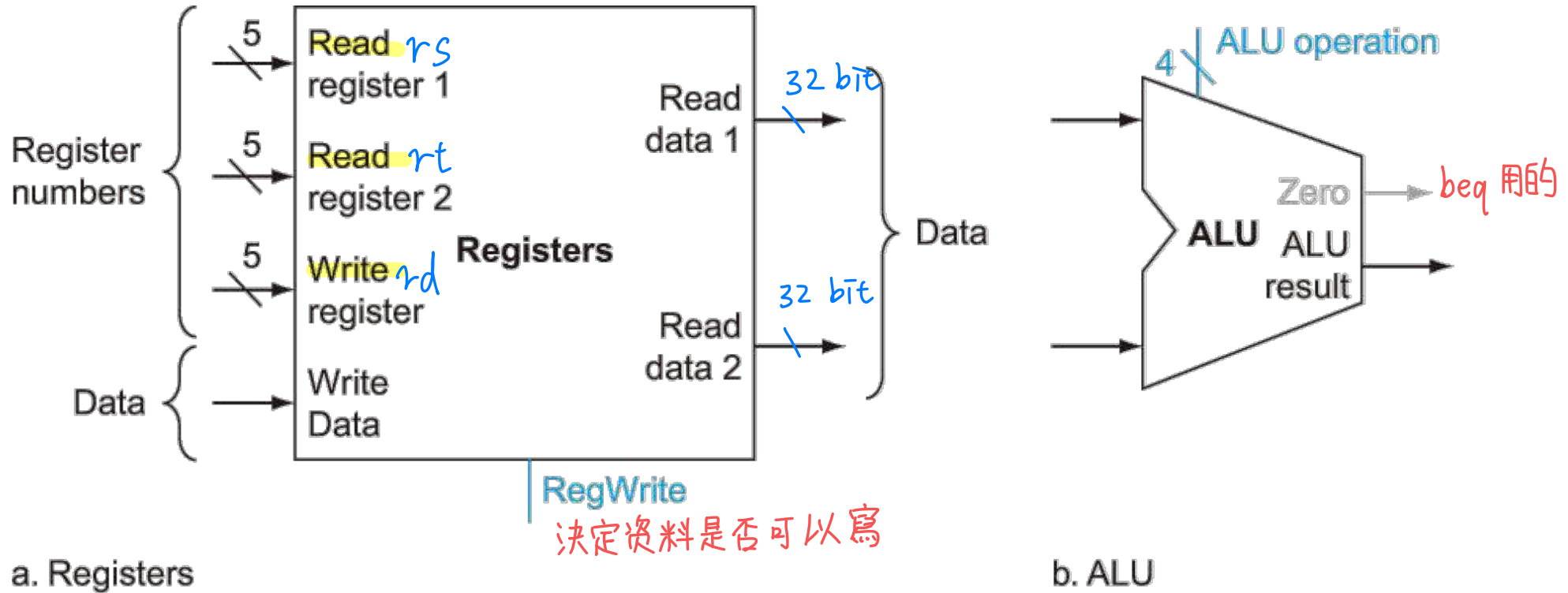
■ 思考 R 格式的指令(見圖2.18)

e.g. $rd \leftarrow rs + rt$

圖2.18 MIPS 指令格式

名稱	欄 位						註 解
欄位大小	6 位元	5 位元	5 位元	5 位元	5 位元	6 位元	所有 MIPS 指令的長度都是 32 位元
R 格式	op	rs	rt	rd	shamt	funct	算術指令格式
I 格式	op	rs	rt	位址／立即值			傳輸、分支、imm 格式
J 格式	op	目標位址					跳躍指令格式

圖4.7 實作R 格式的ALU 運作所需的兩個元件是暫存器檔案和ALU



op	rs	rt	rd	Shamt	funct
6 個位元	5 個位元	5 個位元	5 個位元	5 個位元	6 個位元

圖4.7 實作R 格式的ALU 運作所需的兩個元件是暫存器檔案和ALU

暫存器檔案包含了所有的暫存器並且有兩個讀取埠和一個寫入埠。暫存器檔案會一直將對應於**Read register**輸入埠的暫存器內容輸出，而不需要其他的控制訊號。相對地，暫存器的寫入就必須要明確地由寫入控制訊號來控制。記得寫入的動作屬於邊緣觸發，所以所有和寫入動作相關的輸入訊號(即被寫入的值、暫存器編號和寫入控制訊號)都必須在時脈邊緣時為有效。因為暫存器檔案的寫入動作屬於邊緣觸發，所以這種設計可以在一個時脈週期內對同一個暫存器讀取和寫入：讀取到的值是稍早的時脈週期所寫入的值，而寫入的值則可以在之後的時脈週期被讀取。暫存器檔案的暫存器編號輸入訊號都是**5** 個位元寬，而運送資料的訊號線都是**32** 個位元寬。要執行的**ALU** 運作是由**4** 位元**ALU** 運作控制訊號線來控制。我們稍後會使用**ALU** 的**Zero** 偵測輸出來實作條件分支(**branches**)。而**ALU** 的滿溢(**overflow**)輸出一直到**4.9** 節討論例外(**exceptions**)之前都還不需要，所以目前我們先忽略它

4.3 建構數據通道

I 格式的指令

- MIPS 的載入字組和儲存字組指令
- 若相等則分支(beq)指令

lw
sw
beq
addi
⋮

e.g. lw \$S1, 4(\$S2)
rt rs
write read

sw \$S1, 4(\$S2)
M[S2+4] ← S1
rs rt
write read

圖2.18 MIPS 指令格式 I-type 的 rs、rt 所表的功能不一定一樣

名稱	欄 位						註 解
欄位大小	6 位元	5 位元	5 位元	5 位元	5 位元	6 位元	所有 MIPS 指令的長度都是 32 位元
R 格式	op	rs	rt	rd	shamt	funct	算術指令格式
I 格式	op	rs	rt	位址／立即值			傳輸、分支、imm 格式
J 格式	op	目標位址					跳躍指令格式

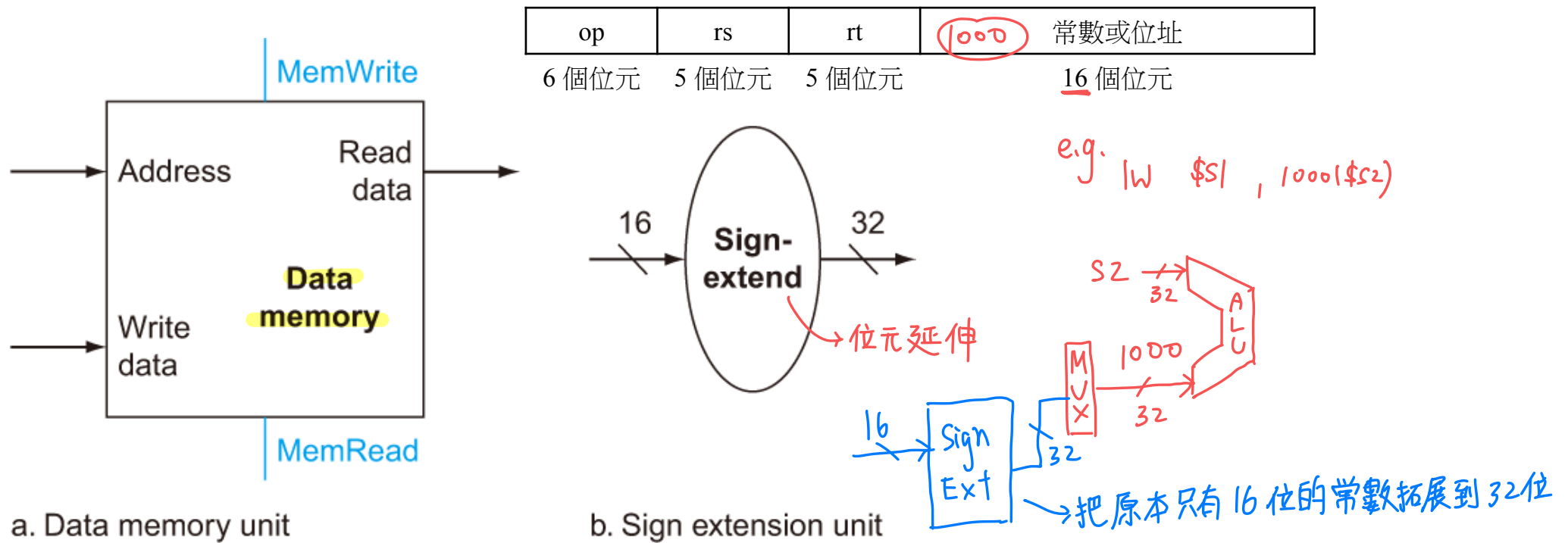


圖4.8 除了圖4.7 的暫存器檔案和ALU 外，實作載入指令和儲存指令還需要的兩個元件是數據記憶體單元和符號延伸單元

數據記憶體單元是一個具有位址輸入、寫入資料輸入和一個讀取結果輸出的狀態元件。它有個別的讀取和寫入控制訊號，即便在任何一個時脈中只有一條可能會被設定。記憶體單元需要讀取控制訊號，因為它並不像暫存器檔案，如果讀取一個無效位址的內容可能造成一些問題，如我們將會在第五章中所見。符號延伸單元具有一個 16 位元的輸入，經符號延伸成 32 位元的結果送到輸出上(見第二章)。我們假設數據記憶體的寫入屬於邊緣觸發。實際上標準的記憶體晶片會有一個寫入致能(write enable)訊號來控制寫入。儘管寫入致能並不屬於邊緣觸發，我們的邊緣觸發設計可以很容易地套用於真實記憶體晶片

圖4.9 分支指令的數據通道使用ALU 來評定分支條件以及加法器將遞增後的PC 值加上指令的最低16 個位元(分支位移量)經過符號延伸再左移2 個位元的值來計算分支目標位址

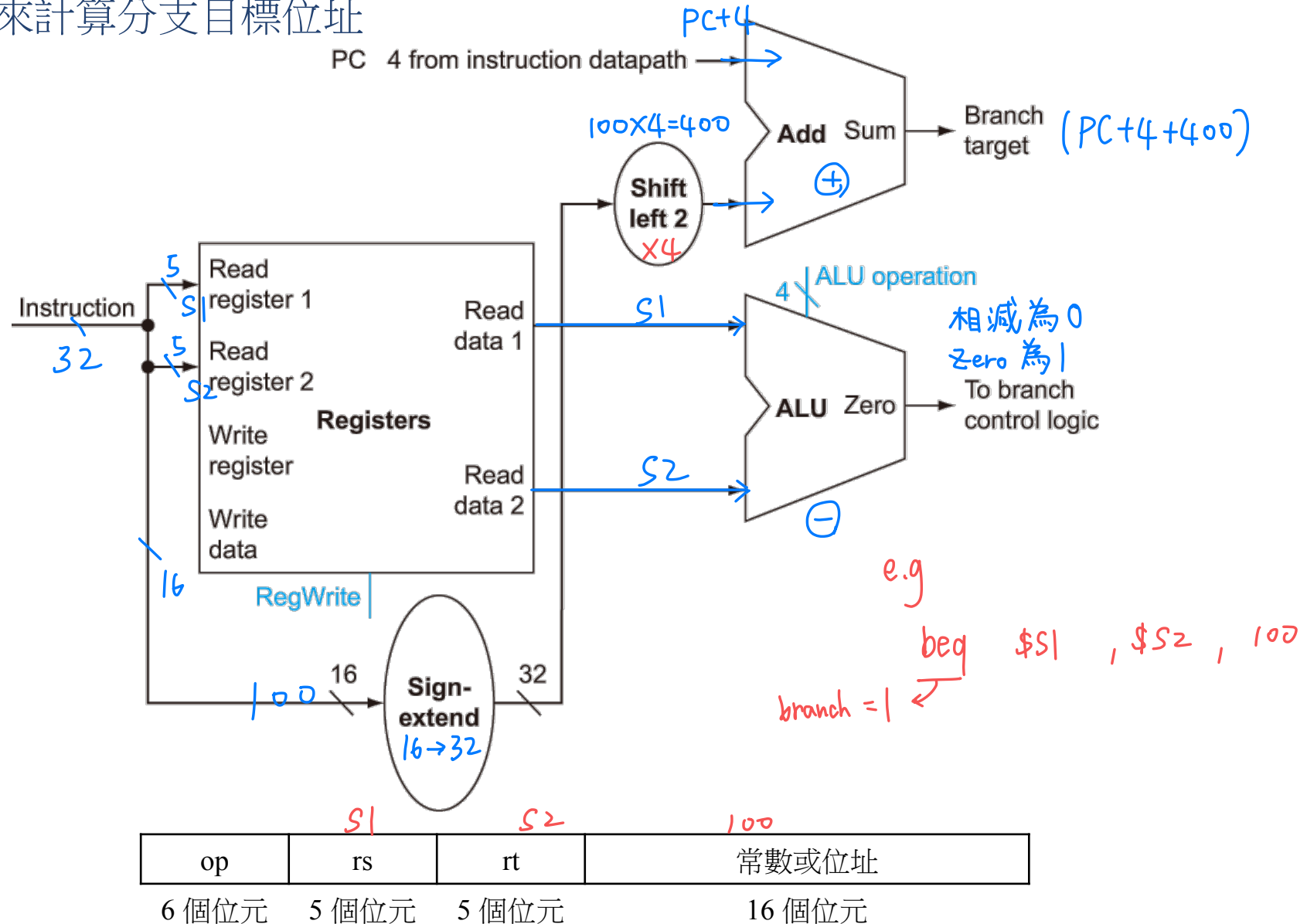


圖4.9 分支指令的數據通道使用**ALU** 來評定分支條件以及加法器將遞增後的**PC** 值加上指令的最低**16** 個位元(分支位移量)經過符號延伸再左移**2** 個位元的值來計算分支目標位址

標記為「左移**2** 位元」的單元其實只是將輸入和輸出之間的訊號線做簡單的繞線，在符號延伸後的偏移量欄位最右方加上**002**；因為位移的數目是常數，所以並不需要真的位移硬體。因為我們知道這個偏移量是由**16** 位元的資料符號延伸而來，所以位移的動作只會把代表正負號的位元扔掉。控制邏輯根據**ALU** 的**Zero**輸出訊號決定將**PC** 值代以遞增後的**PC** 值或是分支目標位址

做出一個數據通道

- ▣ 這個最簡單的數據通道執行每個指令都只需一個時脈週期的時間
 - 代表在執行每道指令時所有數據通道的資源都至多只能被使用一次
 - 還是有許多元件可以被不同的指令共用
 - 一個硬體一時間只給一指令用！
 - 很多指令都會用到相同硬體，但一時間(clock)下
- ▣ 為了讓兩個不同種類的指令共用數據通道中的元件
 - 可能需要將多個來源連接到元件的輸入，並使用多工器和控制訊號來選擇不同的輸入

做出一個數據通道

▣ MIPS架構的簡單數據通道：圖4.11

- 之後即可開始加入控制單元
- 控制單元必須可以接受輸入，並且產生每個狀態元件的寫入控制訊號、每個多工器的選擇控制訊號和ALU 的控制訊號

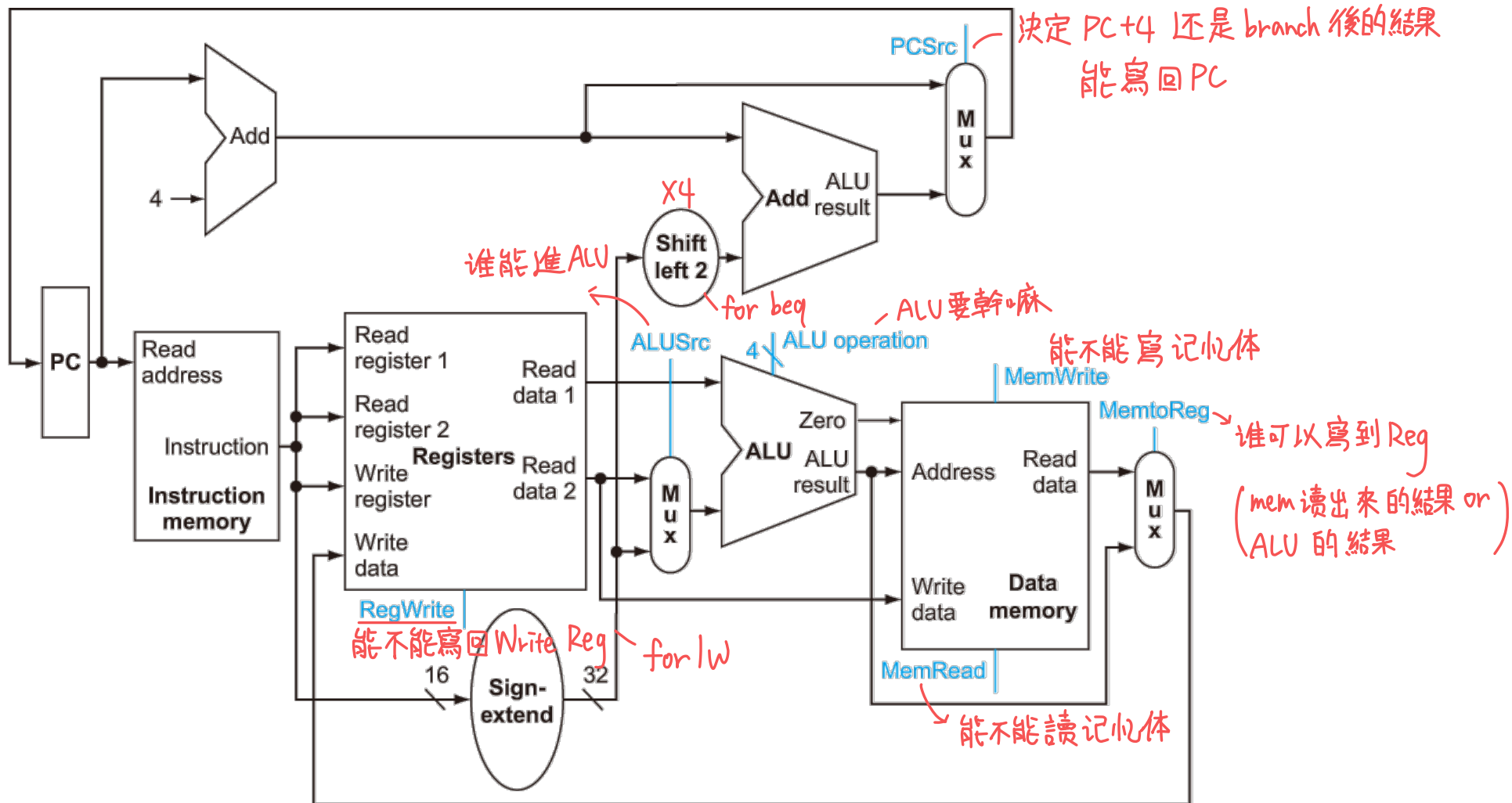


圖4.11 結合不同類別指令所需元件的MIPS 架構簡單數據通道

這些組件來自圖4.6、4.9 與4.10。這個數據通道可以在單一時脈週期中執行基本的指令(載入/儲存字組、算術邏輯運作及分支)。我們需要額外的一個多工器來整合入分支。至於跳躍指令的部分將會在稍後加入

4.4 簡單實作方案

- ▣ 可處理載入字組(lw) 、儲存字組(sw) 、相等時分支(beq)及算術邏輯指令 add 、 sub 、 AND 、 OR 及set on less than
- ▣ 再處理跳躍指令(j)

ALU 的控制

ALU 控制線	功 能
0000	AND
0001	OR
0010	add
0110	subtract
0111	set on less than
1100	NOR

ALU 的控制

e.g $lw \$s1, 4(\$s2) \quad S1 \leftarrow M[S2+4]$
 $sw \$s1, 8(\$s3) \quad M[S2+8] \leftarrow S1$

00 (lw, sw) $\square$ 對於載入字組及儲存字組指令，使用ALU做加法來計算記憶體的位置

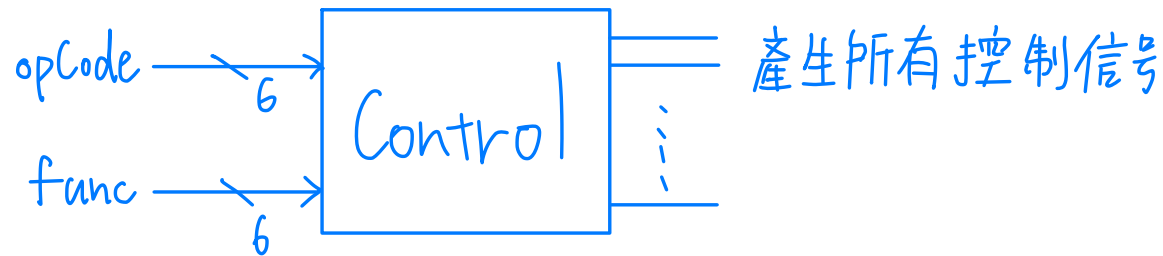
01 (R-type) $\square$ 對於R型指令，ALU則需根據指令中最低6個位元功能(func)欄位的值來決定執行五種動作 (AND、OR、subtract、add或set on less than)之一(見第二章)

10 (beq) $\square$ 對於相等時分支ALU則必須做減法

↓
先把op從6 bit轉成2 bit
Control 設計較簡單

ALU 的控制

- 使用一個小控制單元並以指令的功能(func)欄位以及稱之為ALUOp 的2 位元的控制欄位作為輸入
 - ALUOp 表示要執行的運算是載入與儲存的加法(00)、beq 的減法(01)或是由指令的功能欄位所編碼的動作來決定(10)



ALU 的控制

當ALUOp為00、01時，func為don't Care

指令運作碼	ALUOp	指令運作	功能欄位	需要的 ALU 動作	ALU 控制輸入
LW	00	載入字組	XXXXXX	加	0010
SW	00	儲存字組	XXXXXX	加	0010
Branch equal	01	若等於則分支	XXXXXX	減	0110
R-型	10	加	1 00000	加	0010
R-型	10	減	1 00010	減	0110
R-型	10	及	1 00100	及	0000
R-型	10	或	1 00101	或	0001
R-型	10	若小於則設定	1 01010	若小於則設定	0111

看後四位
即可區隔

圖4.12 ALU 控制訊號如何設定視 ALUOp 控制位元及R-型指令的不同功能碼而定

第一行所列出的運作碼決定了第二行的 ALUOp 位元。所有的編碼都以二進位來表示。注意當 ALUOp 的值為 00 或 01 時，所需的 ALU 動作和功能欄位的值無關；所以我們使用「XXXXXX」來表示我們對功能欄位的值並無所謂(don't care)。如果ALUOp 的值為 10 時，則 ALU 控制輸入的值會依功能欄位而定

ALU 的控制

$$p = \text{ALUOp1} \cdot F3' \cdot F2 \cdot F1' \cdot F0 + \text{ALUOp3} \cdot F3 \cdot F2' \cdot F1 \cdot F0'$$

ALUOp		功能欄位						運作(ALU 控制訊號)
ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0	
0	0	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	0110
1	X	X	X	0	0	0	0	0010
1	X	X	X	0	0	1	0	0110
1	X	X	X	0	1	0	0	0000
1	X	X	X	0	1	0	1	0001
1	X	X	X	1	0	1	0	0111

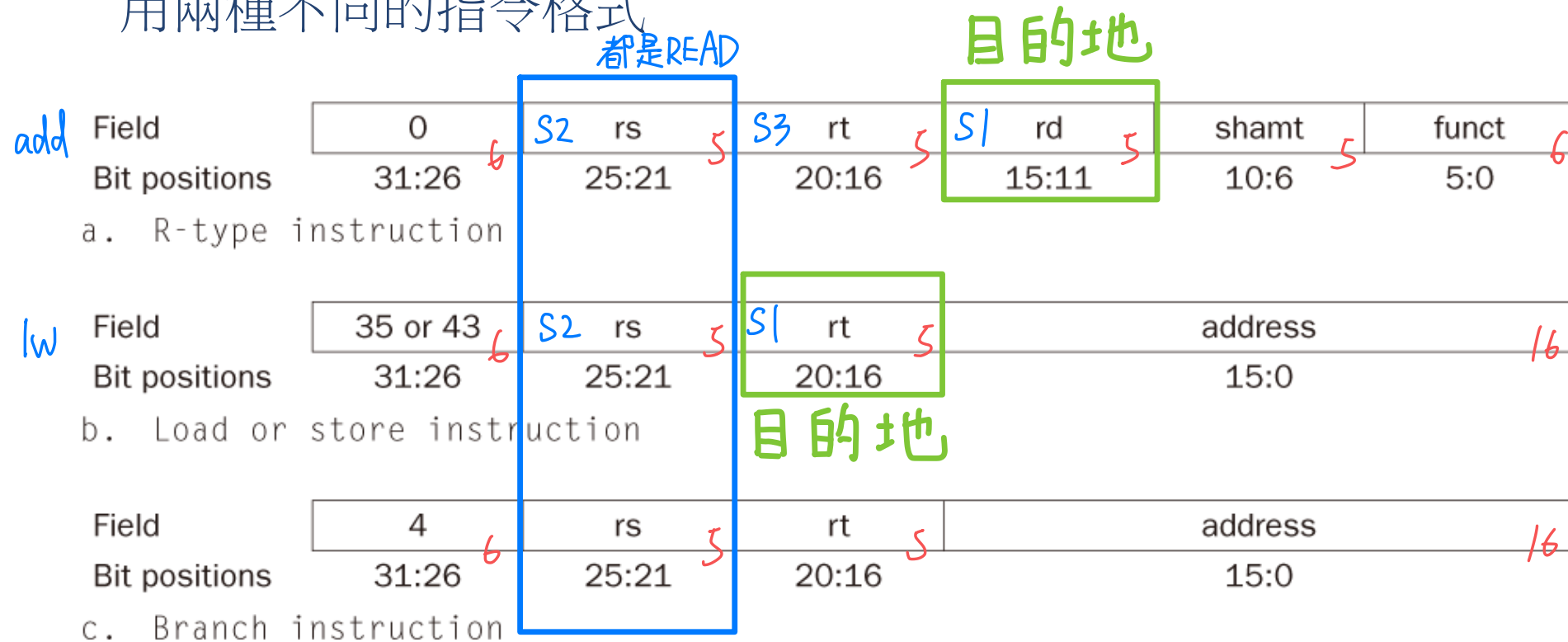
圖4.13 4 位元 ALU 控制訊號(稱為運作(operation))的真值表

輸入部分包含 ALUOp 及功能碼欄位。表中只顯示 ALU 控制被設定的項目。一些「無所謂」的項目也已經表出。例如，ALUOp 並沒有使用 11 的編碼，所以真值表可以使用 1X 和 X1，而不必是 10 和 01。注意當功能欄位被使用時，這些指令前兩個位元(F5 和 F4)的值總是等於 10，所以它們屬於「無所謂」的項目並且在真值表中以 XX 來表示

■ 圖4.14 表示指令的格式

- Op 欄位，Op[5 : 0] 來表示這個欄位
- 在 R 型指令、相等則分支(beq)指令及儲存指令中，被讀取的暫存器總是以位於指令的25 : 21 位元及20 : 16 位元的rs 欄位及 rt 欄位來指出
- 載入及儲存指令中的基底暫存器總是在指令的25 : 21 位元(rs)
- 相等則分支(beq)指令、載入指令及儲存指令的16 位元偏移量(offset)總是在指令的15 : 0 位元
- 目的暫存器有可能出現在兩個地方，20 : 16位元(rt)或15 : 11 位元(rd)，必須加上一個多工器來選擇rt 或是rd 欄位

圖4.14 三種指令類別(R-型指令、載入/儲存指令及分支指令)使用兩種不同的指令格式



e.g.

→ add $\$S1, \$S2, \$S3$ ($S1 \leftarrow S2 + S3$)

→ lw $\$S1, 4(\$S2)$ ($S1 \leftarrow M[S2+4]$)

圖4.14 三種指令類別(R-型指令、載入/儲存指令及分支指令)使用兩種不同的指令格式

跳躍指令(jump)使用另外一種格式於之後討論。

(a)R-型指令的指令格式，其運作碼(opcode)均為零。這些指令有三個暫存器的運算元：**rs**、**rt** 及 **rd**。**rs** 和 **rt** 欄位為來源(source)暫存器，**rd** 欄位為目的(destination)暫存器。**ALU** 的功能是利用前一節所設計的**ALU** 控制單元來對功能(funcnt)欄位解碼所決定。我們實作的R-型指令有**add**、**sub**、**and**、**or** 及 **slt**。位移量(shamt)欄位僅使用在移位指令中；我們在本章中先忽略它。

(b)載入指令(opcode=35₁₀)和儲存指令(opcode=43₁₀)的指令格式。**rs** 暫存器為基底暫存器(base register)，其值與16 位元位址欄位的值相加後得到記憶體有位址。在載入指令中，**rt** 暫存器是載入值的目的暫存器。在儲存指令中，**rt** 暫存器是來源暫存器，其值要儲存到記憶體。(c)相等時分支指令(opcode = 4₁₀) 的指令格式。**rs** 和 **rt** 暫存器是用於比較是否相等的來源暫存器。而16 位元的位址欄位會被符號延伸、移位後和**PC+4** 值相加來計算分支目標位址

設計主要的控制單元

- 圖4.15 顯示上述新增以及ALU 控制區塊、狀態元件寫入控制訊號、數據記憶體의讀取訊號及多工器的控制訊號

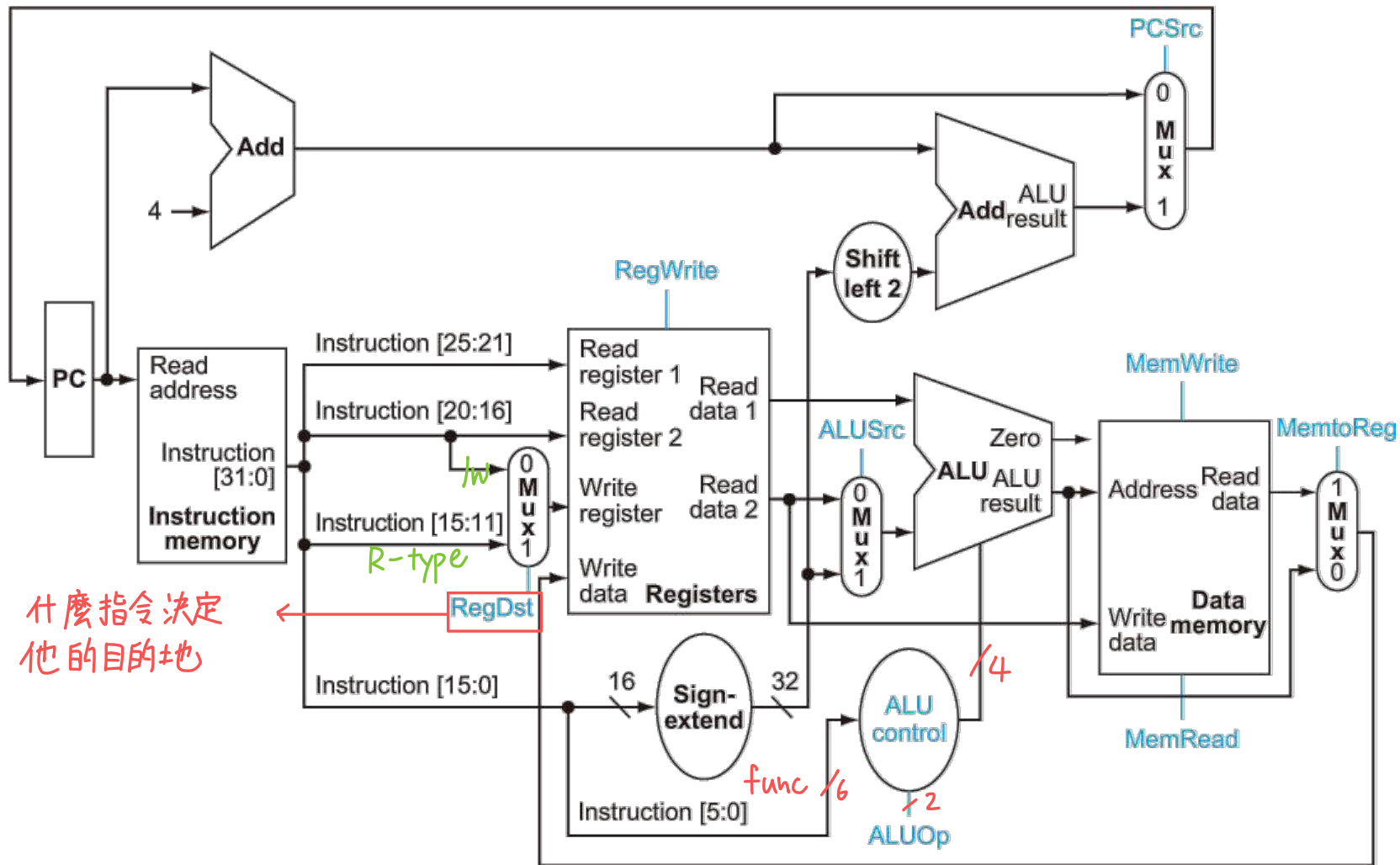


圖4.15 圖4.12 的數據通道以及所有需要的多工器及控制訊號

灰色的線表示控制訊號。ALU 控制區塊也已經加入圖中。因為PC 暫存器在每個時脈週期都會寫入一次，所以不需要寫入控制；分支控制邏輯決定寫入PC；暫存器的值是遞增後的PC 或是分支目標位址

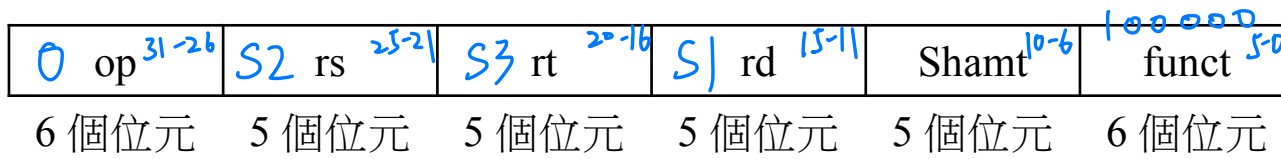
設計主要的控制單元

- ▣ 圖4.16 七條控制訊號的功能
- ▣ 在二對一多工器的1 位元控制訊號被設定(asserted)時，多工器選擇1 號輸入埠的值。否則當控制訊號未被設定(deasserted)時，多工器選擇 0 號輸入埠的值。記得所有的狀態元件都隱含了時脈輸入並使用時脈來控制寫入。在狀態元件之外先對時脈訊號進行邏輯運算可能會造成時序的問題

設計主要的控制單元

圖4.16 七條控制訊號每一條的效果

訊號名稱	未被設定時的效果	被設定時的效果
RegDst	Write register 目的暫存器編號來自 rt 欄位(指令的 20:16 位元)。	Write register 目的暫存器編號來自 rd 欄位(指令的 15:11 位元)。
RegWrite	無。	將 Write data 輸入的值寫入 Write register 輸入所指定的暫存器中。
ALUSrc	ALU 的第二個運算元來自暫存器檔案第二個輸出埠(Read data 2)。	ALU 的第二個運算元來自指令中最低的 16 個位元經過符號延伸後的值。
PCSrc	由計算 $PC + 4$ 的加法器的輸出來替換原本的 PC 值。	由計算分支目標位址的加法器的輸出來替換原本的 PC 值。
MemRead	無。	將數據記憶體中 Address 輸入所指定位址的內容送至 Read data 輸出。
MemWrite	無。	將數據記憶體中 Address 輸入所指定位址的內容置換為 Write data 輸入的值。
MemtoReg	暫存器 Write data 輸入的值來自 ALU。	暫存器 Write data 輸入的值來自數據記憶體。



R-type指令

e.g.
add \$S1, \$S2, \$S3

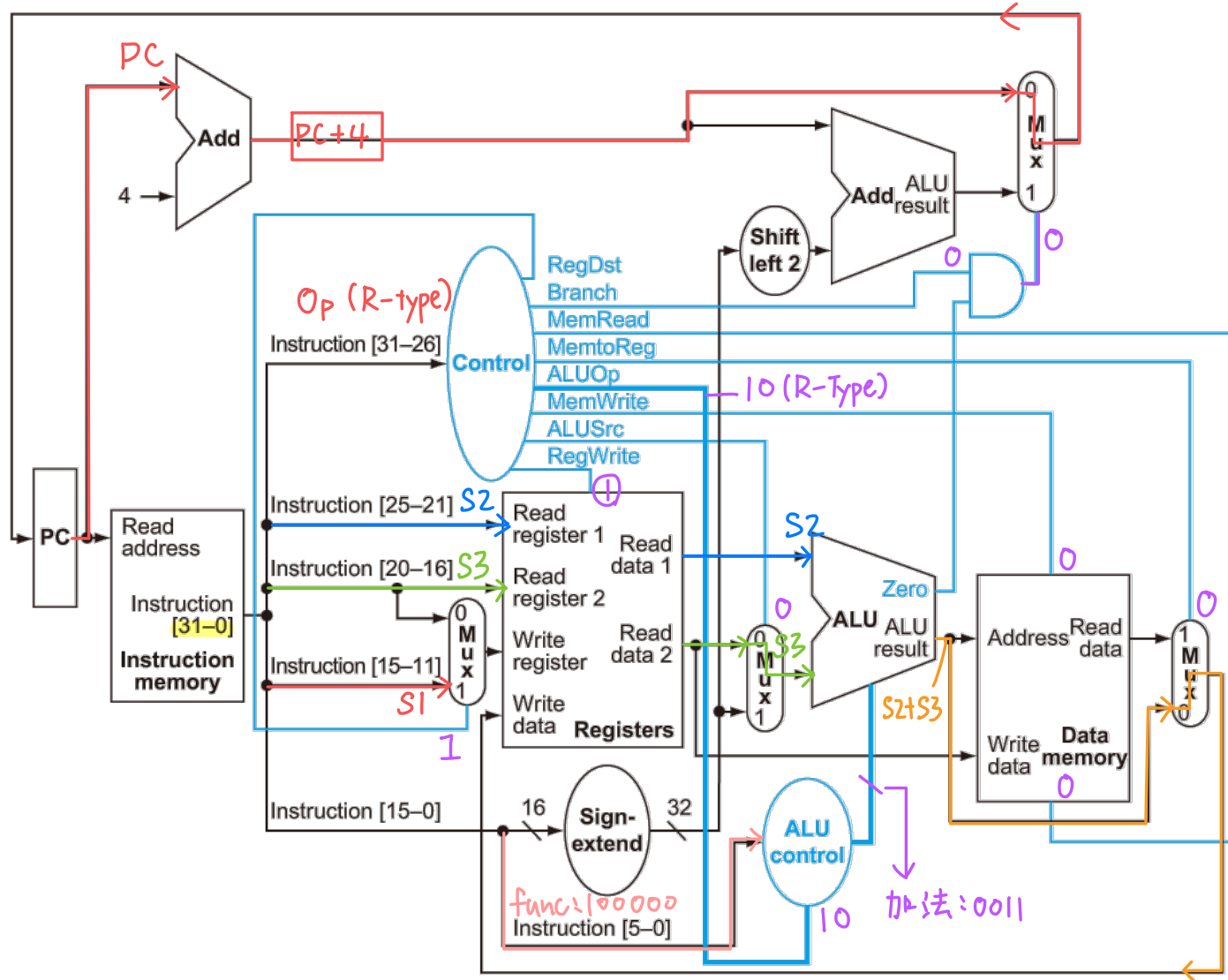
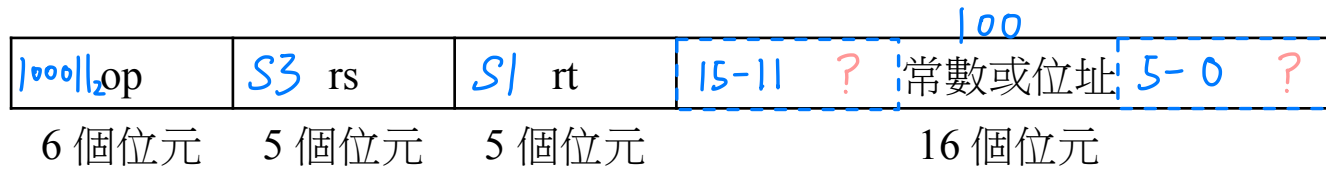


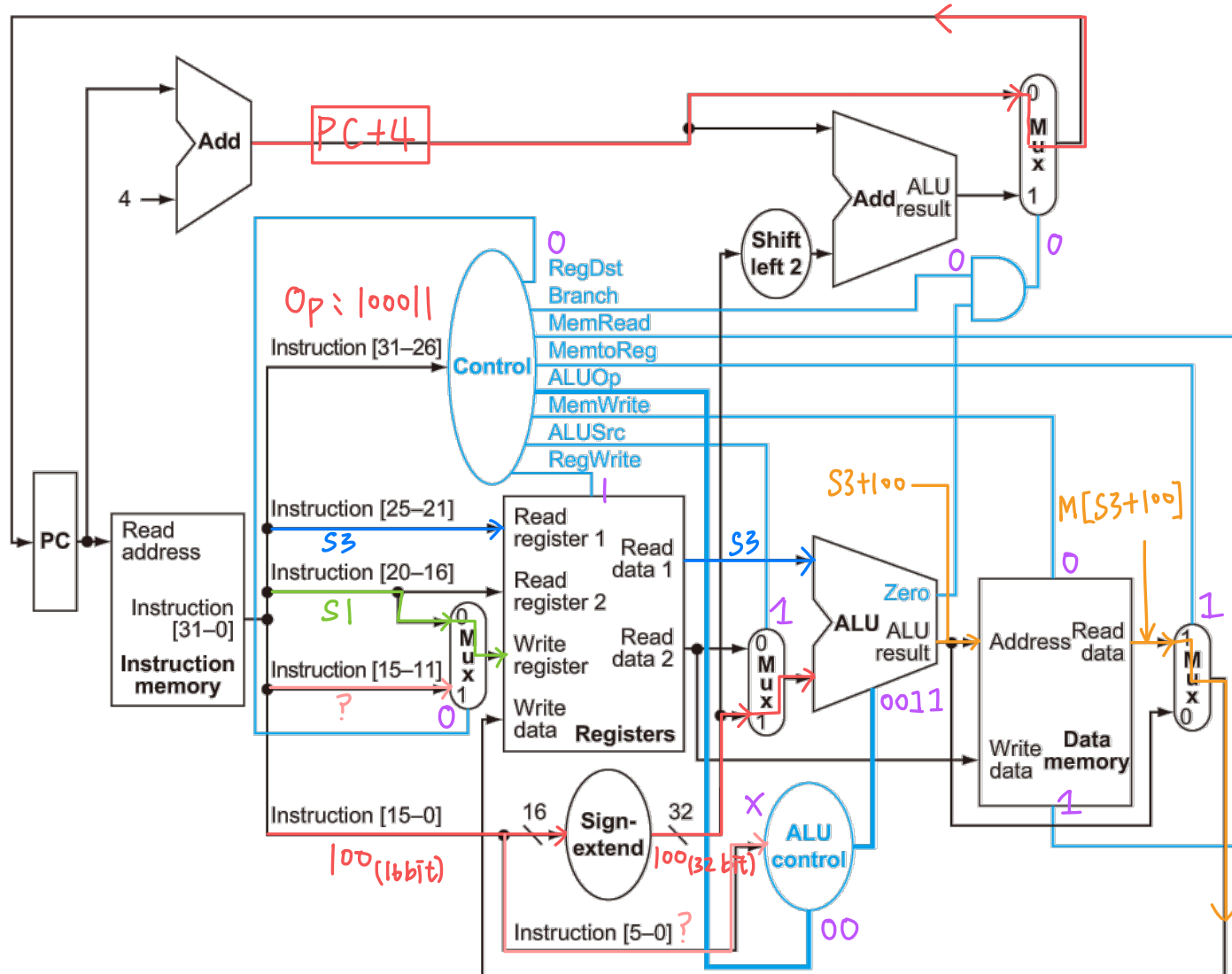
圖4.17表示包含控制單元以及控制訊號的數據通道



I-type指令

lw \$S1, 100(\$S3)

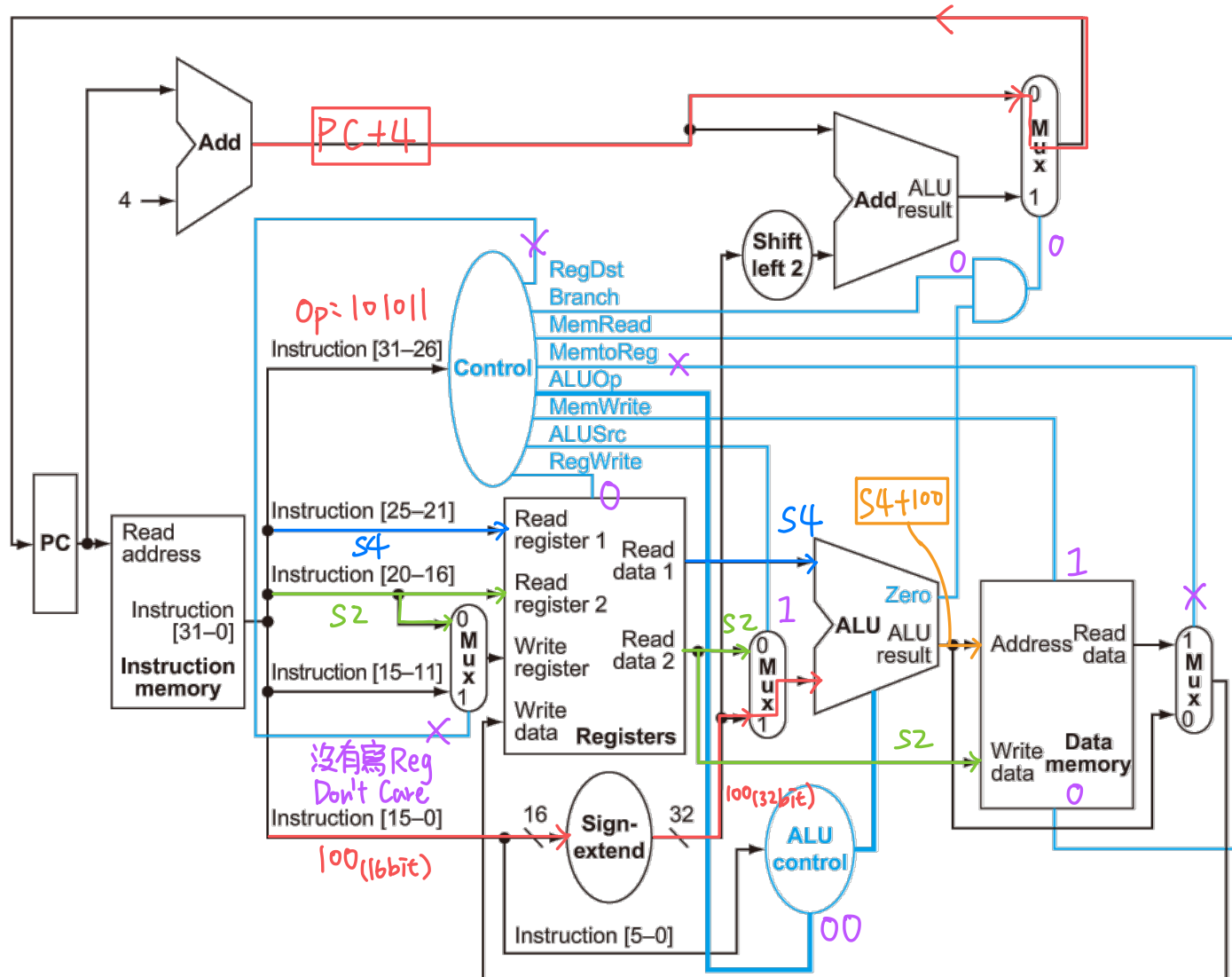
$S1 \leftarrow M[S1+100]$



101011 ₂ op	S4 rs	S2 rt	15-11 ? 常數或位址	5-0 ?
6 個位元	5 個位元	5 個位元	16 個位元	

I-type指令

SW \$S2, 100(\$S4)
 $M[S4+100] \leftarrow S2$





16 個位元

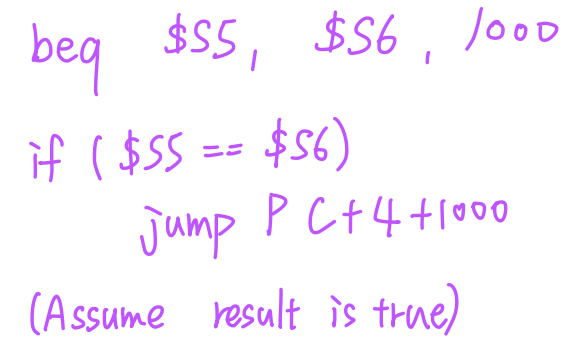


圖4.17 含有控制單元的簡單數據通道

控制單元的輸入為指令中的**6** 位元運作碼欄位。控制單元的輸出包含了三條**1** 位元的訊號線用來控制多工器(**RegDst**、**ALUSrc** 及**MemtoReg**)，三條訊號線用來控制暫存器檔案及數據記憶體的讀與寫(**RegWrite**、**MemRead** 及**MemWrite**)，一條**1** 位元的訊號線用來決定是否分支(**Branch**)，以及**2** 位元的訊號線用來控制**ALU**(**ALUOp**)。使用一個**AND** 閘來運算 **Branch** 控制訊號及**ALU** 的**Zero** 輸出訊號線；**AND** 閘的輸出控制下一個**PC** 值的選擇。注意**PCSrc** 是衍生出來的訊號，而不是直接來自於控制單元，因此在接下來的圖中將不顯示它的名稱

設計主要的控制單元

- 圖4.18 定義所有控制訊號線應該如何對每一種運作碼來設定

指令	RegDst	ALUSrc	Mem-toReg	Reg-Write	Mem-Read	Mem-Write	Branch	ALUOp1	ALUOp0
R 格式	1	0	0	1	0	0	0	1	0
lw	0	1	1	1	1	0	0	0	0
sw	X	1	X	0	0	1	0	0	0
beq	X	0	X	0	0	0	1	0	1

圖4.18 控制訊號線的設定完全取決於指令中的運作碼欄位

表中的第一列對應到R 格式的指令(add、sub、and、or 及slt)。這些指令的來源暫存器欄位都是rs 和rt，目的暫存器都是rd；這定義了ALUSrc 和RegDst 控制訊號線如何設定。此外，R-型指令將運算結果寫入暫存器(RegWrite=1)，但是既不會讀取也不會寫入數據記憶體。當Branch 控制訊號等於0 時，PC 的值無條件地由PC+4 取代；否則，如果ALU 的Zero 輸出值也為1 時，PC 的值由分支目標位址取代。R-型指令的ALUOp 欄位被設定為10，表示ALU 的控制是由功能欄位(func field)來產生。在表中的第二列及第三列說明lw 指令及sw 指令的控制訊號設定。這裡的ALUSrc 和ALUOp 欄位設定成執行位址的計算。而Mem-Read 及MemWrite 設定成執行記憶體的存取。最後，對於載入指令，RegDst 及RegWrite 設定成將結果儲存到rt 暫存器中。分支指令和R 格式指令相似，都是將rs 和rt 暫存器的值送到ALU 去運算。分支指令的ALUOp 欄位設定成執行減法(ALU 控制=01)，用來測試兩個值是否相等。注意如果RegWrite 控制訊號為0 時MemtoReg 欄位是無關緊要的：因為暫存器不會被寫入，所以暫存器的Write data 輸入值不會被用到。因此，在表格最後兩列的MemtoReg 項目用X 來表示，代表「無所謂」。當RegWrite 為0 時，RegDst 也可以使用X 來表示。設計者必須要使用這種「無所謂」項目，因為其關乎對數據通道運作的瞭解

數據通道的運作

- ▣ 接下來的幾張圖表示三種不同種類的指令通過數據通道的流程
 - 每一張圖中，被設定的控制訊號線以及動作的數據通道元件都會以灰色來強調
- ▣ 圖4.19 顯示數據通道對R 型指令
 - 一個時脈週期內可以把它想像成執行了四個步驟：

數據通道的運作

PC+4

- ① 指令被擷取，並且遞增PC 的值
- ② 兩個暫存器從暫存器檔案中讀出；同時，主控制單元計算控制訊號線的值 → 讀rs、rt (t2、t3)
- ③ ALU 根據功能碼(指令中的5 :0 位元功能欄位)來產生ALU 功能的控制，以對從暫存器檔案中讀出的值運算
- ④ ALU 的運算結果使用指令的15:11 位元來選擇目的暫存器 (\$t1)，以寫入暫存器檔案

add \$t1,\$t2,\$t3
rd rs rt

	rs	rt	rd	shamt	func
0	01010	01011	01100	0	100000
6	5	5	5	5	6

名 稱	暫存器號碼
\$zero	0
\$v0-\$v1	2-3
\$a0-\$a3	4-7
\$t0-\$t7	8-15
\$s0-\$s7	16-23
\$t8-\$t9	24-25
\$gp	28
\$sp	29
\$fp	30
\$ra	31

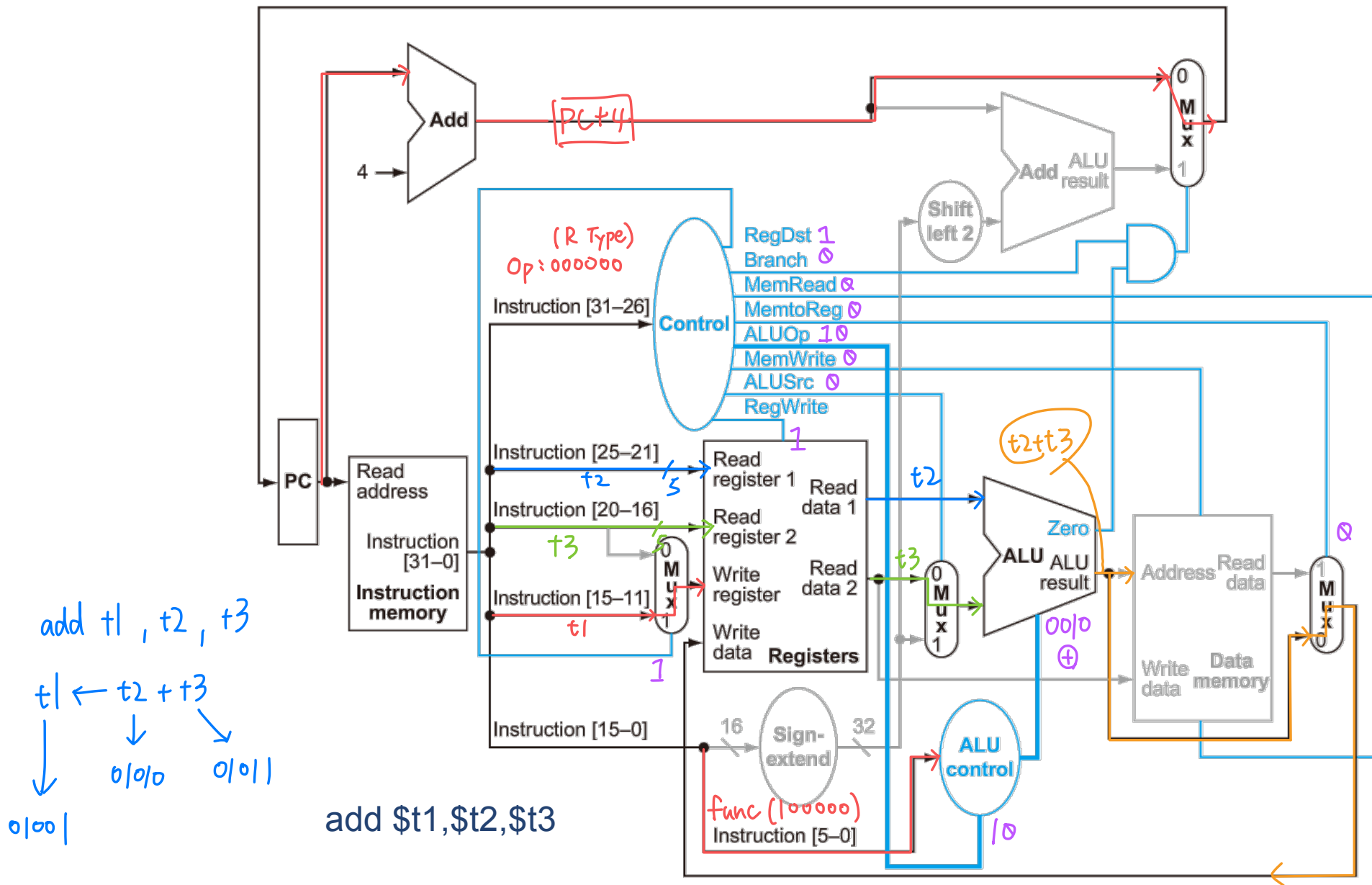


圖4.19 數據通道對於如add \$t1,\$t2,\$t3 的R 型指令運作方式
運作中的控制訊號線、數據通道單元及其連線均以灰色強調

數據通道的運作

$lw \$s1, 20(\$s2)$

$s1 \leftarrow M[s2+20]$

最久的

■ 圖4.20 顯示**載入指令**中有動作的功能單元和被設定的控制訊號

■ 想像成執行了五個步驟：

- ① 指令從指令記憶體中被擷取，並且遞增PC 的值
- ② 暫存器的值從暫存器檔案中讀出
- ③ ALU 計算由暫存器檔案中讀出的值和符號延伸過的指令中較低的16 位元偏移量的和
- ④ ALU 所得的和作為數據記憶體的位址
- ⑤ 記憶體傳回的數據寫入暫存器檔案；暫存器的目的地可由指令中的位元20:16 得知

圖4.20 數據通道對於載入指令的運作方式

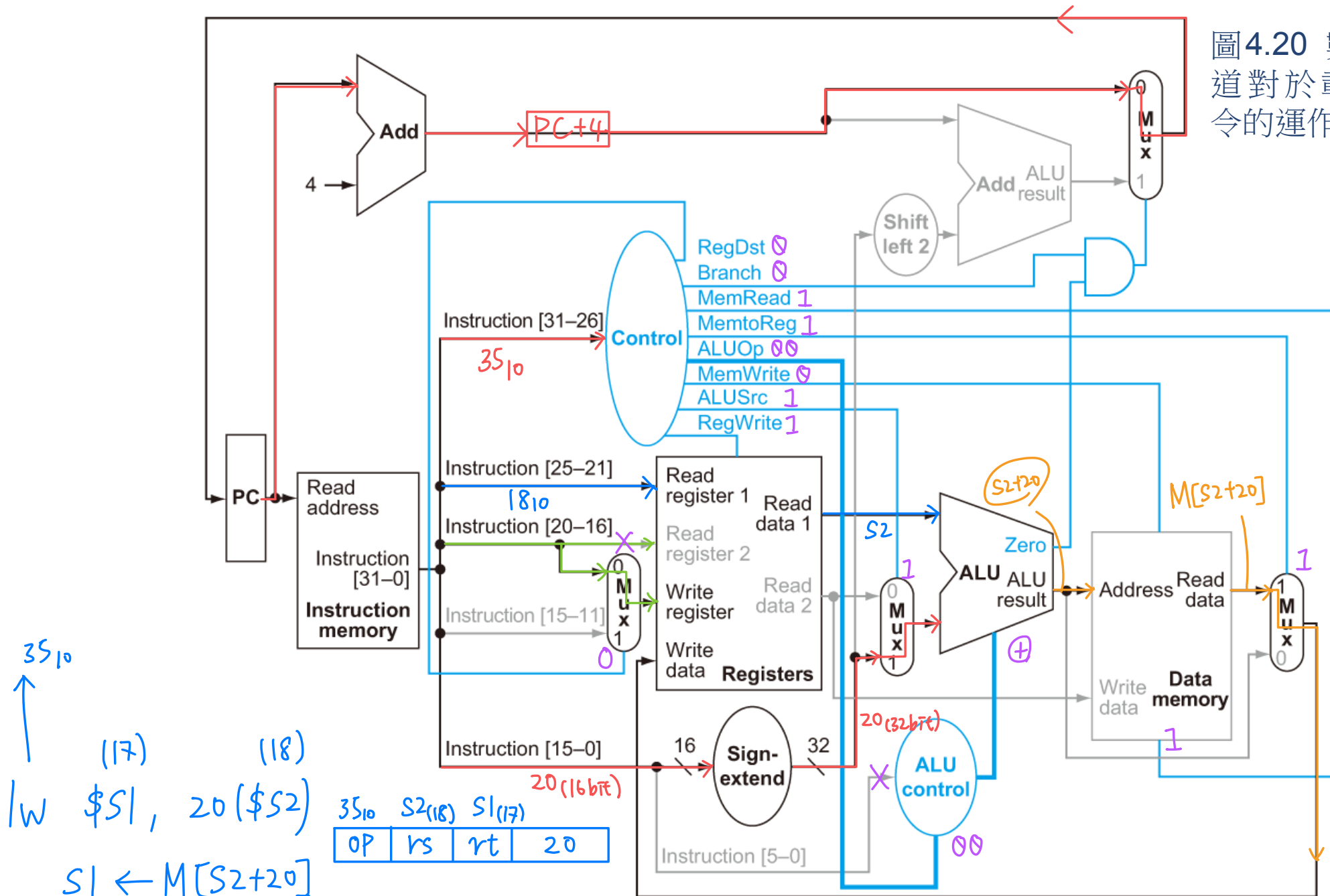


圖4.20 數據通道對於載入指令的運作方式

運作中的控制訊號線、數據通道單元及其連線均以灰色強調。儲存指令的運作方式非常類似。主要的不同在記憶體的控制部分是寫入而不是讀取，而寫入的值來自於讀取第二個暫存器所得的值且不會將數據記憶體的值寫回暫存器檔案

數據通道的運作

beq \$t1 \$t2 100
9 10

■ 圖4.21 顯示**beq**執行時的四個步驟：

- ① 指令從指令記憶體中被擷取，並且遞增PC 的值
- ② 兩個暫存器**\$t1** 和 **\$t2** 從暫存器檔案中被讀出
- ③ ALU 對暫存器檔案中讀出的值**執行減法**。PC+4 的值與符號延伸並左移兩位的指令中較低的16 位元(偏移量)相加；其結果即為分支目的位址
- ④ **ALU 的Zero 輸出被用來決定哪一個加法器的結果要寫回PC**

圖4.21 數據通道對於相等時分支 (branch-on-equal) 指令的運作方式

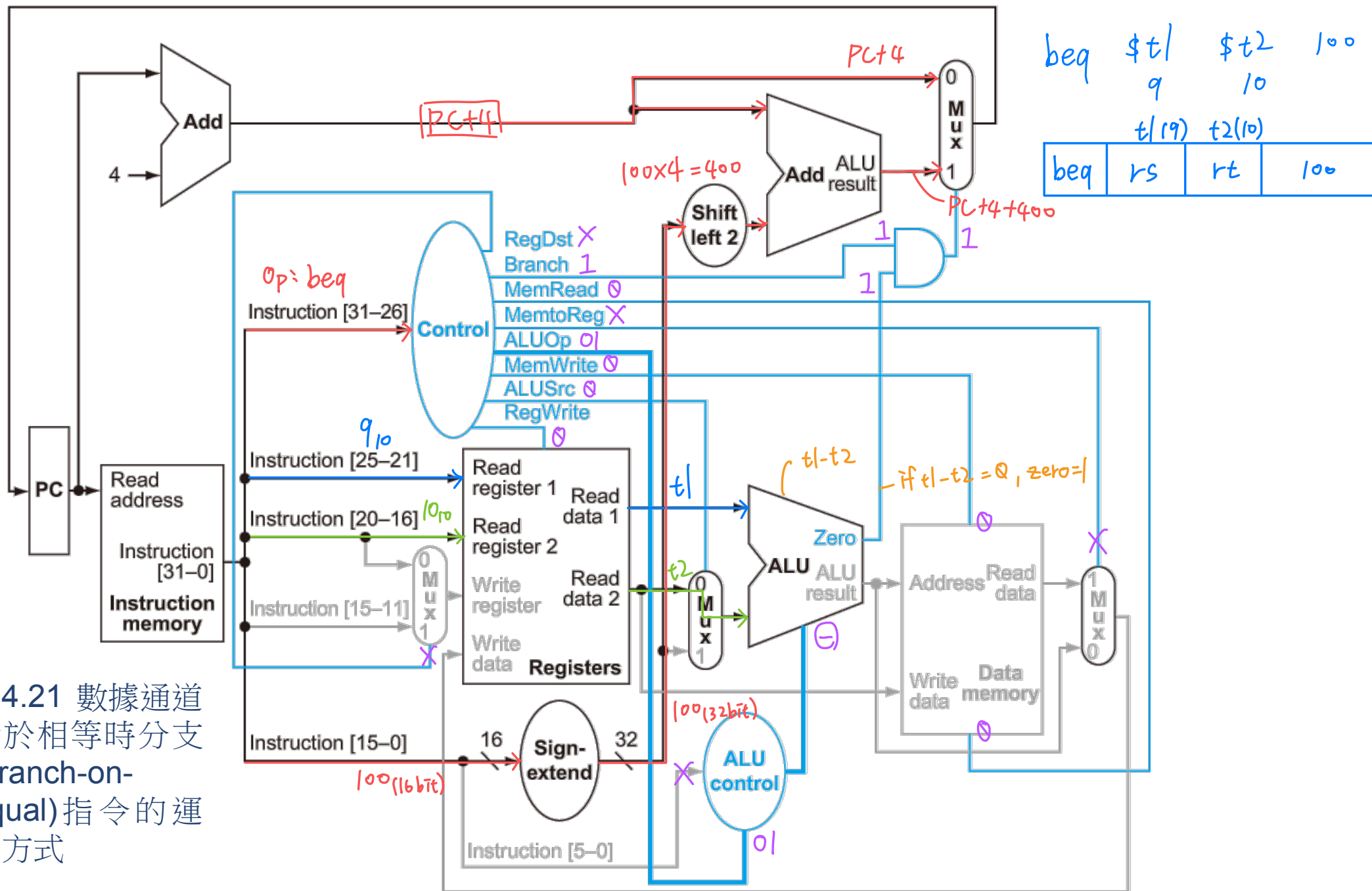


圖4.21 數據通道對於相等時分支(branch-on-equal)指令的運作方式

運作中的控制訊號線、數據通道單元及其連線均以灰色強調。使用暫存器讀出的值和**ALU** 來執行比較之後，**Zero** 輸出在兩個候選值之間選擇下一個程式計數器(**PC**)的值

完成控制

- ▣ 圖4.22 以真值表的形式表示了控制單元中的邏輯電路
- ▣ 圖4.24 顯示圖4.17 中加入跳躍指令所作的額外增加

圖4.22
簡單單一
週期製作
的控制功
能完整說
明於此真
值表中

輸入或輸出	訊號名稱	R 格式	lw	sw	beq
輸 入	Op5	0	1	1	0
	Op4	0	0	0	0
	Op3	0	0	1	0
	Op2	0	0	0	1
	Op1	0	1	1	0
	Op0	0	1	1	0
輸 出	RegDst	1	0	X	X
	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1

圖4.22 簡單單一週期製作的控制功能完整說明於此真值表中
在表的上半部四行中是四種運作碼用來決定輸出控制訊號設定對應的輸入訊號線組合並且(記得Op [5 :0] 是指令31 : 26位元的運作碼欄位。)表格的下半部則是四種運作碼的輸出。因此，輸出**RegWrite** 在兩種輸入的組合會被設定。如果我們只考慮表中的四種運算碼，則我們可以在輸入的部分使用「無所謂」來簡化真值表。例如，我們可以使用Op5 · Op2來偵測出R 格式的指令，因為這已經足夠來區分R 格式指令和lw、sw 及beq。但是因為在完整的實作中會加入其他MIPS 運作碼，所以我們並不採用這樣的簡化方式

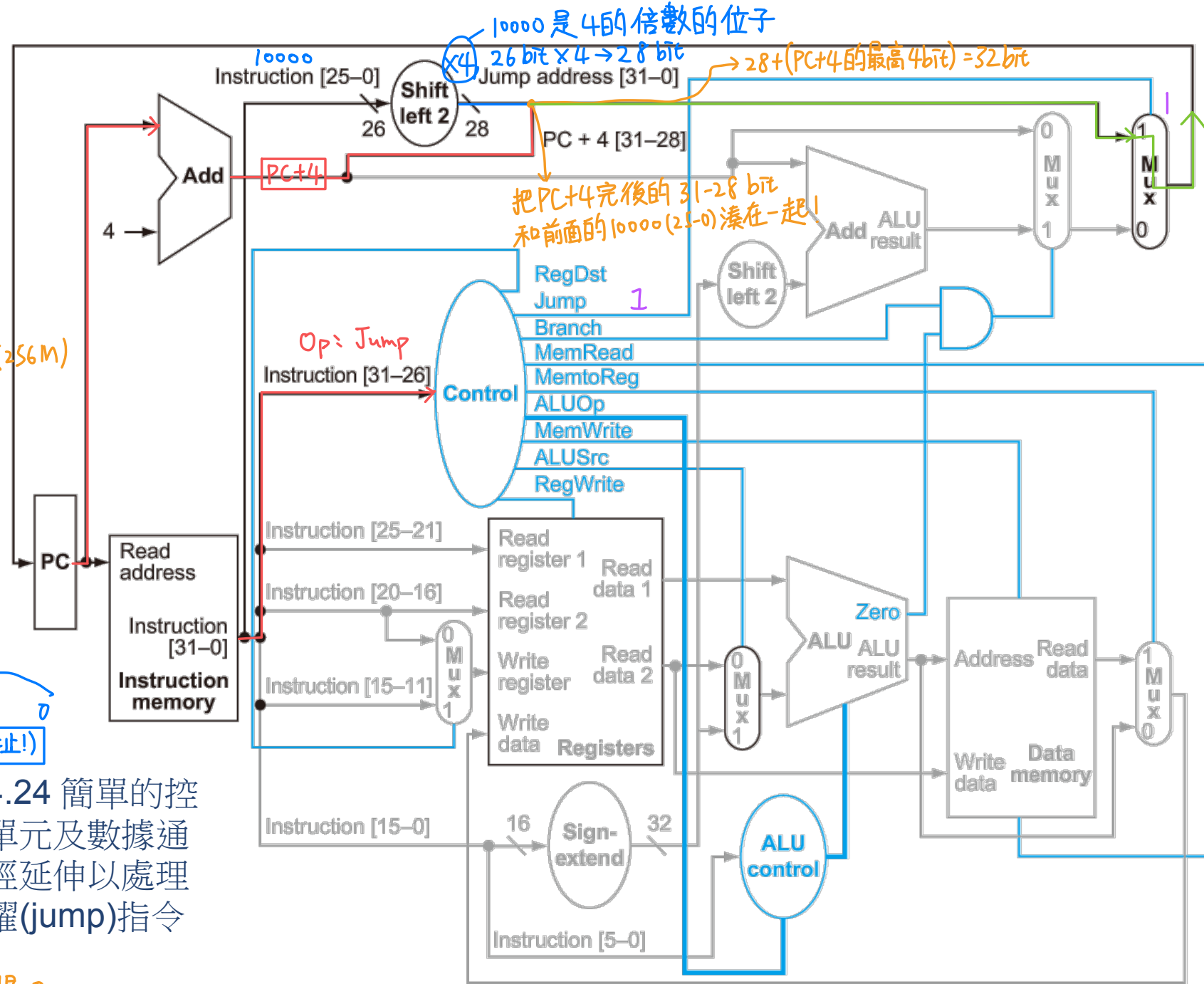
why 湊上 PC+4

→ 讓程式跳不出其所分配的記憶體塊(256M)

e.g. 28 bit
 0010 XXX ... XXX
 0100 XXX ... XXX
 鎖住

e.g. 26 bit
 J 10000 (位址!)

圖4.24 簡單的控
 制單元及數據通
 道經延伸以處理
 跳躍(jump)指令
 4的倍數
 後2位都是0



```

Loop: sll $t1, $s3, 2
      add $t1, $t1, $s6
      lw $t0, 0($t1)
      bne $t0, $s5, Exit
      addi $s3, $s3, 1
      j Loop
Exit: ...

```

從下行開始
跳兩行

PC+4 = 0x80000038

PC+4[31~28] = 1000₂

j Loop => 000010 [000000000000000000000000000000001000]

→ 拼上 PC+4 後的 4 高位

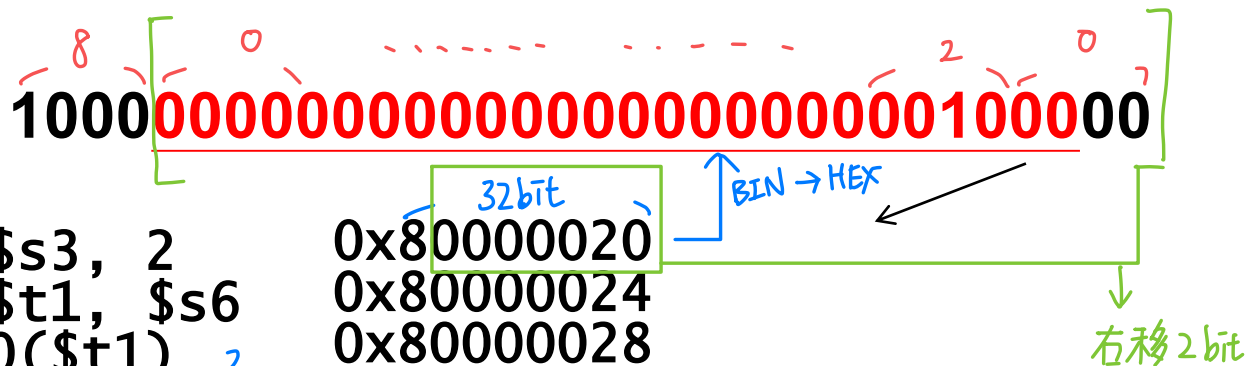
Jump address = PC+4[31~28] +

00000000000000000000000000000000100000

×4 完多 2 bits

= 10000000000000000000000000000000100000

= 0x800000020



<- PC

圖4.24 簡單的控制單元及數據通道經延伸以處理跳躍(jump)指令

以一個額外的多工器(在右上方)來選擇跳躍目標位址還是分支目標位址或目前指令的下一道連續指令位址。這個多工器由 **jump** 控制訊號線來控制。跳躍目標位址是由將跳躍指令中較低的**26** 位元左移**2** 個位元，等同於加上低位的兩個位元**00**，再串接**PC+4** 的最高**4** 位元，因此而得一個**32** 位元的位址

為何今天已不採用單一週期設計

- 每個指令的時脈週期都必須有相同長度
(因此, $CPI = 1$)
- 計算機中最長的路徑將決定時脈週期的長度, 這條路徑幾乎可以確定是載入指令
- 整體效能似乎不是很好
- 範例: 單一時脈計算機的效能
假設功能單元的運算時間如下:
 - 記憶體單元: 2 ns
 - ALU 及加法器: 2 ns
 - 暫存器檔案 (讀取或寫入): 1 ns下列的製作方式那一種會比較快?
 1. 每個指令在一個固定長度的時脈週期內運作完成
 2. 每個指令在一個時脈週期內運作完成, 但時脈週期長度是可變動

• 範例 (續)

為了計算效能，假設我們使用下列指令的混合比例：

24% 載入， 12% 儲存， 44% R形態指令， 18% 分支及2%跳躍指令

» 解答

1. CPU 時脈週期為 8 ns. → 固定周期, 都是 8ns

$$2. \text{ CPU 時脈週期} = 8 \times 24\% + 7 \times 12\% + 6 \times 44\% + 5 \times 18\% + 2 \times 2\% = 6.3 \text{ ns}$$

效能改進的比例為 $8/6.3 = 1.27$.

	指令種類所用到的功能單元					指令種類	指令記憶體	暫存器讀取	ALU運算	資料記憶體	暫存器寫入	總和
R格式	指令擷取	暫存器存取	ALU	暫存器存取		R格式	2	1	2	0	1	6ns
載入字組	指令擷取	暫存器存取	ALU	記憶體存取	暫存器存取	載入字組	2	1	2	2	1	8ns
儲存字組	指令擷取	暫存器存取	ALU	記憶體存取		儲存字組	2	1	2	2		7ns
分支	指令擷取	暫存器存取	ALU			分支	2	1	2			5ns
跳躍	指令擷取					跳躍	2					2ns

- 範例：

假設我們有浮點指令單元：執行浮點加法需要8ns，執行浮點乘法需要16ns，所有功能單元所需的時間如同上例。下列的製作方式那一種會比較快？

1. 每個指令在一個固定長度的時脈週期內運作完成。
2. 每個指令在一個時脈週期內運作完成，但時脈週期長度是可變動。

為了計算效能，假設我們使用下列指令的混合比例：31%載入，21%儲存，27%R形態指令，5%分支，2%跳躍指令，7%浮點加法及7% FP浮點乘法。

» 解答

1. 最長的指令為浮點乘法，其時脈週期為

$$\textcircled{2} + \textcircled{1} + \textcircled{16} + \textcircled{1} = 20 \text{ ns}$$

2. 浮點指令的加法須時 $2 + 1 + \textcircled{8} + 1 = 12 \text{ ns}$.

$$\text{CPU 時脈週期} = 8 \times 31\% + 7 \times 21\% + \textcircled{6} \times 27\% + 5 \times 5\% + 2 \times 2\% + 20 \times 7\% + 12 \times 7\% = 7.0 \text{ ns}$$

效能改進的比例為 $20/7 = 2.9$.

FP 乘法

從 mem 拿

讀 Reg

Write Reg

FP 加法

為何今天已不採用單一週期設計

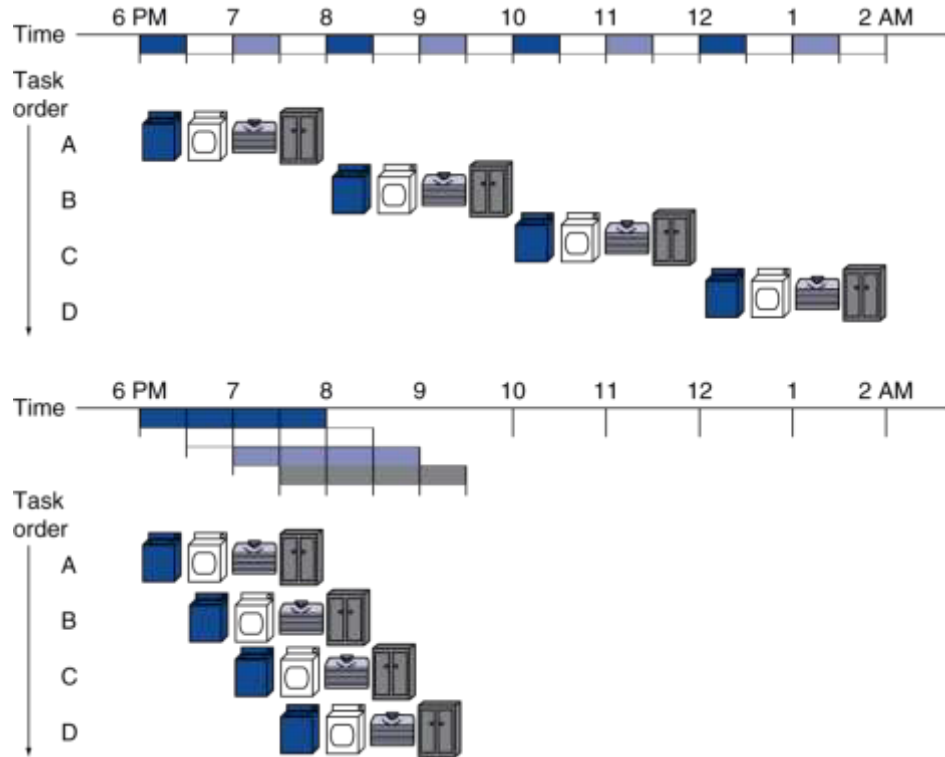
- ▣ 另一種稱為管道化(pipelining)的製作技法，其使用一個和單一週期數據通路非常類似的數據通路，然而具有很高的處理量，因此非常有效率
 - 管道化藉由同時執行多道指令來增加效率

4.5 管道化處理概觀

- ▣ 管道化處理(pipelining)是一種能讓多個指令在執行時重疊(overlapped)的實作技巧，目前管道化幾乎已被普遍採用
- ▣ 管道化的方式需時遠較少，如圖4.25 所示
 - 只要我們有各別的資源提供給每一個階段，我們就可以將各項工作管道化
 - 管道化改善了處理量 (throughput)
 - 其並不會減少單獨一件工作所需時間

Pipelining Analogy

- ▣ Pipelined laundry: overlapping execution
 - Parallelism improves performance



- **Four loads:**

- **Speedup**
 $= 8 / 3.5 = 2.3$

- **Non-stop:**

- **Speedup**
 $= 2n / (0.5n + 1.5) \approx 4$
 $= \text{number of stages}$

MIPS Pipeline

MIPS中，將管線化分成五階

▣ Five stages, one step per stage

1. IF: Instruction fetch from memory 指令截取
2. ID: Instruction decode & register read 指令解碼
3. EX: Execute operation or calculate address 執行階段
4. MEM: Access memory operand 讀寫記憶體
5. WB: Write result back to register 寫回暫存器

Pipeline Performance

- Assume time for stages is
 - 100ps for register read or write
 - 200ps for other stages

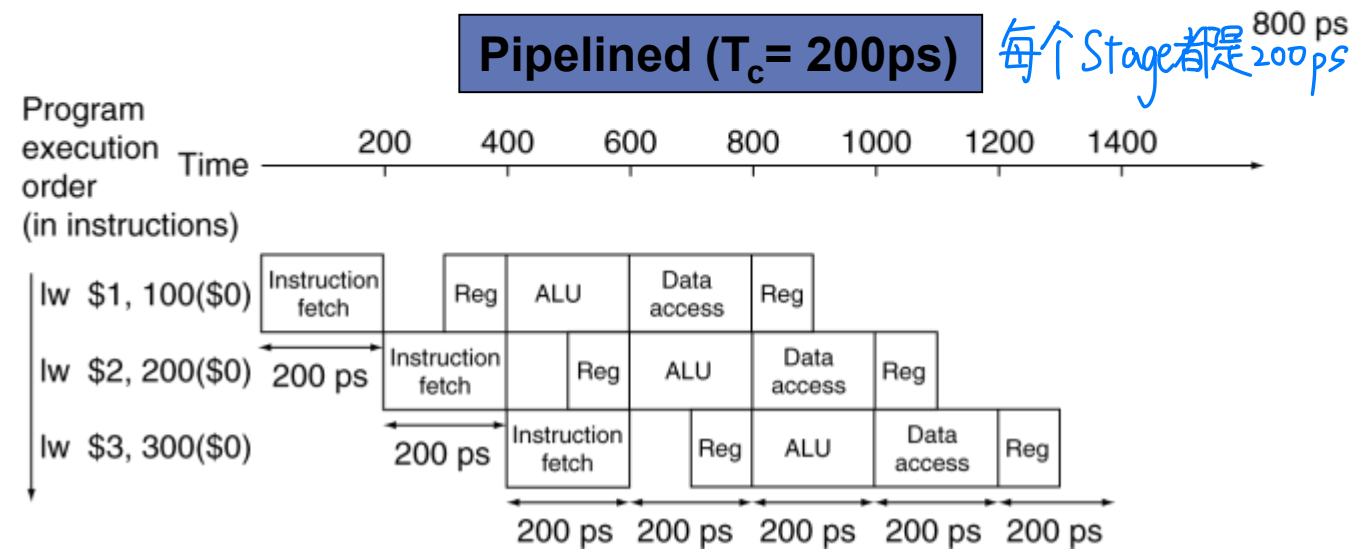
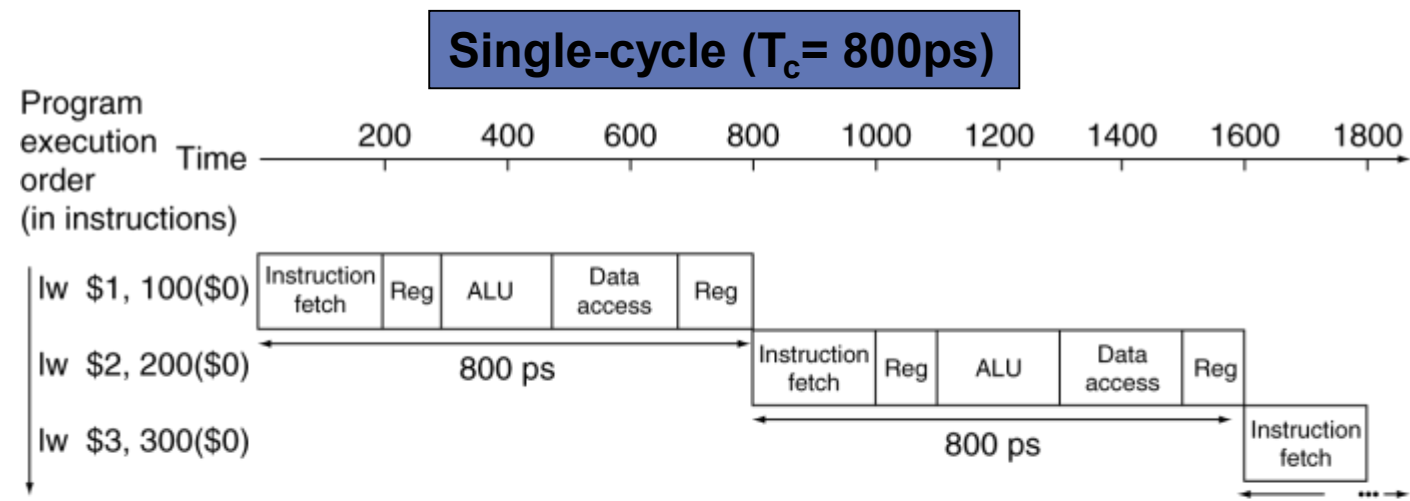
- Compare pipelined datapath with **single-cycle** datapath

→ 管線化 - 找最長的 Stage: 200ps

800ps (用最長的 Total Time)

Instr	Instr fetch	Register read	ALU op	Memory access	Register write	Total time
lw	200ps	100 ps	200ps	200ps	100 ps	800ps
sw	200ps	100 ps	200ps	200ps		700ps
R-format	200ps	100 ps	200ps		100 ps	600ps
beq	200ps	100 ps	200ps			500ps

Pipeline Performance



Pipeline Speedup

／ 若所有的 Stage 所花時間幾乎相同

▣ If all stages are balanced

- i.e., all take the same time
- Time between instructions_{pipelined}
=
$$\frac{\text{Time between instructions}_{\text{nonpipelined}}}{\text{Number of stages}}$$

▣ If not balanced, speedup is less → 若沒有balanced, speedup 会降低

▣ Speedup due to increased throughput

- Latency (time for each instruction) does not decrease

／ 不是所有指令執行時間都有降低

4.5 管道化處理概觀

▣ MIPS 的指令基本上需要五個步驟：

1. 從記憶體中擷取(fetch)指令 IF
2. 解碼(decode)指令並同時讀取暫存器的值。MIPS 規律的指令格式允許解碼和讀取同時進行 ID
3. 執行(execute)運作或計算位址 EX
4. 存取數據記憶體(memory)中的運算元 MEM
5. 將結果寫回(write back)暫存器中 WB

▣ 本章探討的MIPS 管道具有五級

4.5 管道化處理概觀

- ▣ 在理想的情況下並且具有大量指令時，從管道化所得到的加速(speed-up)約略等於管道級數
 - 管道階段的時間可能不是那麼平均
 - 管道化會造成一些額外的負擔(overhead)
- ▣ 管道化藉著增加指令處理量來改善效能，而不是減少個別指令的執行時間
 - 由於真實程式會執行以億計的指令，所以指令的處理量是重要的指標

為管道化來設計指令集

ISA

- MIPS 指令集設計的意涵，即是為了管道化的執行而設計
 - 第一，所有的MIPS 指令都擁有相同的長度
 - 第二，MIPS 只有少數幾種指令格式，並且每道指令中來源暫存器欄位都在相同的位置。解碼時可以同時開始讀取暫存器檔案
 - 第三，MIPS 中的記憶體運算元只出現於載入或儲存指令中。可以利用執行級來計算記憶體位址，然後於下一級存取記憶體

管道的危障

- 在管道中有時下一道指令不能緊跟著在下個時脈週期中執行。這樣的情況稱為危障(hazards)

- 其有三種不同的類型：

① 結構危障(structural hazard) (硬體不支援)

表示硬體無法支援我們要在同一個時脈週期中執行的指令的組合

① MIPS 中記憶體分成兩塊

- Inst MEM: 指令記憶體

- Data MEM: 資料記憶體

→ 不分成兩塊, 會發生 structural hazard!

② 數據危障(data hazards)：當管道因為某一個步驟必須等待其他步驟的完成而停滯(stall)時 (Now要用的資料前一個指令根本還沒算出來)

- 因為一道指令對稍早的一道指令有相依性(dependence)，且稍早的那道指令還在管道中未完成而發生

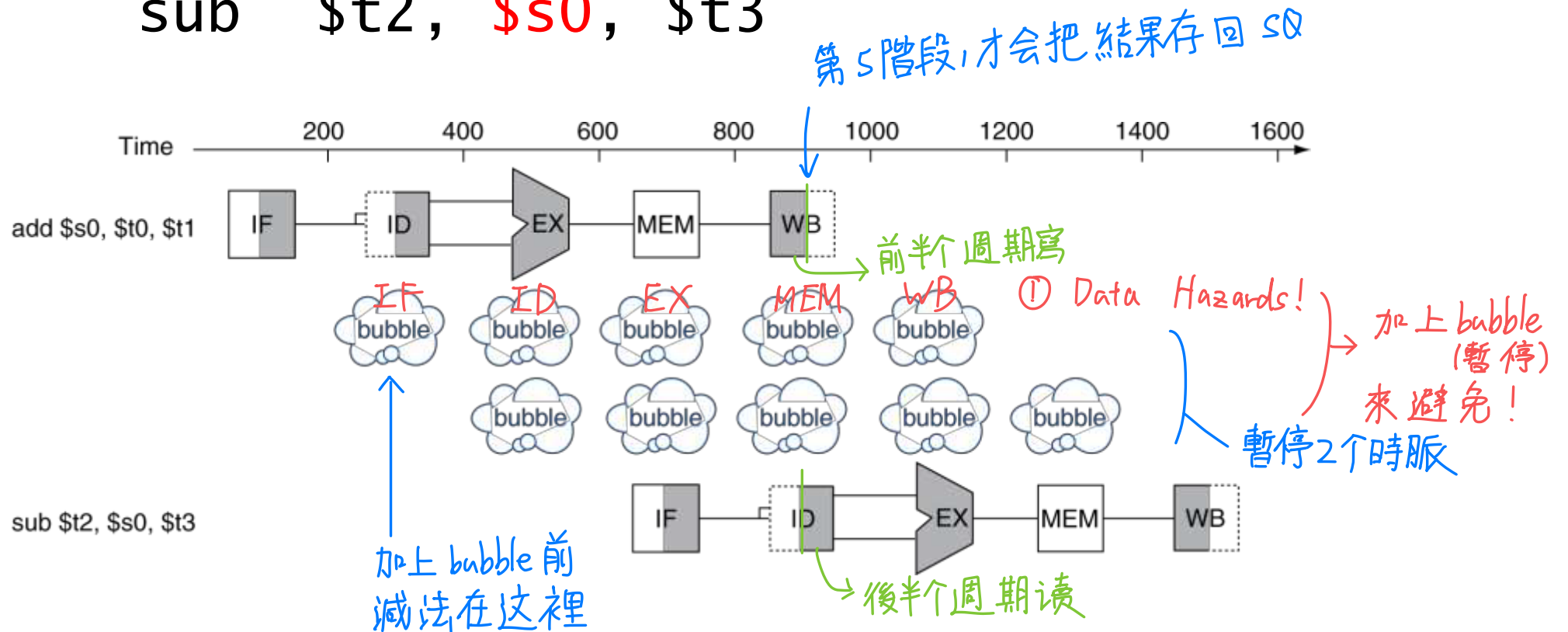
管道的危障

- 一旦ALU產生加法的和，我們就可以把它送去當成減法的輸入
- 加入特殊的硬體來提早從內部資源獲取所需的資料稱為前饋(forwarding)或繞送(bypassing)
 - 圖4.29

Data Hazards

- An instruction depends on completion of data access by a previous instruction

- `add $s0, $t0, $t1`
`sub $t2, $s0, $t3`



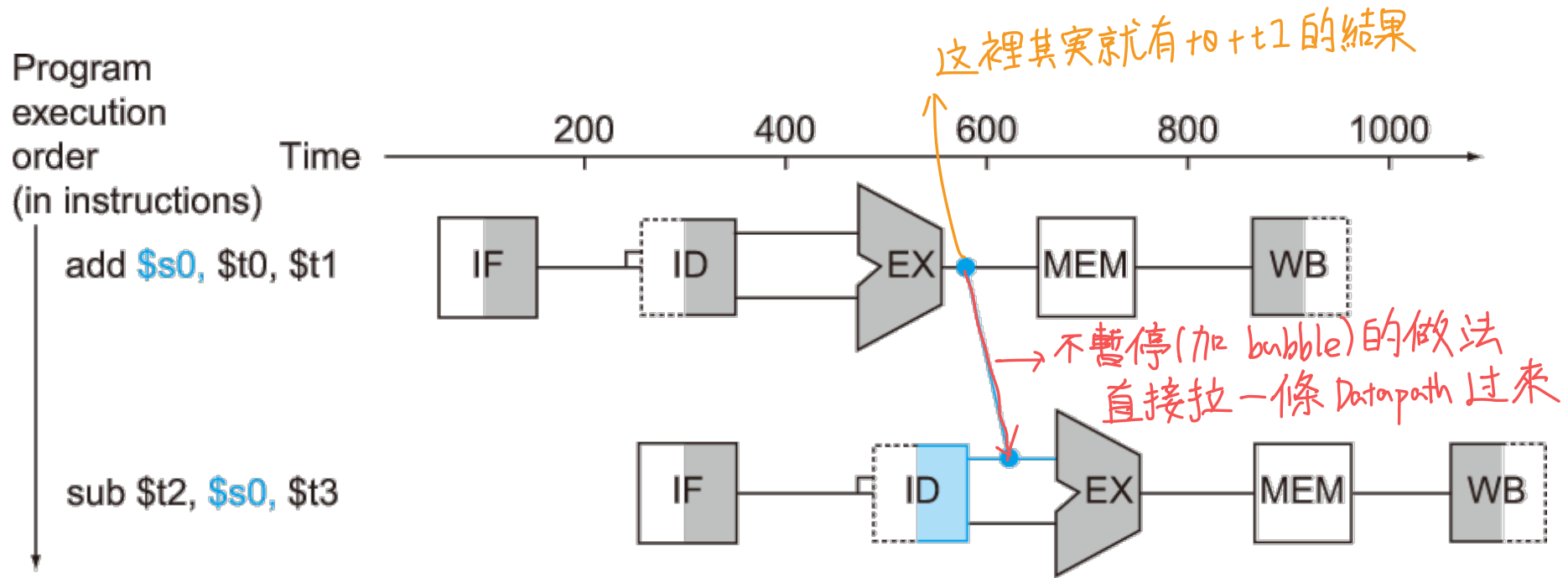


圖4.29 前饋的圖形表示法

連線表示從 **add** 的 **EX** 級輸出到 **sub** 的 **EX** 級輸入的前饋路徑，以取代 **sub** 在第二級從暫存器 **\$s0** 中讀取的值

不加 Bubble 的方法：前饋(forwording)

管道的危障

② 數據危障(data hazards)

- 前饋並無法避免所有的管道停滯。

- 仍需對載入—使用數據危障(load-use data hazard)停滯一級，如圖4.30所示

- 正式的名稱是管道停滯(pipeline stall)，但通常俗稱氣泡(bubble)

③ 控制危障(control hazard) (條件分支、beq or bne 指令)

- 起因為當某些指令正在執行時卻又需要基於另一指令的結果(如條件分支)，來決定該等指令是否該執行

add sq t1 t2
sub s1 sq t3

→ 不加 bubble 下，
可以利用 forwarding 解決!

lw sq 20(s1)
sub s1 sq s2

→ 如果是 lw，即使用了 forwarding
还是要加上一個 bubble
來等待!

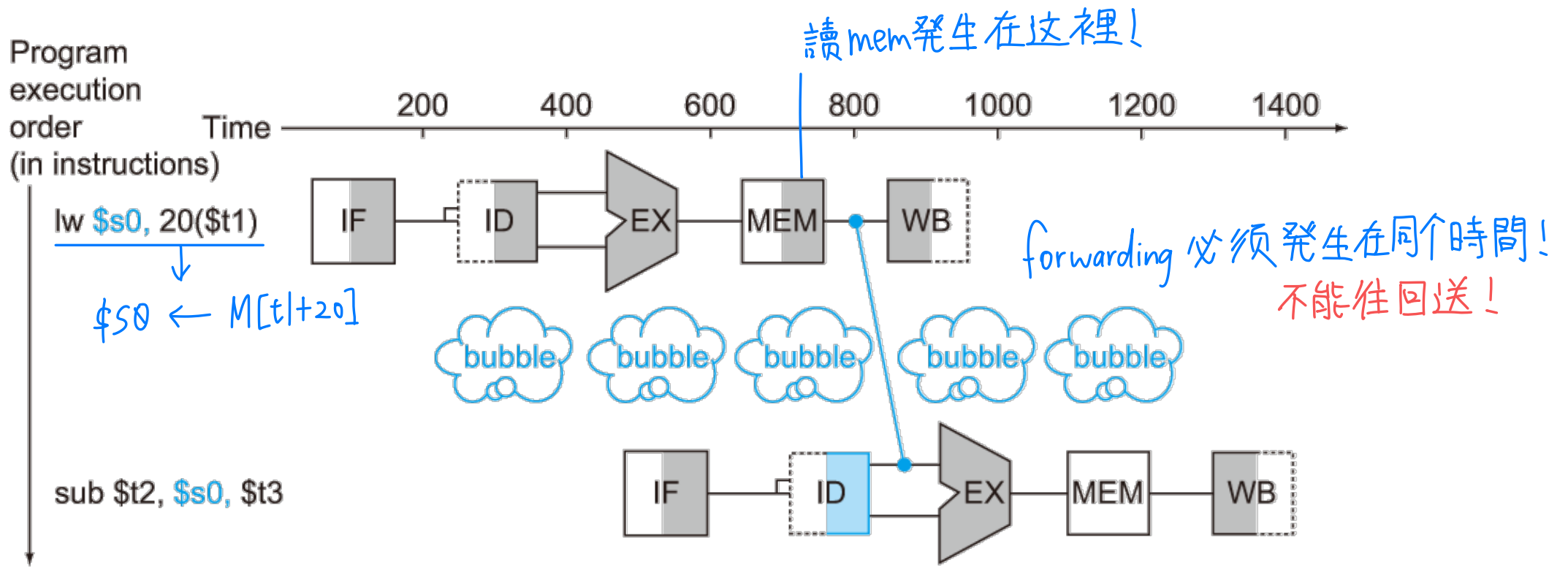


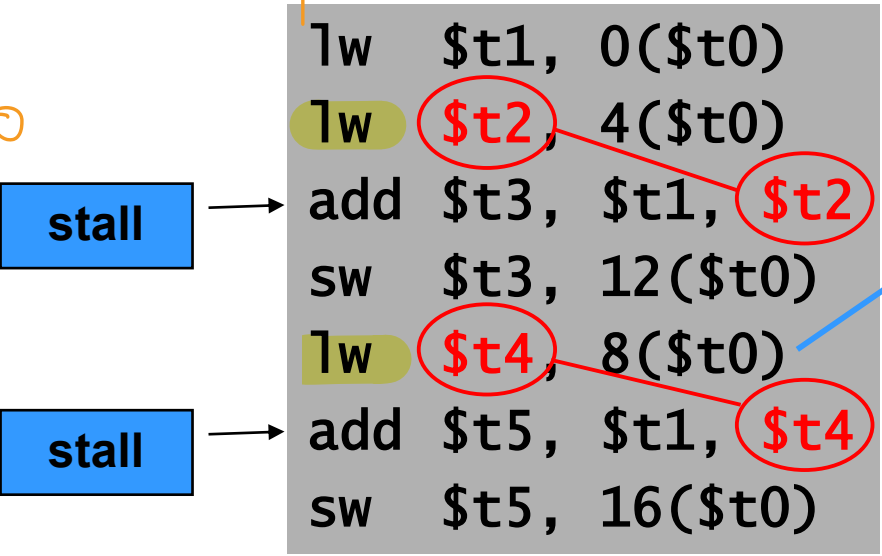
圖4.30 我們在R 格式的指令緊跟在載入指令之後並且要使用載入的數據時即使有前饋，還是需要停滯

如果沒有停滯，從記憶體存取階段輸出到執行階段輸入的路徑將會逆著時間走，這是不可能的。這個圖實際上作了簡化，因為我們在減法指令被擷取與解碼前，並不能知道是否需要暫停。4.7 節說明在這種危障下實際上會發生的細節

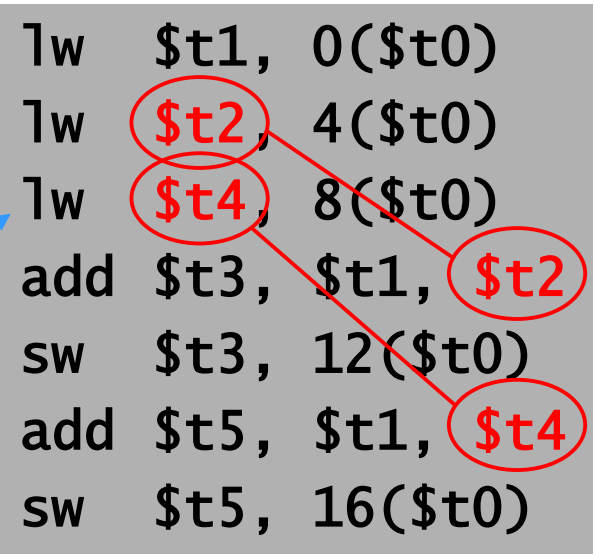
Code Scheduling to Avoid Stalls

- Reorder code to avoid use of load result in the next instruction
藉由重新排序指令來避免載入指令的相依性 (不影響執行結果)
- C code for $A = B + E$; $C = B + F$;

lw₁ 00000
lw₂ 00000
bubble □□□□□
add 00000
⋮



13 cycles



11 cycles

管道的危障

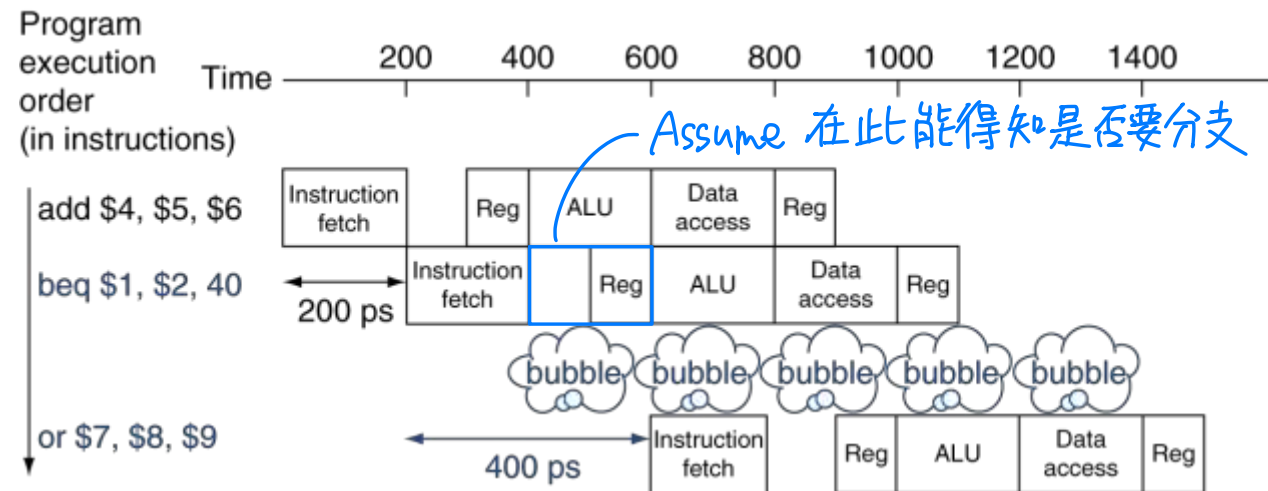
③ 控制危障(control hazard)

- 對控制危障兩種解決方法的第一種
 - 停滯：
- 第一種方法嚴重的減速導致了第二種解決方法
 - 預測(predict)：
→ 分支預測

Stall on Branch

方法 1 : 一直等!

- Wait until branch outcome determined before fetching next instruction



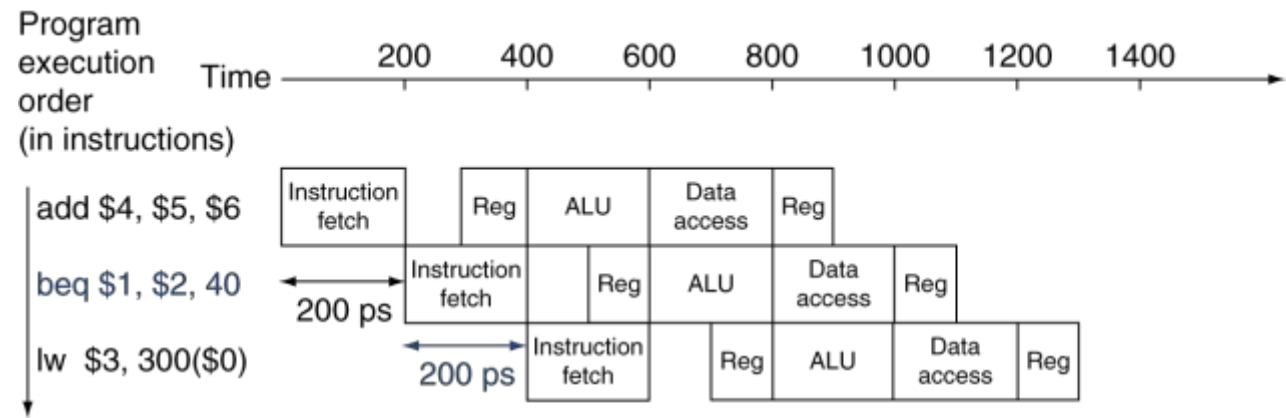
管道的危障

③ 控制危障(control hazard)

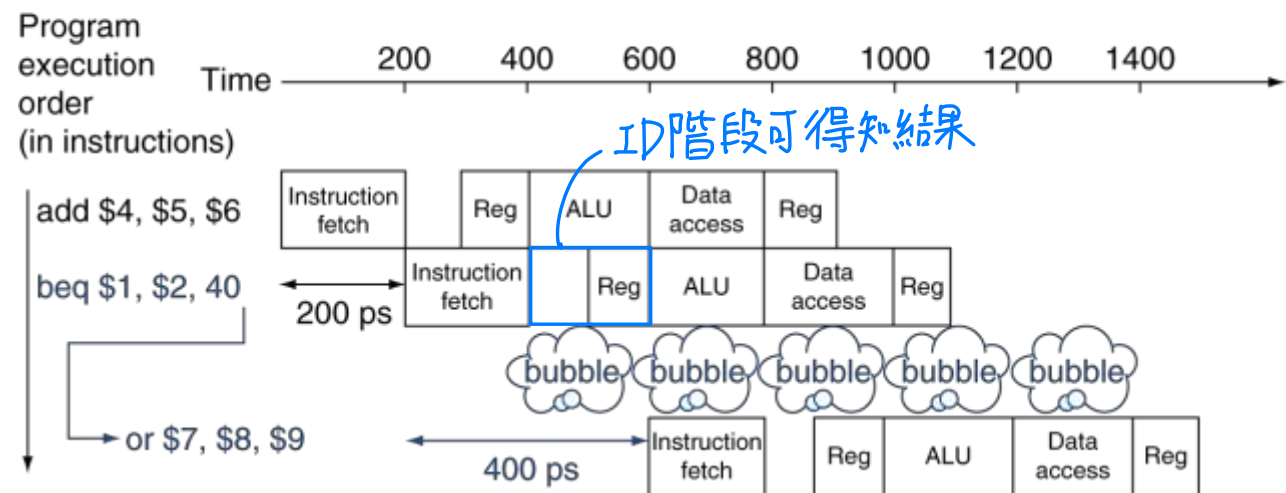
- 預測正確時並不會使管道減速。然而當你猜錯時，就必須清除預測錯誤造成的影響並且重新開始執行該做的工作
- 簡單的預測方法是永遠預測分支不發生(untaken)
- 更複雜的分支預測(branch prediction)將可以預測某些分支會發生而某些不發生
- 動態(dynamic)硬體預測器根據每道分支的行為來作預測，並且在程式運作過程中可以改變對一道分支的預測

MIPS with Predict Not Taken

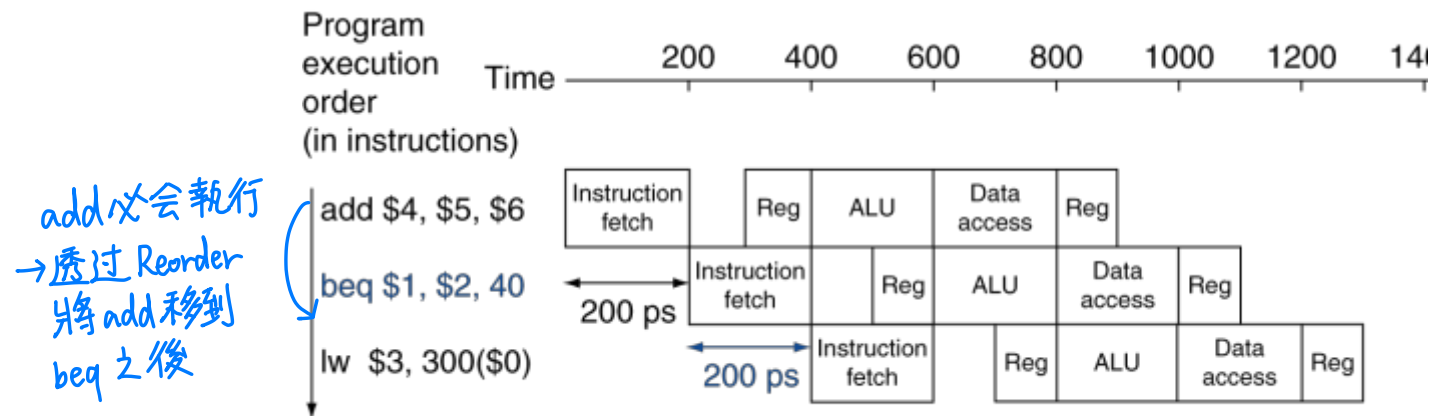
Prediction
correct



Prediction
incorrect



- 控制危障有第三種解法，如前所述稱為延遲的判斷(delayed decision)。將與分支無關的指令交換至分支指令之後，如下圖，add與beq指令無關，可移至beq之後



管道化處理概觀總結

- ▣ 在循序指令流中開發指令間平行度(parallelism)
- ▣ 基本上是不需要程式師去過問的
- ▣ 管道化改善了指令的處理量然而反而會增加單一指令的執行時間或延遲

Throughput ↑

latency ↑

4.6 管道化數據通道及控制

▣ 圖4.33 表示了標記出管道階級後的第4.4 節中的單一週期數據通道

- 5 級的管道
- 代表在任一時脈週期內最多有5 道指令正在執行

1. IF：指令擷取

2. ID：指令解碼與暫存器檔案讀取

3. EX：執行或位址計算

4. MEM：數據記憶體存取

5. WB：寫回

IF: Instruction fetch

ID: Instruction decode/
register file readEX: Execute/
address calculation

MEM: Memory access

WB: Write back

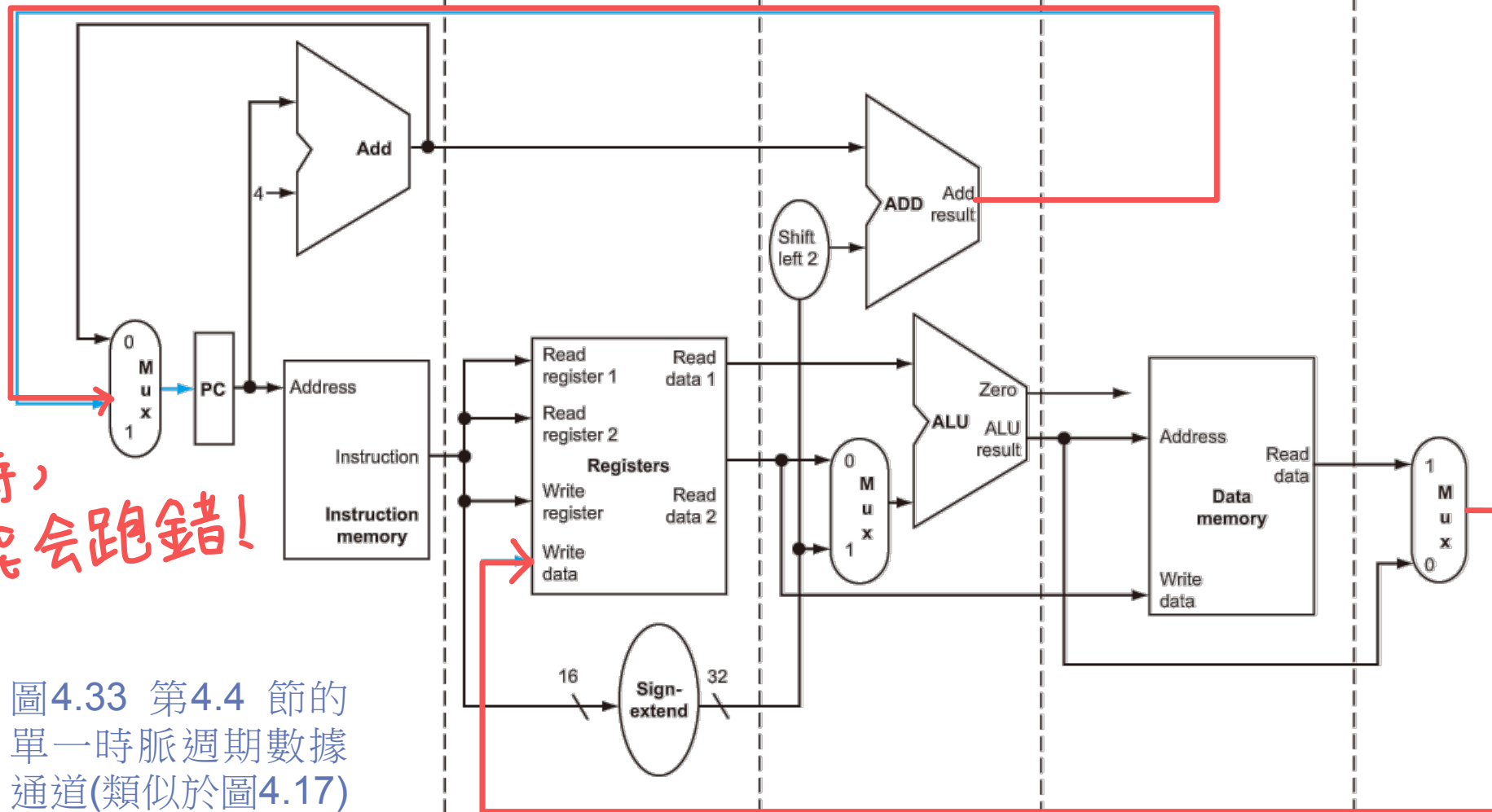
IF

ID

EXE

MEM

WB



FeedBack時，
PC 可能会跑錯！

圖4.33 第4.4 節的
單一時脈週期數據
通道(類似於圖4.17)

Write Back 時可能会寫到別指令！

圖4.33 第4.4 節的單一時脈週期數據通道(類似於圖4.17)

指令的每個步驟都能由左至右對應到數據通道中。例外的是以灰色表示的程式計數器(PC)的更新以及將ALU的結果或左方的記憶體的数据寫回暫存器檔案中的寫回(wite-back)。(一般我們會用灰色的線來代表控制訊號，但在本圖中灰色線則是代表數據線。)

4.6 管道化數據通道及控制

- ▣ 指令與數據大致由左向右經過五個階段的移動來完成執行。兩個例外：
 - 寫回階段，其將結果儲存回位於數據通道中間的暫存器檔案中
 - 下一個PC值的選擇，在遞增的PC以及由MEM階段來的分支位址間選擇
 - 注意上述第一種右到左的數據移動可能導致數據危障而第二種可能導致控制危障
- ▣ 圖4.34 以在時間軸上畫出所有指令各自使用數據通道的方式來表示圖4.27 中指令的執行過程

圖4.34 指令用圖4.33 的單一時脈週期數據路徑假設以管道化的方式來執行

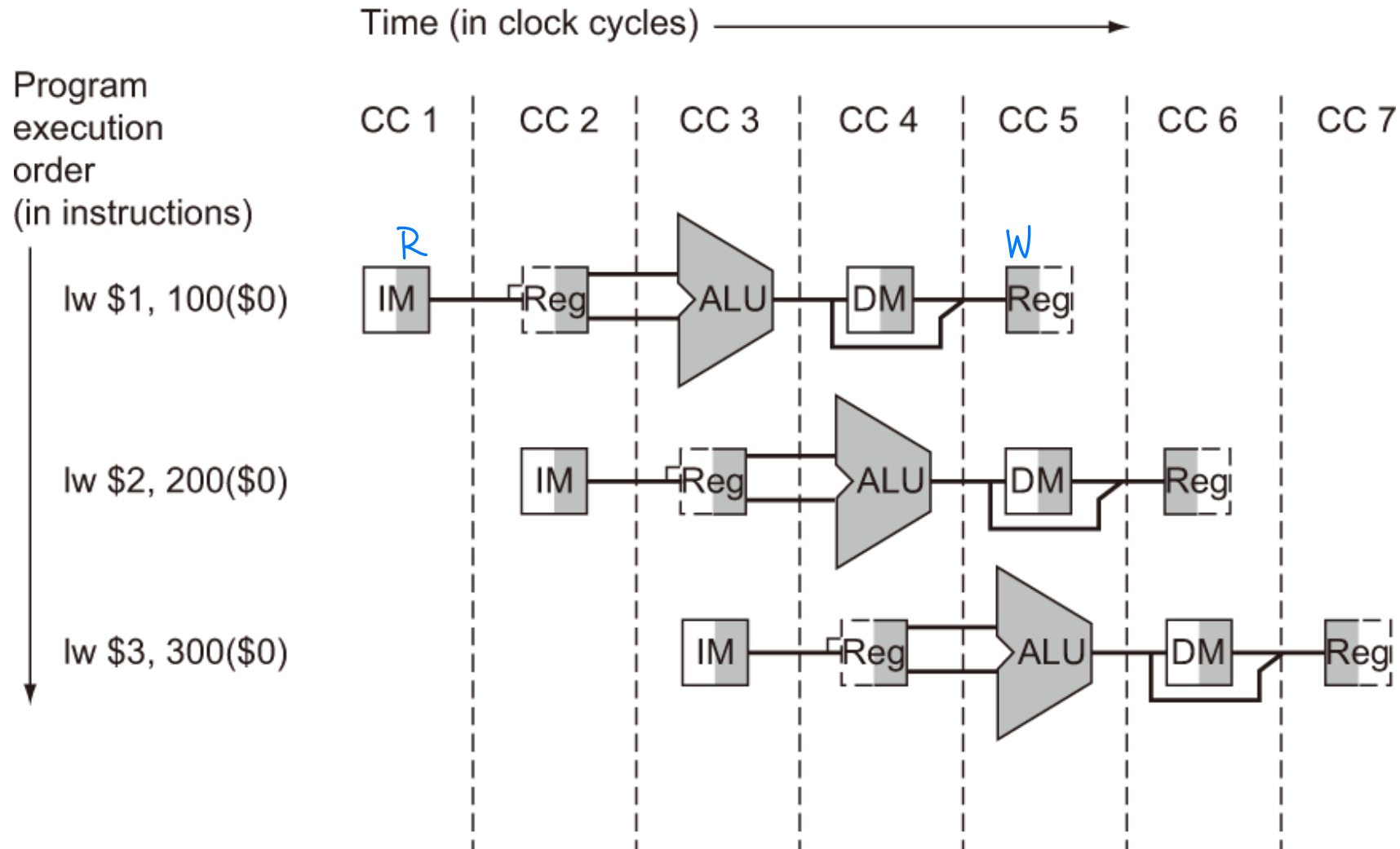


圖4.34 指令用圖4.33 的單一時脈週期數據路徑假設以管道化的方式來執行

這張圖類似於圖4.28 到圖4.30，假設每一個指令擁有自己的數據通道，並且將使用到的部分塗上陰影。不同於那些圖的是每個階段都標示出該級中使用到的對應到圖4.33 中數據通道該部分的實體資源。*IM* 代表指令擷取階段中的指令記憶體和程式計數器，*Reg* 代表指令解碼/暫存器檔案讀取階段(ID)中的暫存器檔案和符號延伸器，以及其他。為了維持適當的時間順序，暫存器檔案分成邏輯上的兩個部分：在暫存器擷取(ID)階段被讀取的暫存器和寫回(WB)階段被寫回的暫存器。在ID 階段內暫存器陣列並沒有被寫入，所以我們表示的方法是將左半邊用虛線表示且不加陰影；而在WB 階段內暫存器陣列並沒有被讀取，所以是將右半邊用虛線表示且不加陰影。我們如前假設暫存器檔案在前半時脈週期被寫入，而在後半時脈週期被讀取

4.6 管道化數據通道及控制

- ▣ 加入暫存器來保存數據，以便一個數據通道的不同部分在執行指令時可分別被使用
- ▣ 圖4.35 顯示強調管道暫存器(pipeline registers)的管道化數據通道
 - 這些暫存器以前後兩個階段的名稱來命名

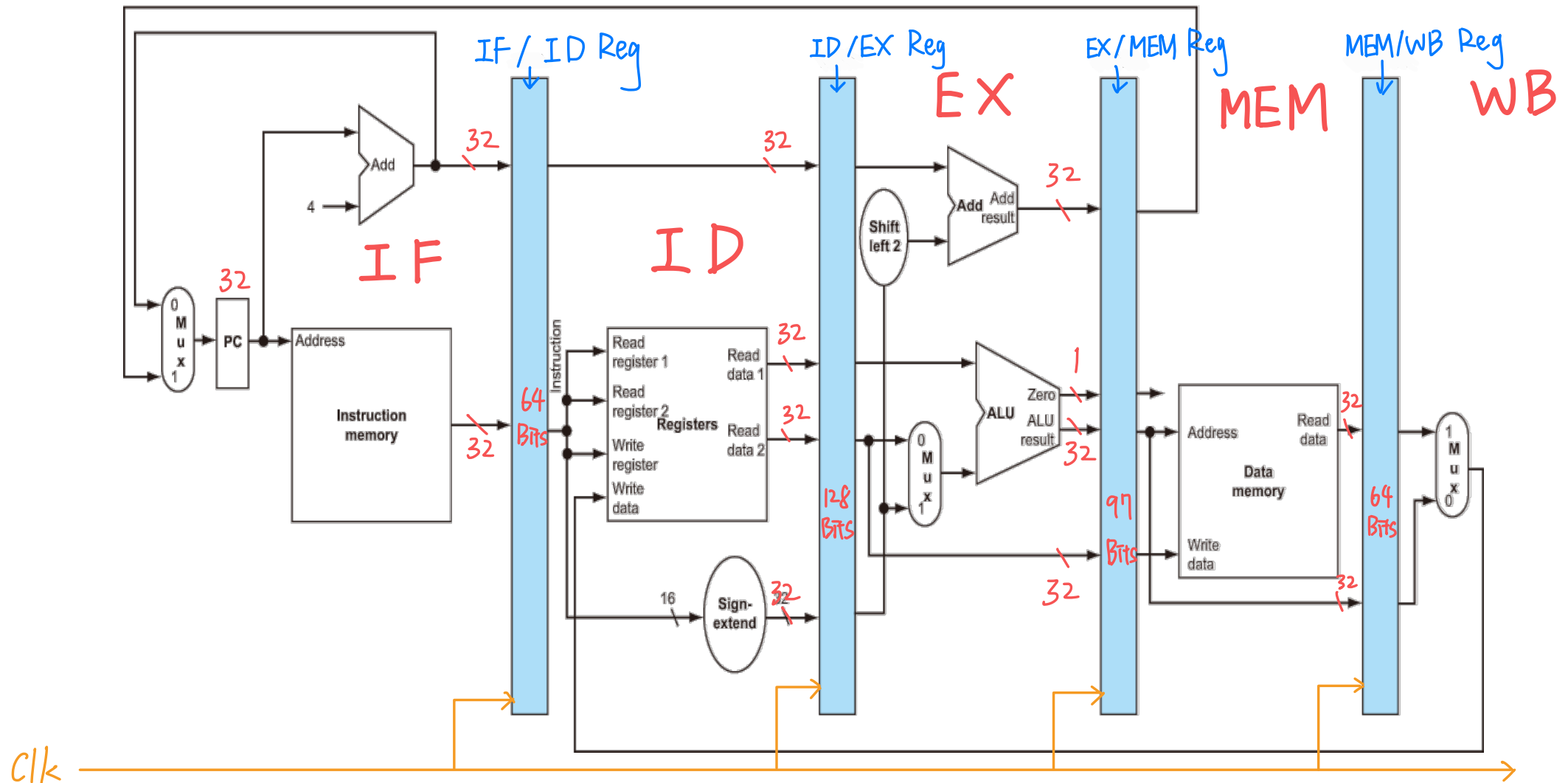


圖4.35 圖4.33 中數據通道的管道化版本

圖4.35 圖4.33 中數據通道的管道化版本

灰色的管道暫存器(**pipeline registers**)將每個管道階段隔開。它們以所隔開的階段來標示；例如第一個被標示為**IF/ID**，因為它將指令擷取和指令解碼階段隔開。這些暫存器必須夠寬以儲存所有會經過它的資料。例如**IF/ID** 暫存器必須是**64** 位元寬，因為它必須儲存從記憶體擷取的**32** 位元指令以及遞增後的**32** 位元**PC** 位址。目前另外的三個管道暫存器各包含**128**、**97** 和**64** 位元，不過我們會在本章稍後逐漸加寬這些暫存器

4.6 管道化數據通道及控制

- 圖4.36到圖4.38，顯示載入指令通過管道中五個階段時數據通道中強調出有動作的部分的情形
 - 任何在後方管道階段會使用到的資訊都必須透過管道暫存器來傳遞給各該階段
 - 數據通道中的每個邏輯元件如指令記憶體、暫存器讀取埠、ALU、數據記憶體和暫存器寫入埠只能用於唯一的管道階段期間，否則會發生結構危障
- 現在我們來發現在載入指令設計中的一個錯誤。你看出來了嗎？
 - 寫入暫存器的編號！

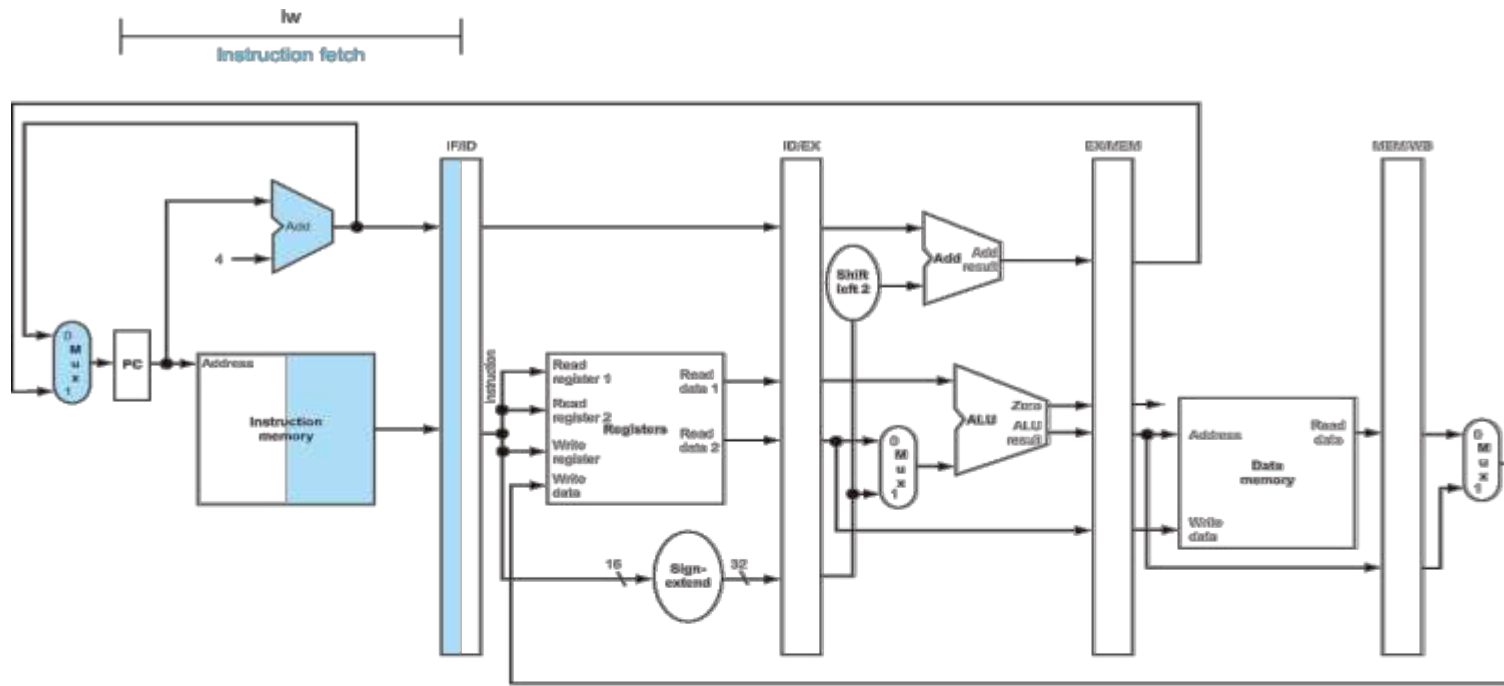


圖 4.36 IF 和 ID：指令的第一和第二個階段，並將圖4.35的數據通道中運作的部分著灰色

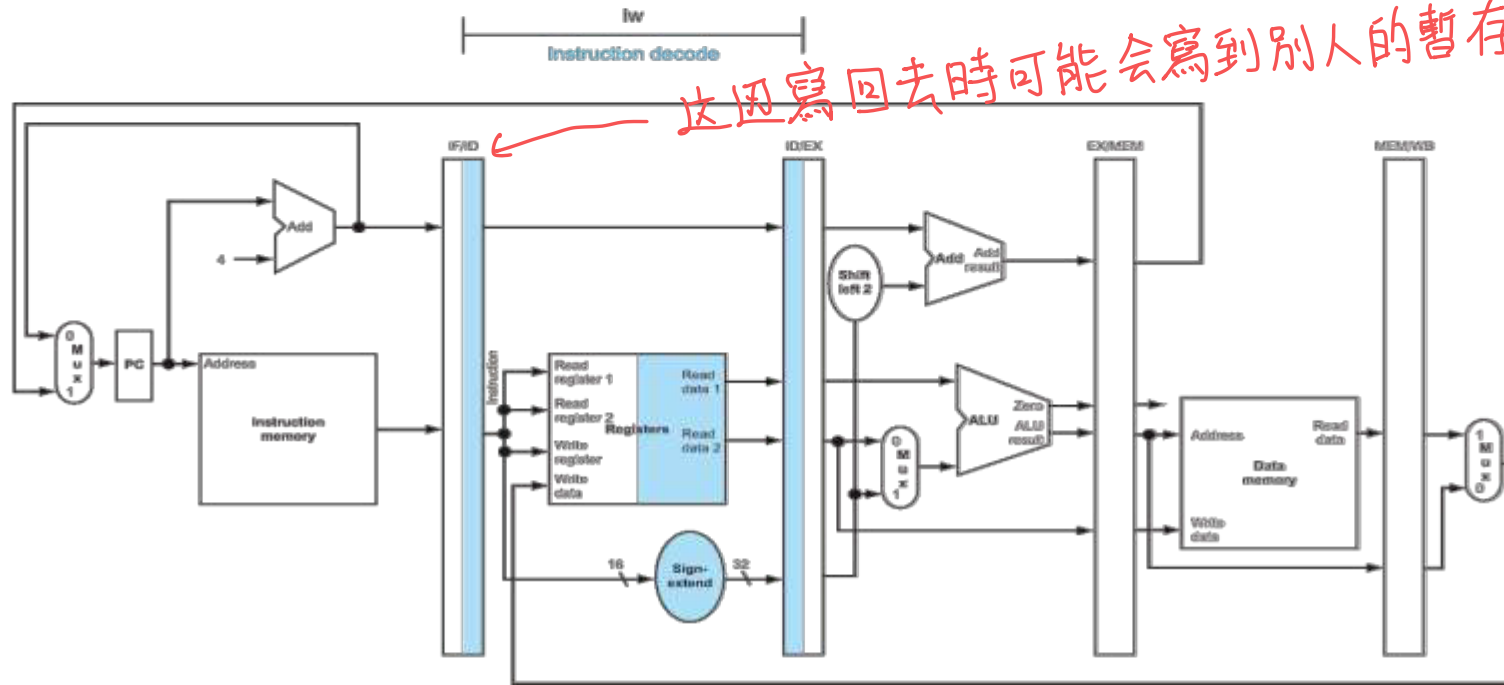


圖4.36 IF 和ID：指令的第一和第二個階段，並將圖4.35 的數據通道中運作的部分著灰色

著色的規則和圖4.28 中所使用的相同。如第4.2 節所述，因為暫存器的內容只會在時脈的邊緣才會改變，所以其讀取與寫入並不會發生混淆。雖然載入指令只需要在第2 級上方的暫存器內容，但是處理器並不知道目前被解碼的是什麼指令，所以符號延伸16位元的結果以及讀取的兩個暫存器內容都會被儲存到ID/EX管道暫存器中。雖然我們不全需要這三個運算元，但是三個都保存可以簡化控制

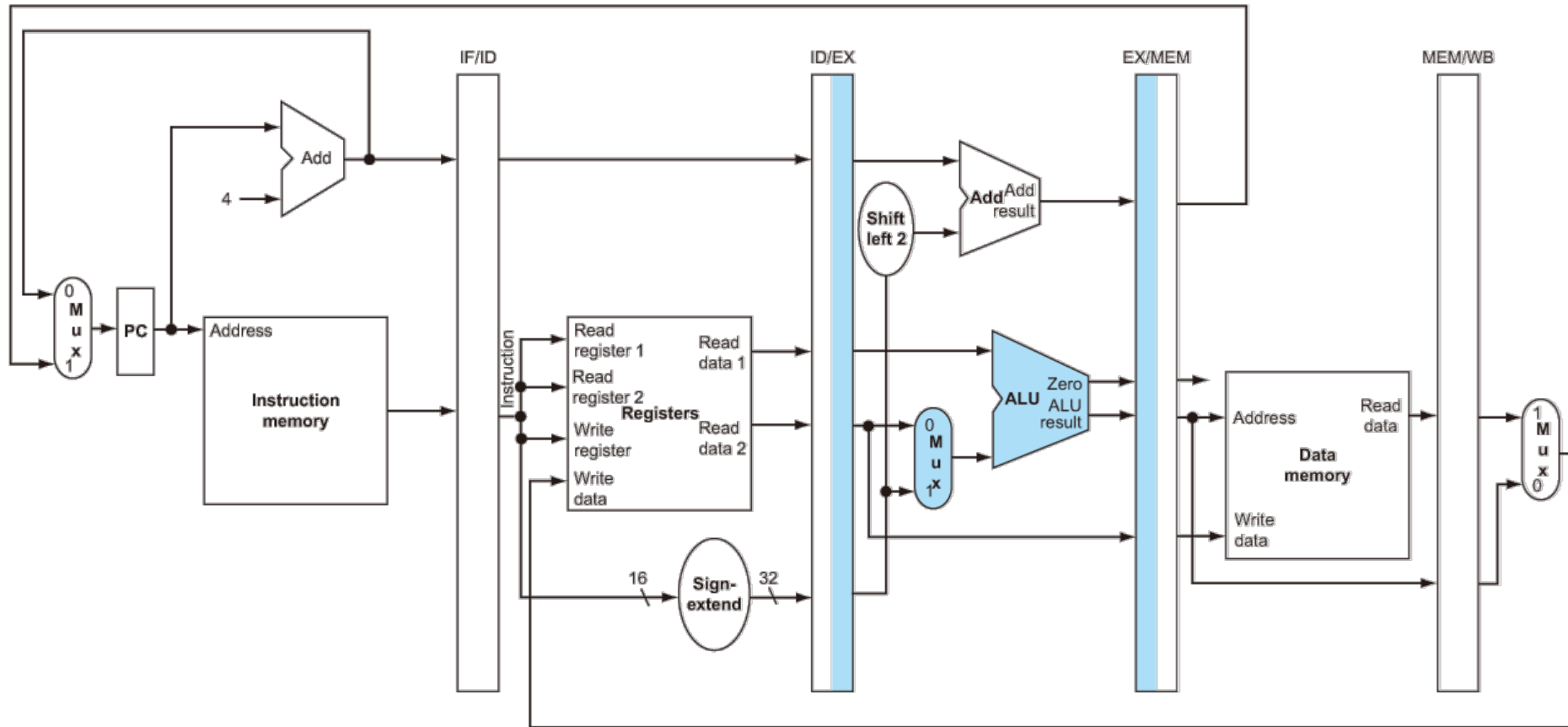


圖4.37 EX：載入指令的第三個管道階段，並將圖4.35 的數據通道中在這個管道階段用到的部分著灰色

暫存器的值和符號延伸後的立即值相加，並將和放置在EX/MEM 管道暫存器中

資料往前送，但目標暫存器沒有一起跑！

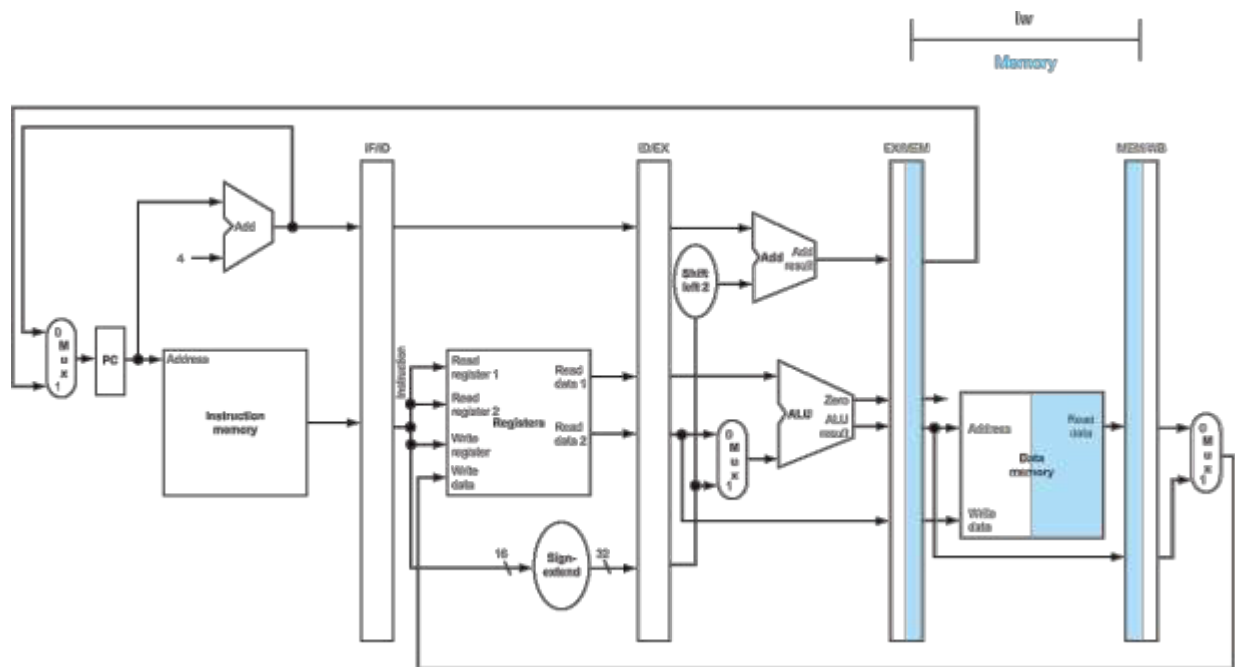
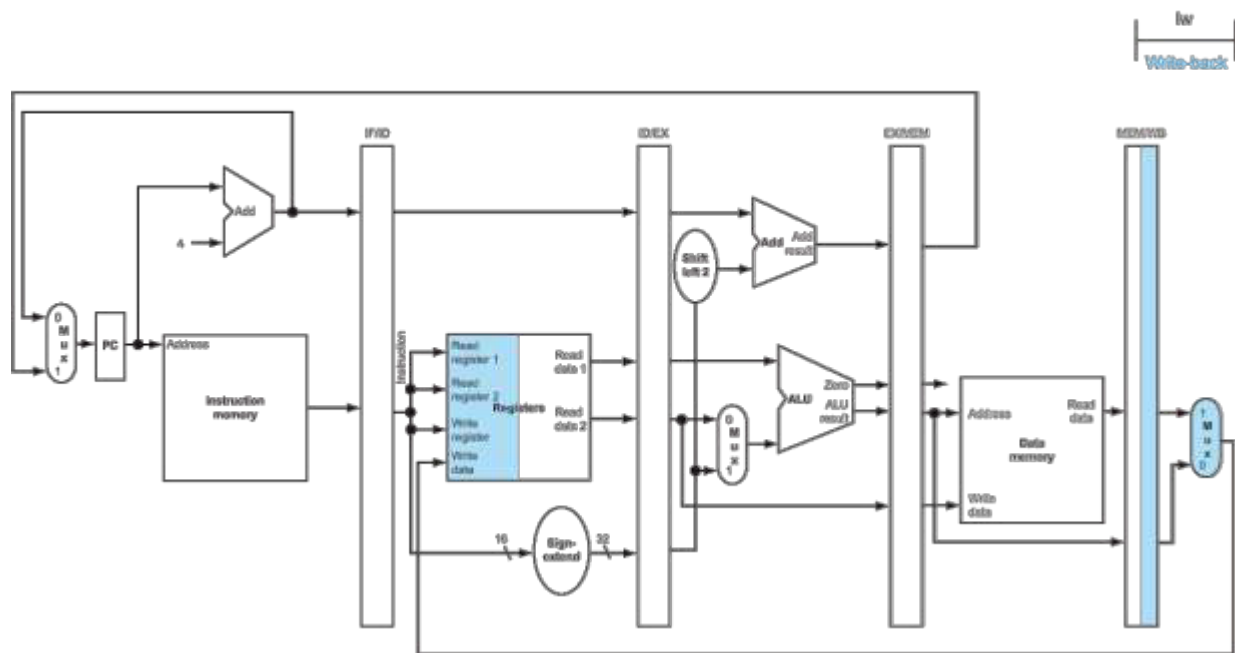


圖 4.38 MEM 和 WB：載入指令的第四個和第五個管道階段，並將圖 4.35 的數據通道中在該管道階段用到的部分著灰色。

使用 **EX/MEM** 管道暫存器中的位址來讀取數據記憶體，並將數據放置在 **MEM/WB** 管道暫存器中。接下來，數據從 **MEM/WB** 管道暫存器讀取並寫入圖中間的暫存器檔案中。注意：這個設計有一個錯誤並將於圖 4.41 中修正。

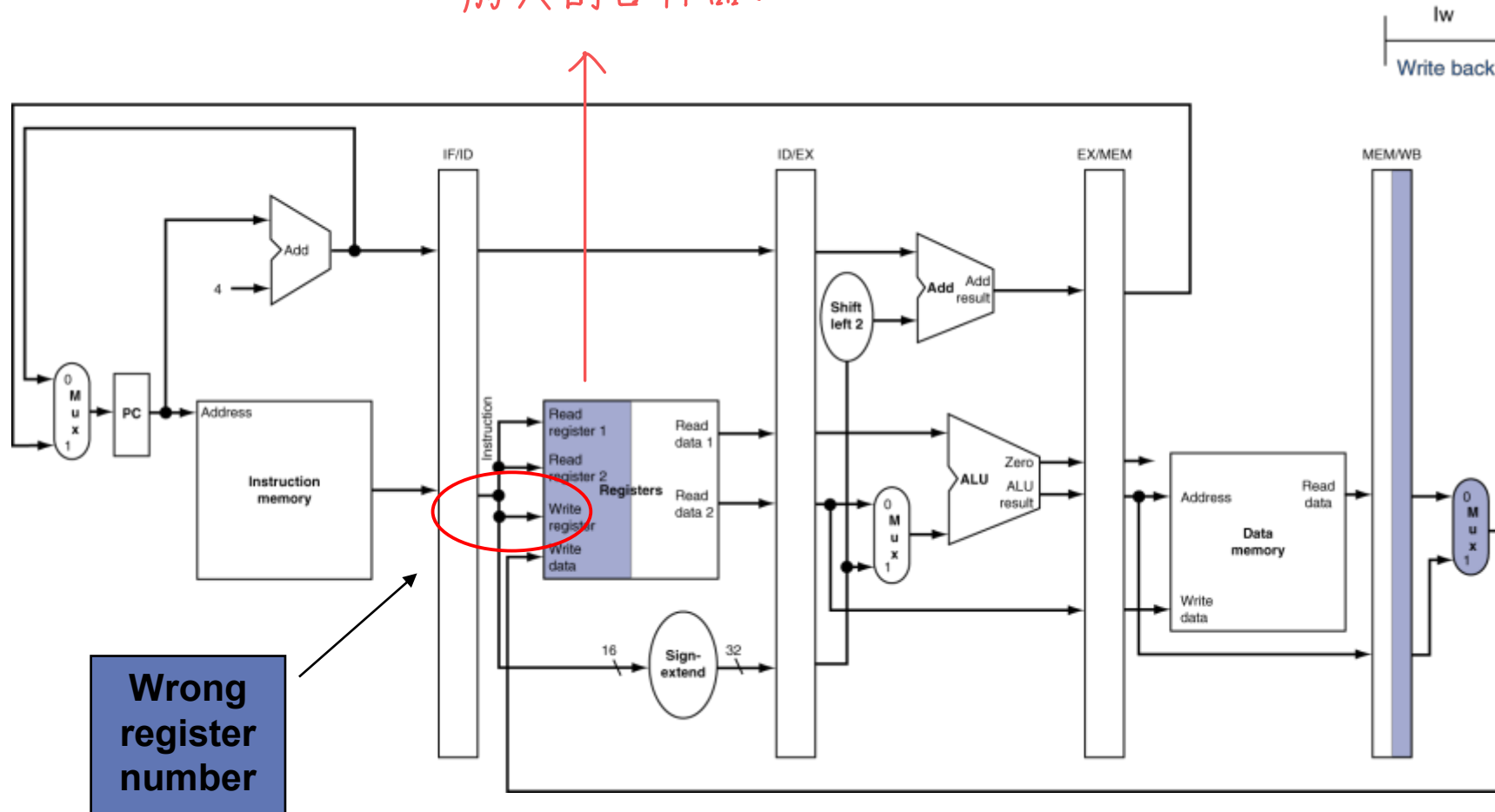


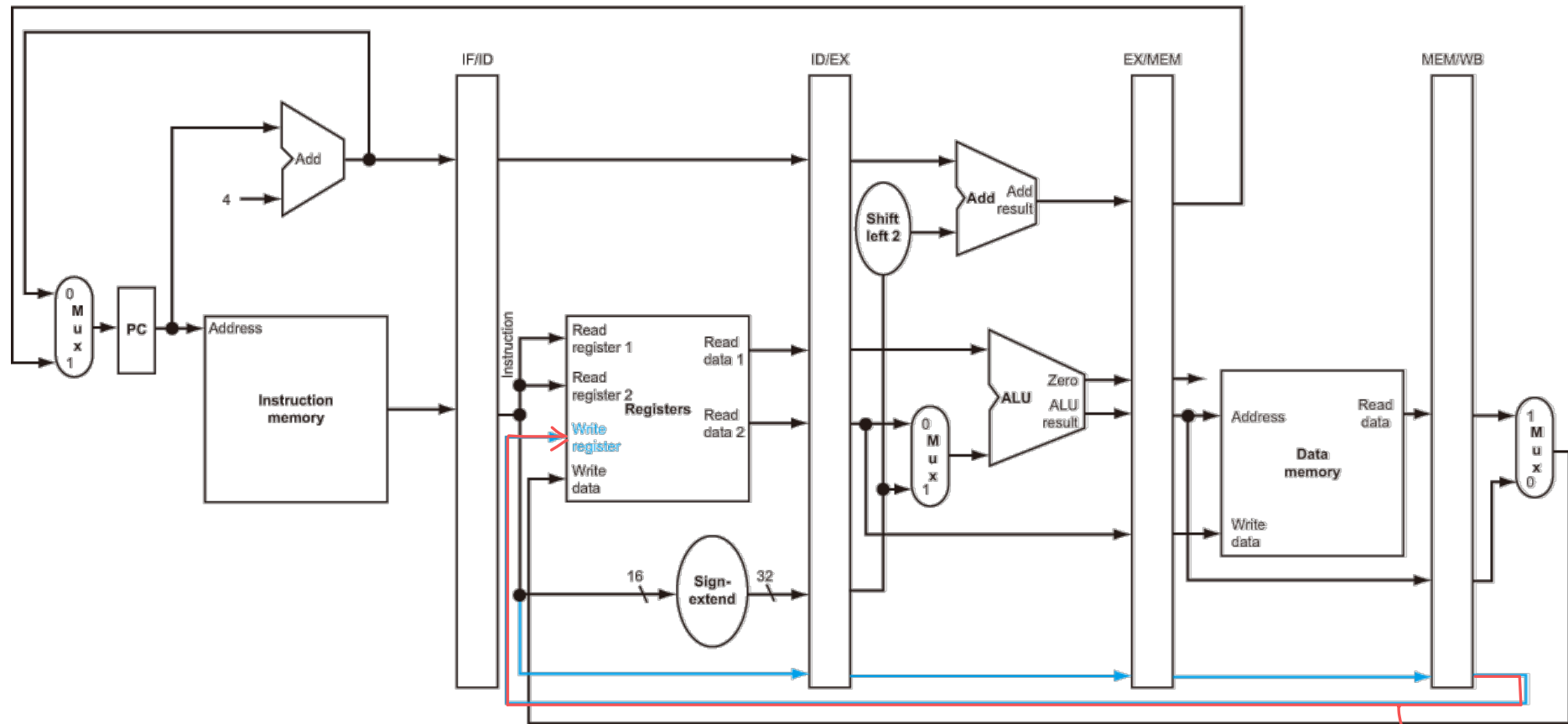
4.6 管道化數據通道及控制

- 我們需要將IF 階段所擷取到的指令一路上都保存下來，因此每個管道暫存器會保留一道指令在目前這一級及之後各級所需的內容
- 圖4.41 表示了正確的數據通

WB for Load

这边可能寫到
別人的暫存器!





把目標Reg的值
帶著一起走

圖4.41 為了正確處理載入指令而修正後的管道化數據通道

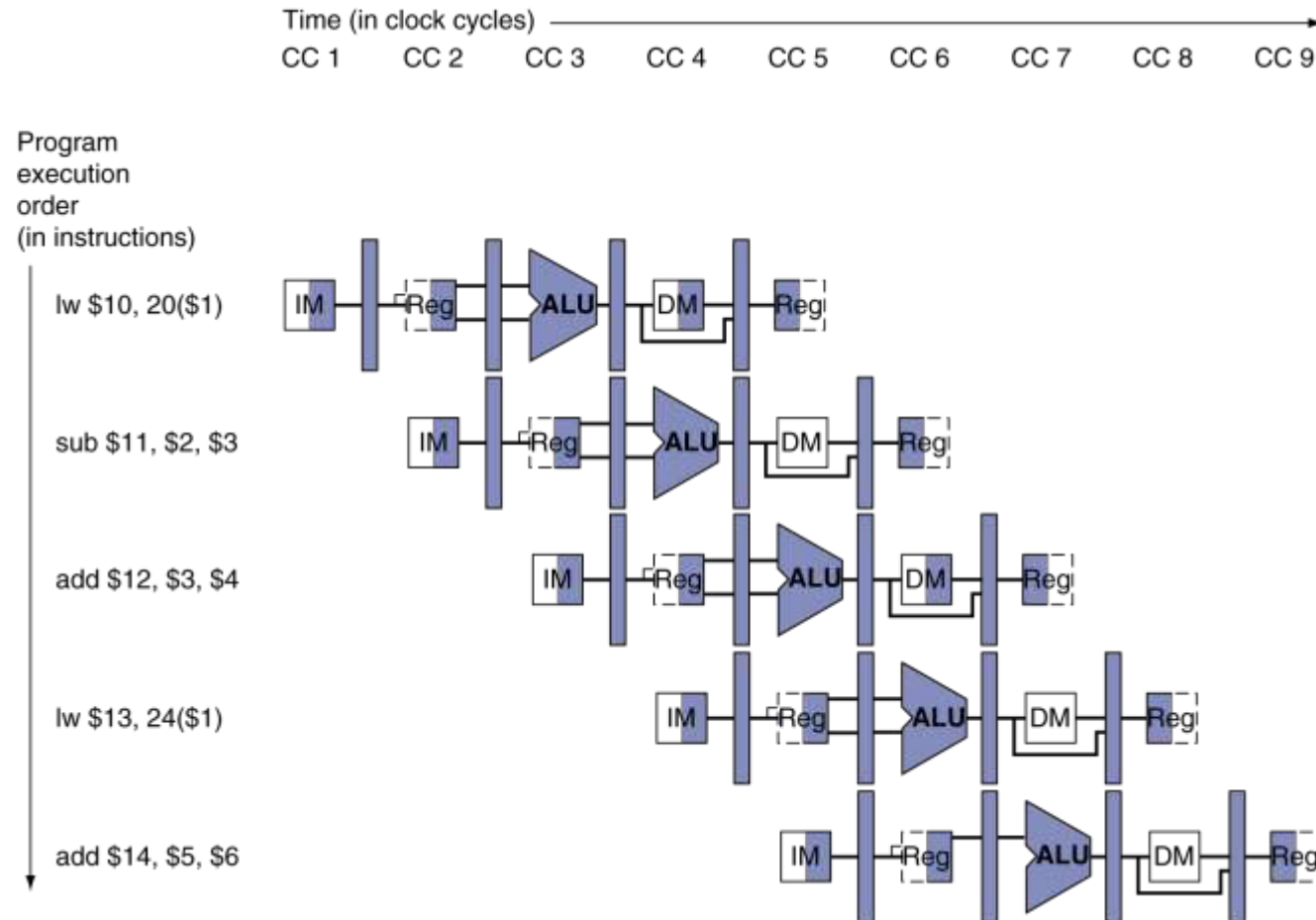
現在寫入暫存器編號與數據都來自MEM/WB 管道暫存器。這個暫存器編號從ID 管道階段傳送過來直到它到達MEM/WB 管道暫存器為止，使得最後三個管道暫存器中各增加5 個位元。圖中新增的路徑以灰色表示

以圖形來表示管道

- ▣ 有兩種基本型態的圖形化表示法
 - 如同圖4.34 的多個時脈週期管道圖(multiple-clock-cycle pipeline diagram)
 - 如同圖4.36 到圖4.40 的單一時脈週期管道圖

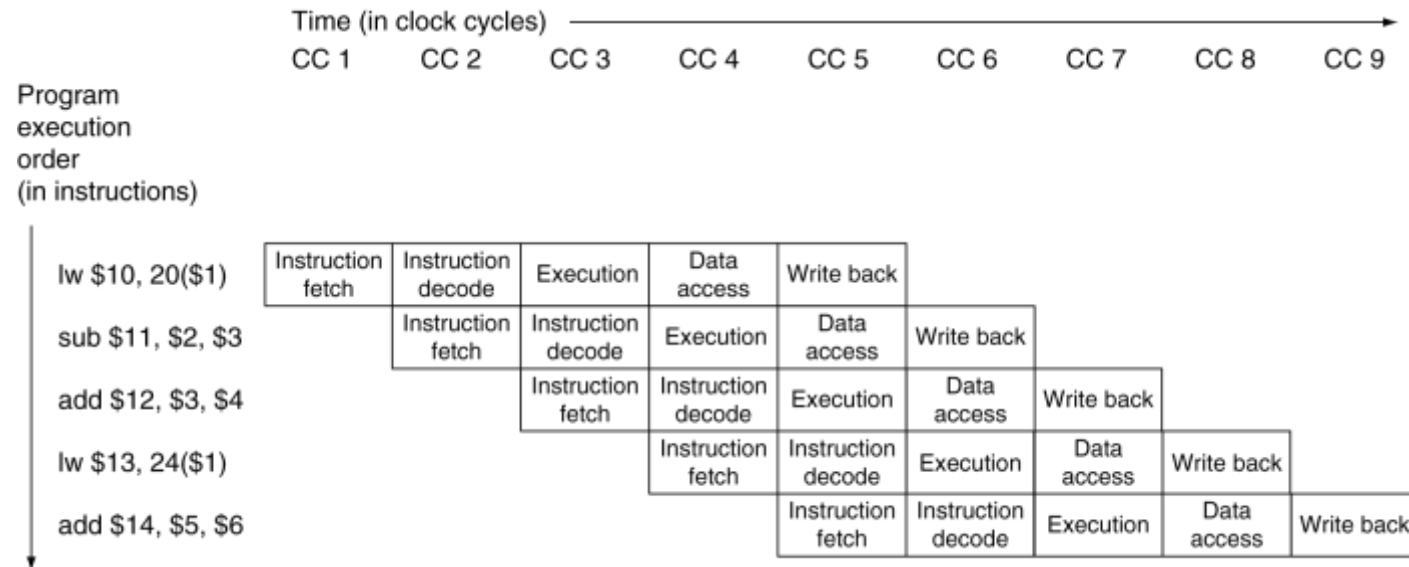
Multi-Cycle Pipeline Diagram

- Form showing resource usage



Multi-Cycle Pipeline Diagram

▣ Traditional form



管道化中的控制

- 第一步是在既有的數據通道中的控制訊號線作標記。
 - 圖4.46 表示這些控制訊號線。

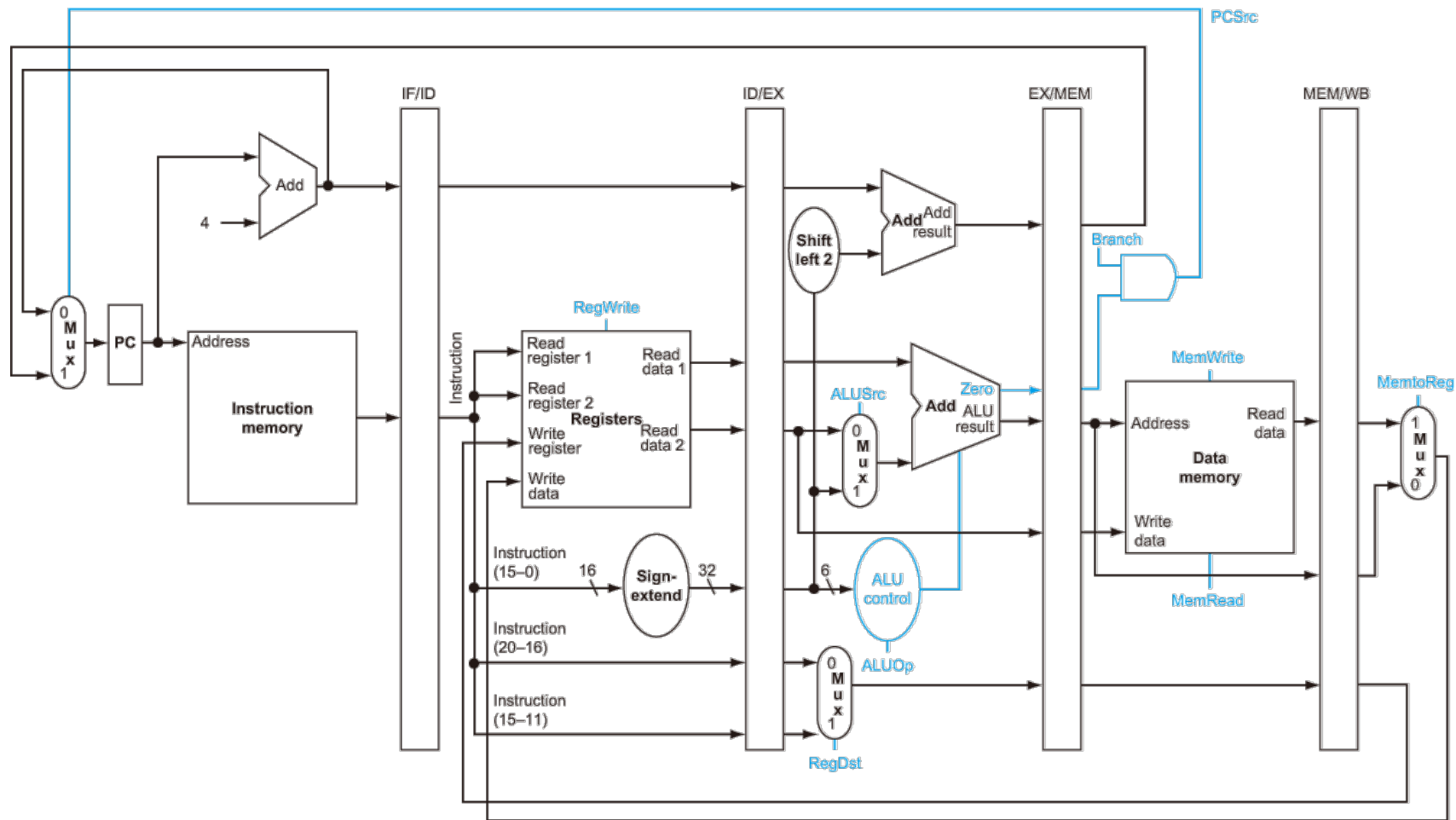


圖4.46 標出控制訊號後的圖4.41 中的管道化數據通道

此數據通道加上了4.4 節中的PC 來源控制邏輯、目的暫存器編號以及ALU 控制。注意現在我們在EX 階段中需要指令的6 位元的功能碼欄位來作為ALU 控制器的輸入，所以這些位元也必須包括在ID/EX 管道暫存器中。回想一下這6 位元其實就是立即值欄位中最低的6 個位元，所以ID/EX 管道暫存器可以由立即值欄位來提供它們，因為符號延伸後這些位元仍是不變的

管道化中的控制

▣ 圖4.47 到圖4.49 中重新將這些重要的資訊呈現

指令運作碼	ALUOp	指令運作	功能欄位	需要的 ALU 動作	ALU 控制輸入
LW	00	載入字組	XXXXXX	加	0010
SW	00	儲存字組	XXXXXX	加	0010
Branch equal	01	若等於則分支	XXXXXX	減	0110
R 型	10	加	100000	加	0010
R 型	10	減	100010	減	0110
R 型	10	及	100100	及	0000
R 型	10	或	100101	或	0001
R 型	10	若小於則設定	101010	若小於則設定	0111

func Don't Care!

圖4.47 此即圖4.12

R-Type 要參考 func

這個圖表示ALU 控制位元如何根據ALUOp 控制位元與R 型指令的不同功能碼來設定

訊號名稱	未被設定時的效果	被設定時的效果
RegDst	Write register 的目的暫存器編號來自 rt 欄位(指令的 20:16 位元)。	Write register 目的暫存器編號來自 rd 欄位(指令的 15:11 位元)。
RegWrite	無。	將 Write data 輸入的值寫入 Write register 輸入所指定的暫存器中。
ALUSrc	ALU 的第二個運算元來自暫存器檔案第二輸出埠(Read data 2)。	ALU 的第二個運算元來自指令中最低的 16 個位元經過符號延伸後的值。
PCSrc	由計算 $PC + 4$ 的加法器的輸出來替換原本 PC 的值。	由計算分支目標位址的加法器的輸出來替換原本 PC 的值。
MemRead	無。	將數據記憶體中 address 輸入所指定位址的內容送至 Read data 輸出。
MemWrite	無。	將數據記憶體中 address 輸入所指定位址的內容置換為 Write data 輸入的值。
MemtoReg	暫存器 Write data 輸入的值來自 ALU。	暫存器 Write data 輸入的值來自數據記憶體。

圖4.48 此即圖4.16

七個控制訊號均在此定義。ALU 的控制線(ALUOp)定義在圖4.47 的第二行。當一個二選一多工器的一位元控制訊號被設定(asserted)時，多工器選擇對應到1 的輸入。若該控制訊號沒被設定時，多工器選擇對應到0 的輸入。注意PCSrc 是由圖4.46 中的AND 閘所控制。在Branch 訊號與ALU Zero 訊號都為1 時，PCSrc才是1；否則其為0。控制器只有在執行beq 時才會設定Branch 訊號；否則PCSrc 設為0

指令	執行 / 記憶體位址計算階段 控制訊號線				記憶體存取階段 控制訊號線			寫回階段 控制訊號線	
	Reg-Dst	ALU-Op1	ALU-Op0	ALU-Src	Branch	Mem-Read	Mem-Write	Reg-Write	Memto-Reg
R 格式	1	1	0	0	0	0	0	1	0
lw	0	0	0	1	0	1	0	1	1
sw	X	0	0	1	0	0	1	0	X
beq	X	0	1	0	1	0	0	0	X

圖4.49 控制線的訊號值與圖4.18 中相同，不過已經分別對應最後的三個管道階段分成三群

管道化中的控制

- 不需要特別的PC 寫入訊號
- IF/ID、ID/EX、EX/MEM 和MEM/WB 這些管道暫存器也不需要特別的寫入訊號 → 僅扮演資料移動的角色！
- 根據管道的五級，把控制訊號線分成五組：
 1. 指令擷取：並沒有什麼特別要控制的
 2. 指令解碼暫存器檔案讀取：並沒有需要設定的控制線
 3. 執行/記憶體位址計算：要被設定的控制訊號有RegDst、ALUOp以及ALUSrc
 4. 記憶體存取：要設定的控制訊號有Branch、MemRead與MemWrite

管道化中的控制

- 5. **寫回**：兩條控制訊號線。**MemtoReg** 決定要將ALU 運算結果或是來自記憶體的值送到暫存器檔案；而**RegWrite** 用來將所選的值寫入
 - 將數據通道管道化之後控制線的意義也相同 (每個 Instruction 產生的控制信號 要跟指令一起往前跑！)
- 控制單元的實作就是針對每個指令在每一級中設定這九條控制訊號線的值
 - 最簡單的方法就是將管道暫存器擴充以容納這些控制訊號
 - 圖4.50 顯示這些控制訊號隨著指令在管道中前進而用於適當的管道階段中

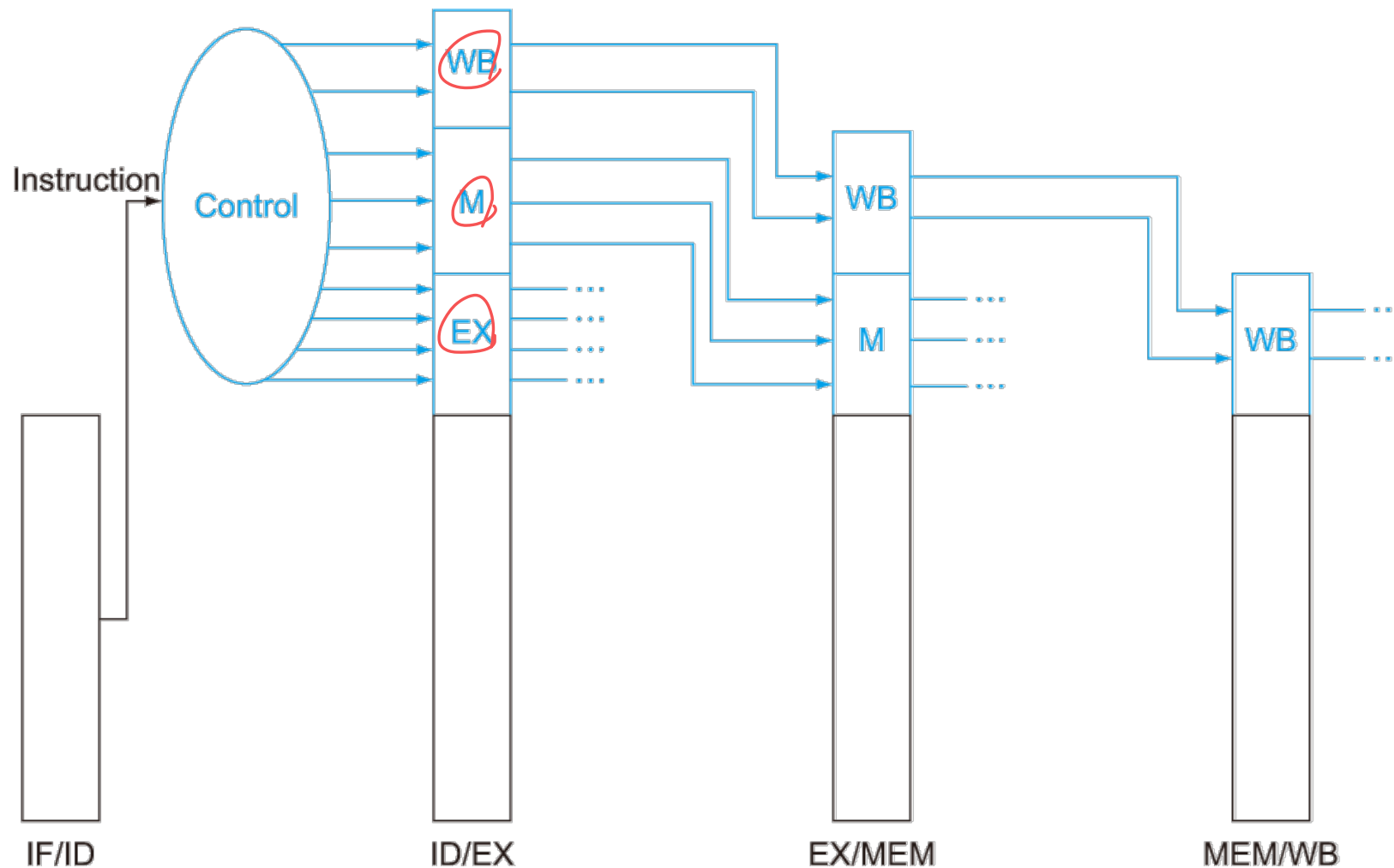


圖4.50 最後三級的控制訊號線

注意九條控制訊號線中有四條用於EX 級，其餘五條繼續送往EX/MEM 管道暫存器中擴充以保存它們的地方；三條用於MEM 級，最後的兩條則再送往MEM/WB 管道暫存器以便用於WB 級

管道化中的控制

■ 圖4.51表示完整數據通道

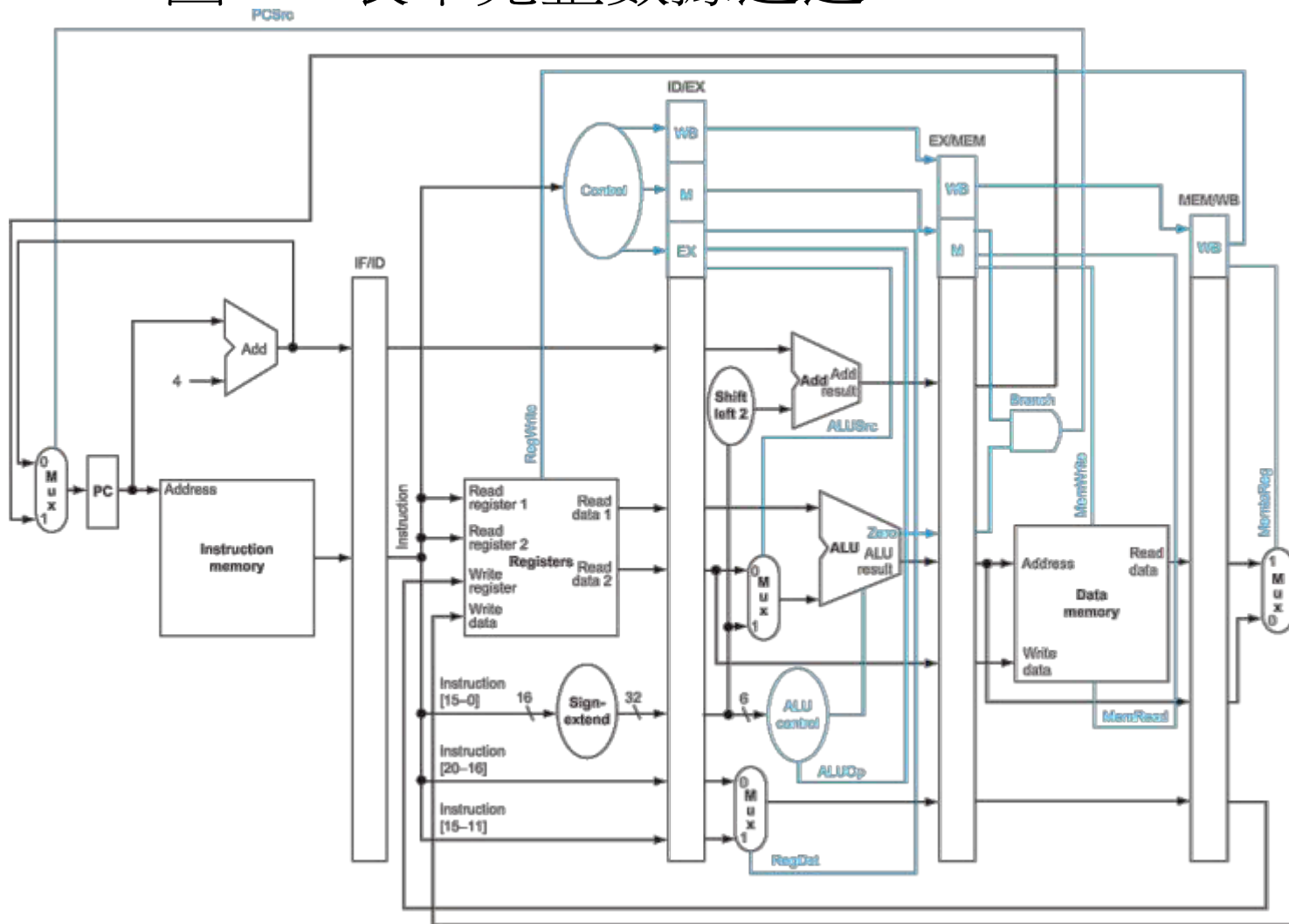


圖4.51 將控制訊號接到管道暫存器中儲存控制訊的部分後的圖4.46 中的管道化數據通道

最後三級的控制訊號在指令解碼的時候產生，然後放到ID/EX 管道暫存器中。每個管道階段的控制訊號被使用之後，剩下來的控制訊號就被送往下一個管道階段

4.7 數據危障：前饋或停滯

▣ 指令序列：

sub \$2, \$1, \$3 # 暫存器\$2被sub指令寫入

and \$12, \$2, \$5 # 第一個運算元(\$2)相依於sub指令

or \$13, \$6, \$2 # 第二個運算元(\$2)相依於sub指令

add \$14, \$2, \$2 # 暫存器\$14被add指令寫入

sw \$15, 100(\$2) # 記憶體基底位置(\$2)相依於sub指令

- 假設暫存器\$2 的值在減法指令運算前為10 以及運算後為-20
- 圖4.52 以多個時脈週期管道表示法來說明這些指令的執行上方顯示了\$2 暫存器的值

圖4.52 以簡化的數據通道來表示五道指令的序列在管道中的相依關係

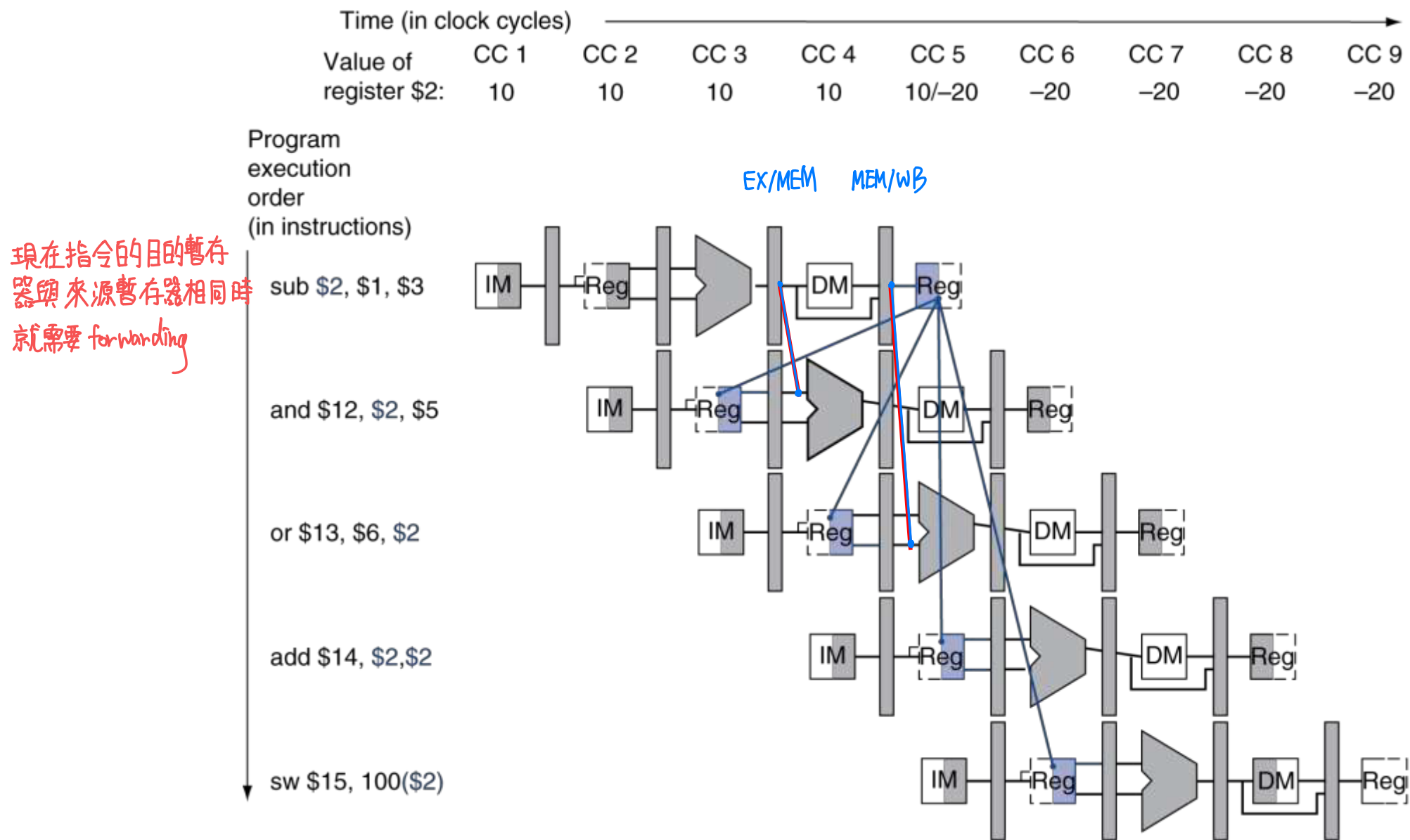


圖4.52 以簡化的數據通道來表示五道指令的序列在管道中的相依關係

所有相依的動作都以灰色來表示，而圖中最上面的**CC 1** 代表第一個時脈週期。第一個指令寫入**\$2**，而之後所有的指令都需要讀取**\$2**。這個暫存器在第五個時脈週期才被寫入，所以適當的值在第五個時脈週期前還不存在(在時脈週期中讀取暫存器時若同時有對該暫存器的寫入發生，則會得到該週期前半所寫入的值)。最上方的數據通道到下方各數據通道的拉線表示出相依關係。那些必須要逆著時間走的拉線就是管道數據危障

4.7 數據危障：前饋或停滯

- 最後的可能危障可以藉由暫存器檔案的硬體設計解決：當一個暫存器在一個週期內同時要讀取和寫入時會如何？
 - 假設寫入在時脈週期的前半完成，讀取在時脈週期的後半完成
- 如第4.5節中所提及，所需的結果在EX階段或第三個時脈週期結束時就出來了
 - and 與or 指令實際上是何時才需要這資料呢？
 - 在EX階段開始的時候，或是分別在第四、五個時脈週期

4.7 數據危障：前饋或停滯

- 立即將資料前饋到需要的地方，我們就可以不需停滯而執行這些指令

▣ 前饋是如何辦到的？

- 將管道暫存器的欄位命名。例如，“ID/EX.RegisterRs, ID/EX.RegisterRt”

✱偵測兩對的危障情形可表為：

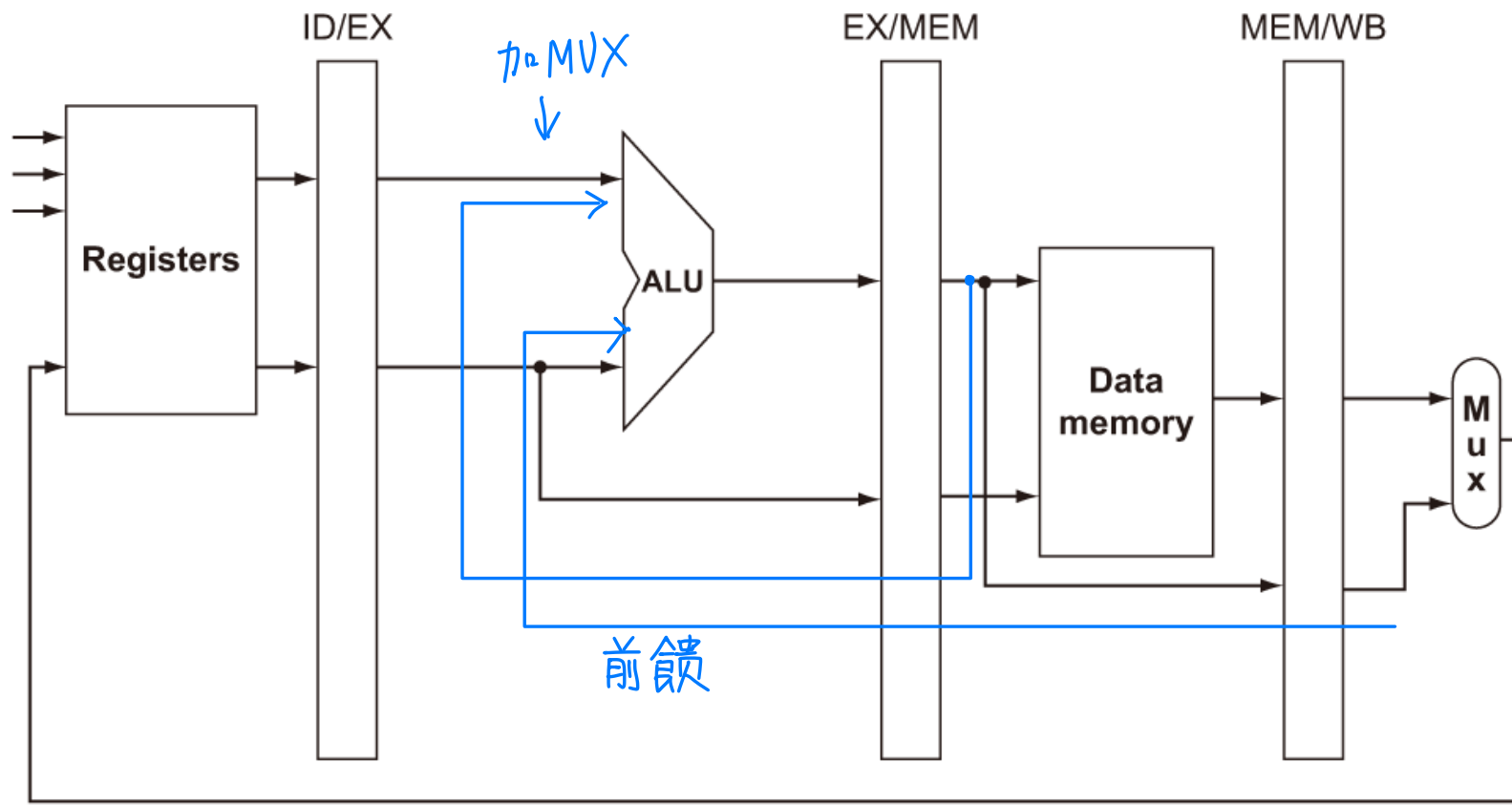


4.7 數據危障：前饋或停滯

- 另因為有些指令並不會寫入暫存器，所以上述的檢查法並不完全正確；

／ 前面指令是 R-Type 指令

- ① 檢查RegWrite 的值是否為真
 - ② 如果在管道中有指令以\$0 作為目的暫存器，我們要避免前饋可能非0 的值
- 前饋正確的資料：藉著加入ALU 輸入的多工器以及適當的控制
 - ▣ 圖4.54 顯示了清晰的ALU 與管道暫存器在加入前饋之前與之後的情形
 - ▣ 圖4.55 則表示位於EX級中用以選擇暫存器檔案或是前饋過來的值的ALU多工器其控制線的值

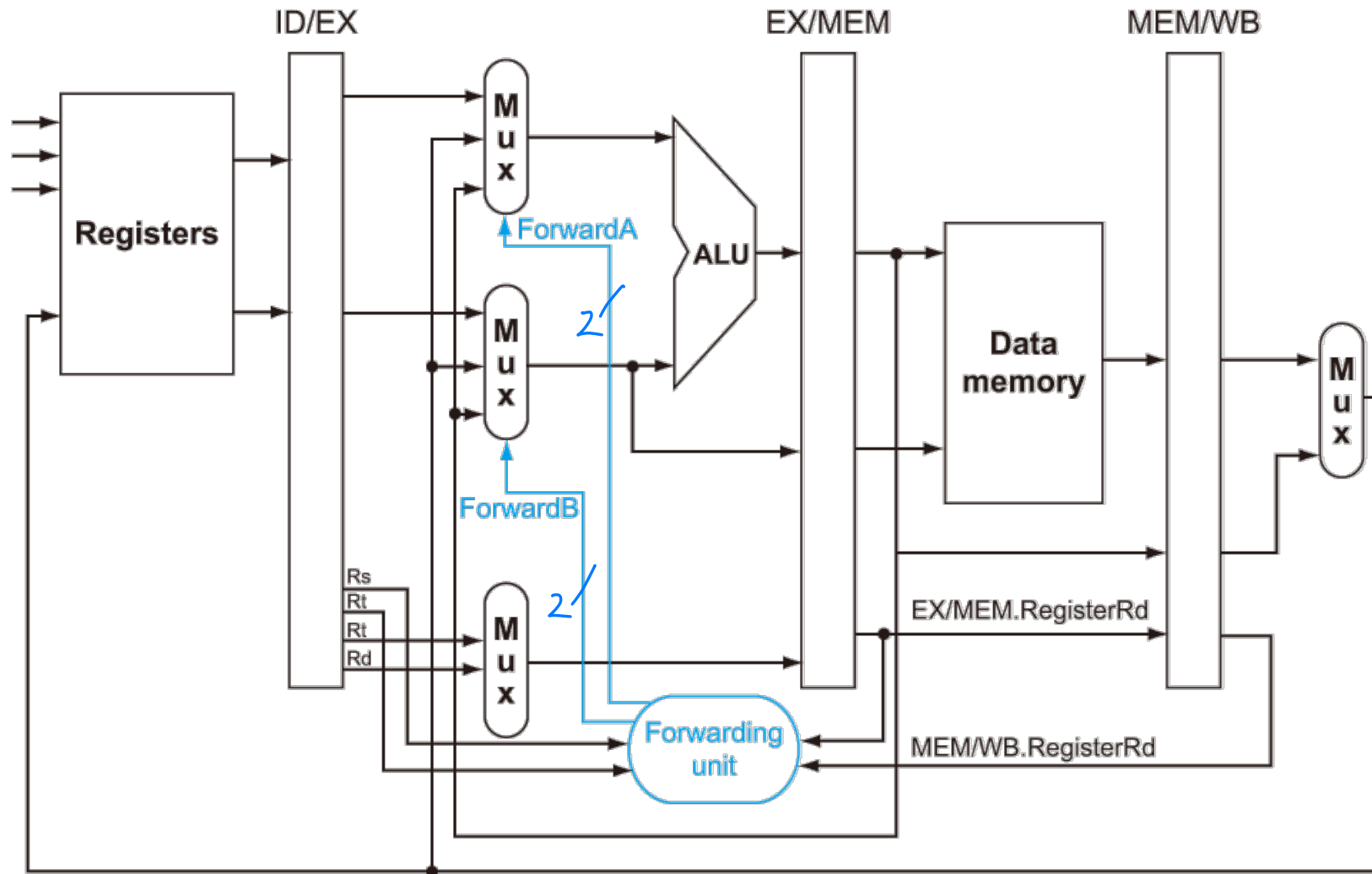


a. No forwarding

圖4.54 上圖是加入前饋功能前的ALU 和管道暫存器

下圖的多工器已經被擴充以加入前饋的路徑，而且也顯示了前饋單元。新加入的硬體以灰色呈現。本圖只是示意圖，省略了完整數據通道的細節如符號延伸的硬體。注意 **D/EX.RegisterRt** 的欄位出現了兩次。一次連接到多工器，另一次則是前饋單元，但這是相同的訊號。如前所述，這裡忽略了前饋儲存指令要存入的值。也注意這個機制對 **slt** 指令同樣可用

圖4.54 上圖是加入前饋功能前的ALU和管道暫存器

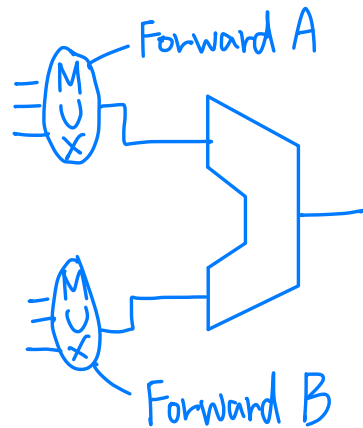


b. With forwarding

多工器控制	來 源	解 釋
ForwardA = 00	ID/EX	ALU 的第一個運算元來自暫存器檔案
ForwardA = 10	EX/MEM	ALU 的第一個運算元前饋自前一個 ALU 運算結果
ForwardA = 01	MEM/WB	ALU 的第一個運算元前饋自數據記憶體或再早的 ALU 運算結果
ForwardB = 00	ID/EX	ALU 的第二個運算元來自暫存器檔案
ForwardB = 10	EX/MEM	ALU 的第二個運算元前饋自前一個 ALU 運算結果
ForwardB = 01	MEM/WB	ALU 的第二個運算元前饋自數據記憶體或再早的 ALU 運算結果

圖4.55 圖4.54 中多工器的控制值

有關ALU 的另外一個輸入有號立即值(signed immediate)在本節最後的仔細深思中說明



Forwarding Conditions

sub \$s2 \$s1 \$s3
add \$s5 \$s2 \$s6

上一个目的是
下一个来源

EX hazard

R-Type

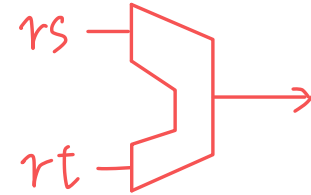
目的地非0

- if (EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0)
and (EX/MEM.RegisterRd = ID/EX.RegisterRs))

ForwardA = 10 $\rightarrow$ 從 EX / MEM forward 到 ALU

- if (EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0)
and (EX/MEM.RegisterRd = ID/EX.RegisterRt))

ForwardB = 10



sub \$s2 \$s1 \$s3
add \$s5 \$s3 \$s6
and \$s6 \$s4 \$s2

上二个目的是
下二个来源

MEM hazard

R-Type

- if (MEM/WB.RegWrite and (MEM/WB.RegisterRd $\neq$ 0)
and (MEM/WB.RegisterRd = ID/EX.RegisterRs))

ForwardA = 01 R-Type

- if (MEM/WB.RegWrite and (MEM/WB.RegisterRd $\neq$ 0)
and (MEM/WB.RegisterRd = ID/EX.RegisterRt))

ForwardB = 01

4.7 數據危障：前饋或停滯

- ▣ 如果WB 級中指令的結果以及MEM 級中指令的結果與ALU 級中指令的來源運算元同時存在數據危障
 - 此MEM 危障的控制應如下(並強調其中新增的部分)

Double Data Hazard

- Consider the sequence:



- Both hazards occur

- Want to use the most recent

- Revise MEM hazard condition

- Only fwd if EX hazard condition isn't true

執行階段的 forwarding 有優先權

Revised Forwarding Condition

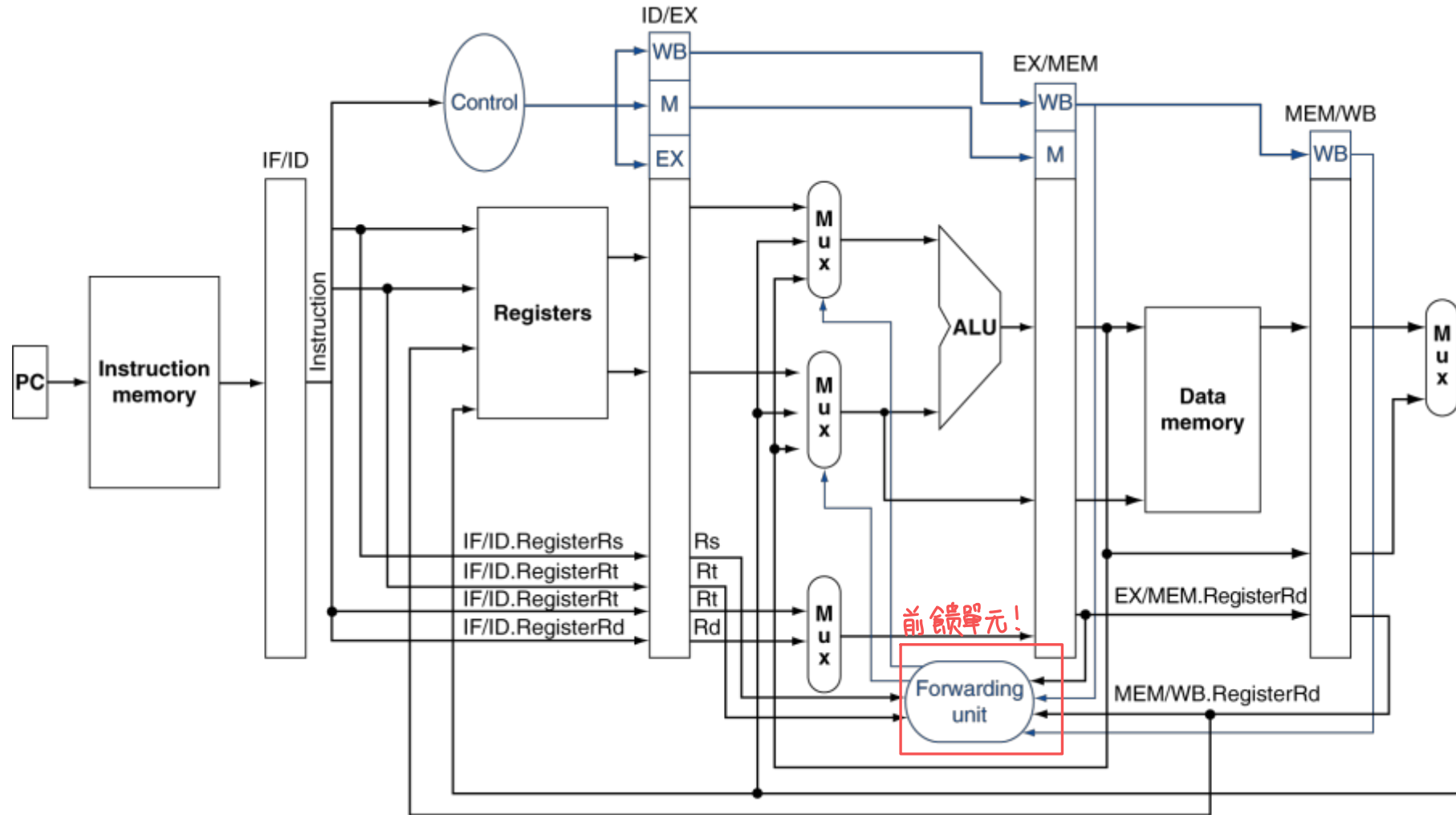
MEM hazard

- if (MEM/WB.RegWrite and (MEM/WB.RegisterRd $\neq$ 0)
and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0)
and (EX/MEM.RegisterRd = ID/EX.RegisterRs))
and (MEM/WB.RegisterRd = ID/EX.RegisterRs))
ForwardA = 01
- if (MEM/WB.RegWrite and (MEM/WB.RegisterRd $\neq$ 0)
and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0)
and (EX/MEM.RegisterRd = ID/EX.RegisterRt))
and (MEM/WB.RegisterRd = ID/EX.RegisterRt))
ForwardB = 01

Ensure 執行階段的 forwarding
不成立



Datapath with Forwarding



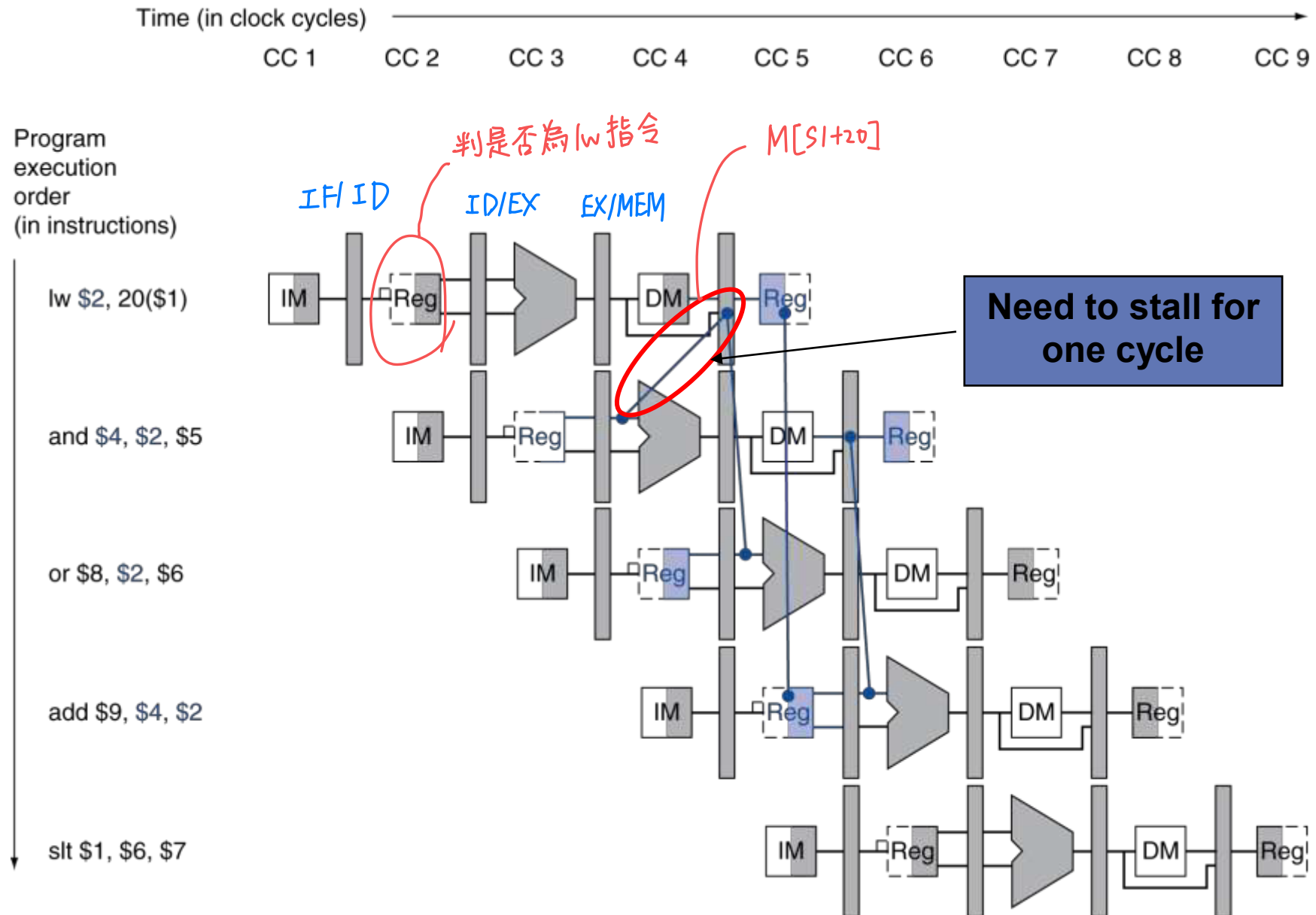
數據危障與停滯

- ▣ 當一個指令要讀取前一個載入指令寫入的暫存器時，前饋也無法完全解決
 - 圖4.58

圖4.58 管道化的指令序列

因為在載入指令與緊接在後的指令(**and**)之間的相依關係在時間上是逆著走的，這個危障不能靠前饋解決。因此，這種指令組合得由危障偵測單元來產生一個停滯

圖4.58 管道化的指令序列



數據危障與停滯

- 除了前饋單元之外，我們還需要一個危障偵測單元(hazard detection unit)。

是否為 lw

if (ID/EX.MemRead and
((ID/EX.RegisterRt = IF/ID.RegisterRs) or
(ID/EX.RegisterRt = IF/ID.RegisterRt)))
stall the pipeline(停滯管道) ← 插bubble(暫停)

*Assume lw \$s2, 20(\$s1)
(M[\$s1+20])*



- 第一行檢查指令是否為載入
- 後面兩行檢查EX 級中的載入指令其目的暫存器欄位是否與ID 級中指令的某一來源暫存器相符

How to Stall the Pipeline

▣ Force control values in ID/EX register

to 0

这三个阶段做nop

→ 控制信号强制设置为0

- EX, MEM and WB do nop (no-operation)

▣ Prevent update of PC and IF/ID register

→ PC、IF/ID 保護他們別動 (再做一次这个 Instruction)
→ 重拿一次 PC

- Using instruction is decoded again
- Following instruction is fetched again
- 1-cycle stall allows MEM to read data for 1w
 - Can subsequently forward to EX stage

圖4.59 停滯實際上如何插入管道

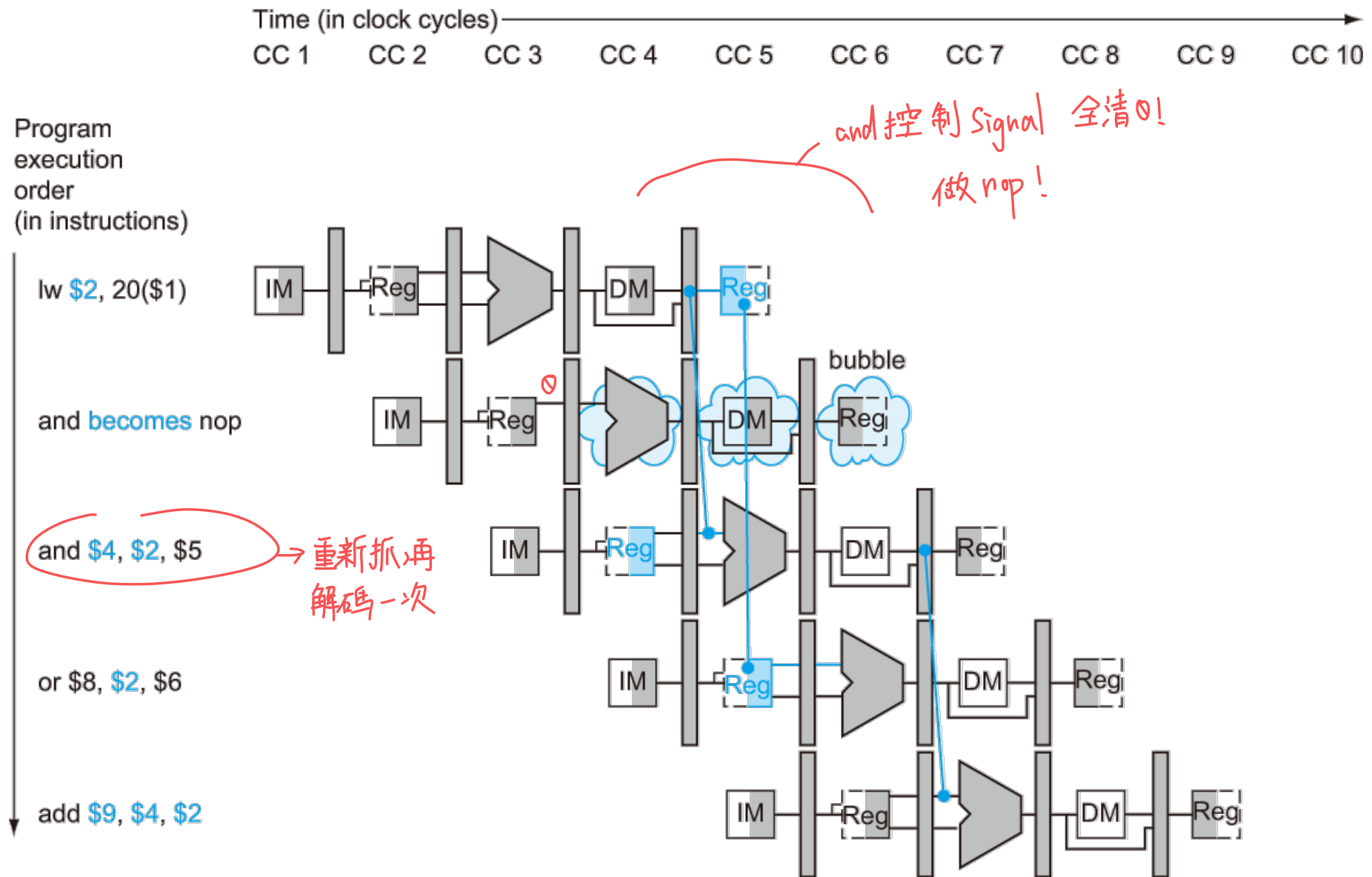


圖4.59 停滯實際上如何插入管道

藉由將**and** 指令改成**nop**，一個氣泡在時脈週期4 開始加了進來。注意**and** 指令在時脈週期2 與3 確實被擷取與解碼，但是其**EX** 級則是延遲到時脈週期5 (相對於未停滯時的時脈週期4)。同樣地**OR** 指令在時脈週期3 被擷取，但是其**ID** 級則延遲到時脈週期5 (相對於未停滯時的時脈週期4)。在插入氣泡之後，所有的相依關係都順著時間走，於是不會有危障發生

圖4.60 表示出前饋使用的兩個多工器、危障偵測單元以及前饋單元的管道化控制概覽

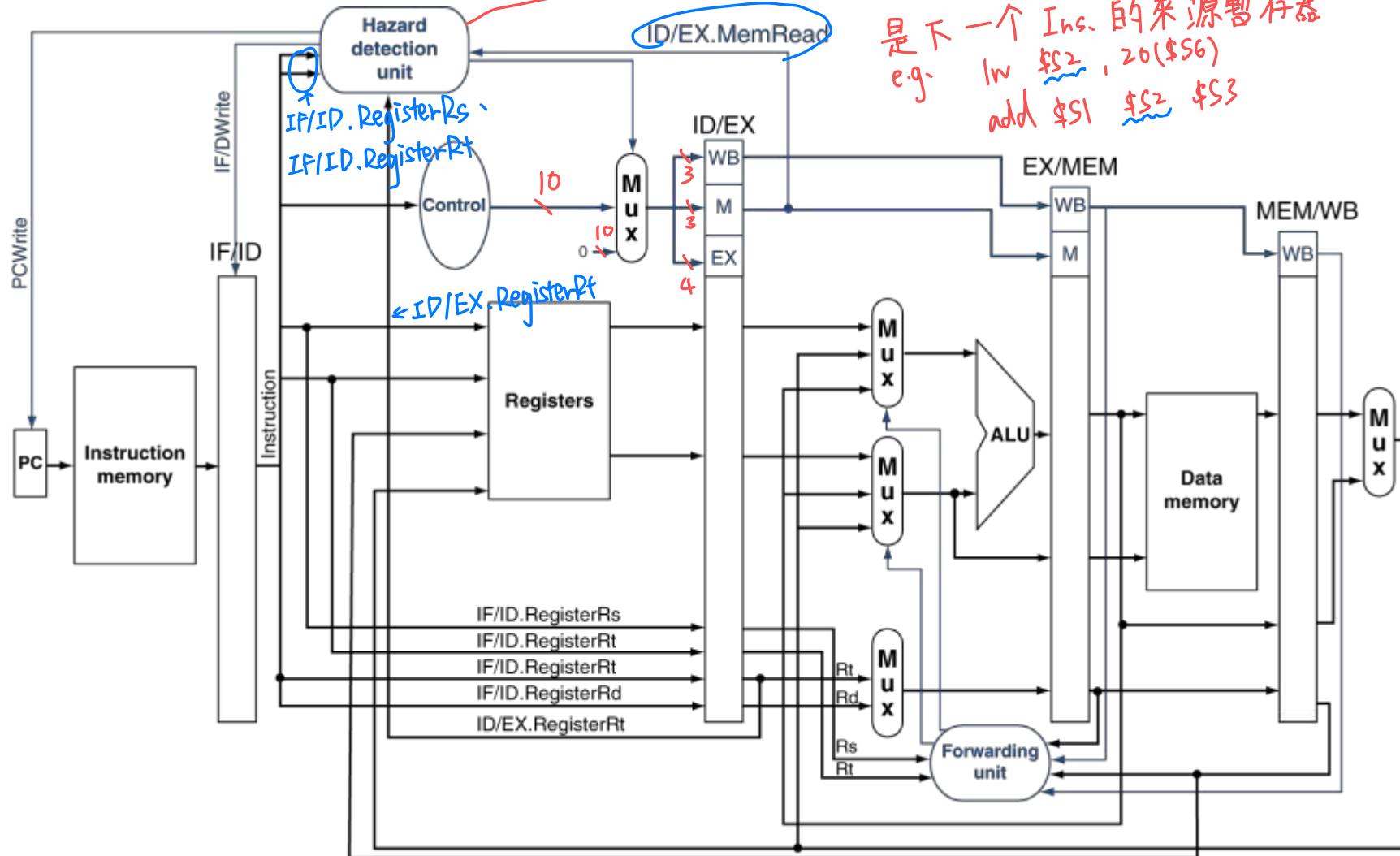


圖4.60 表示出前饋使用的兩個多工器、危障偵測單元以及前饋單元的管道化控制概覽

雖然ID 與EX 級被簡化了(省略了符號延伸的立即值部分與分支邏輯)，這個圖表示出前饋硬體的基本需求

雖然編譯器一般都依賴硬體來解決相依性所造成的危障以確保正確的執行，編譯器仍然必須瞭解管線結構以求得到最佳的效能。否則意外的停滯會降低編譯出來的程式碼的效能

4.8 控制危障

- ▣ 圖4.61表示了一序列的指令並指出分支何時在管道中發生
 - 稱為控制危障(control hazard)或分支危障(branch hazard)
 - 介紹兩種解決控制危障的方法以及一種用來改善這兩者的最佳化方法

Branch Hazards

圖4.61 管道對分支指令的影響

- If branch outcome determined in MEM

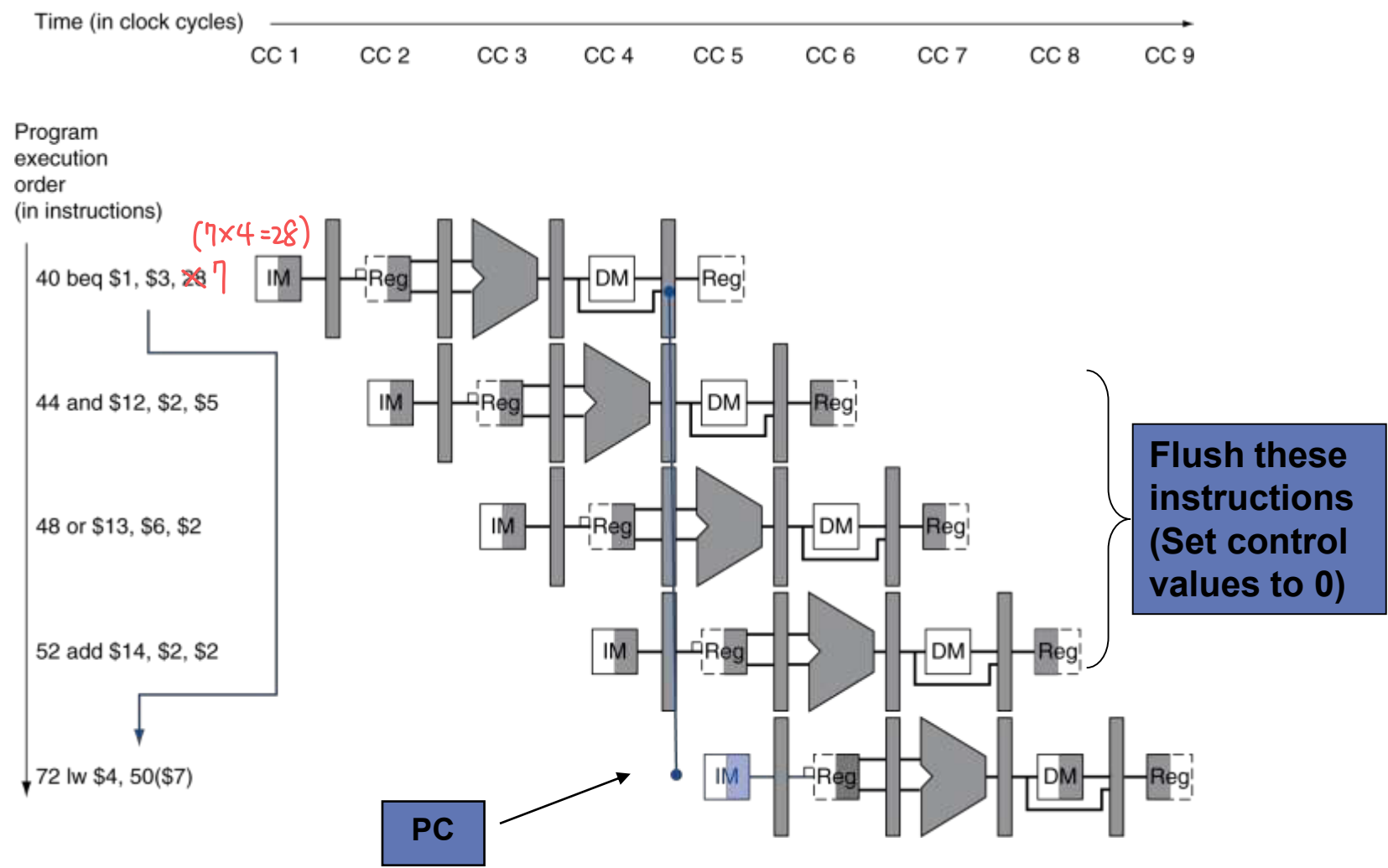


圖4.61 管道對分支指令的影響

指令左邊的數字(40, 44,)是指令的位址。因為分支指令在**MEM** 級(上述**beq** 指令於時脈週期4 中)決定是否要分支，該分支指令後連續三道指令會被擷取並開始執行。在沒有干預的情況下，這三道指令會在**beq**分支到位置72 的**lw** 指令之前開始執行。(圖4.31 假設使用了特別的硬體來將控制危害降低到一個時脈週期：此圖採用未最佳化的數據通道。)

假設分支不發生

- ▣ 繼續執行接下去的指令

- 若是分支發生了，之後已被擷取與解碼的指令就都要丟棄

減少分支的延遲

把分支的決定提早到IF/ID

- 要提早作分支決定則需提早做兩件事：計算分支的目標位址以及做分支判斷
 - 在IF/ID 管道暫存器中已經有PC 值與立即值欄位，所以只需將分支位址加法器從EX 級移到ID 級
 - 如果是相等則分支(beq)，我們可以比較在ID 級所讀取的兩個暫存器是否相等
 - 要用到額外的前饋與危障偵測硬體
 - 有兩個複雜的因素：

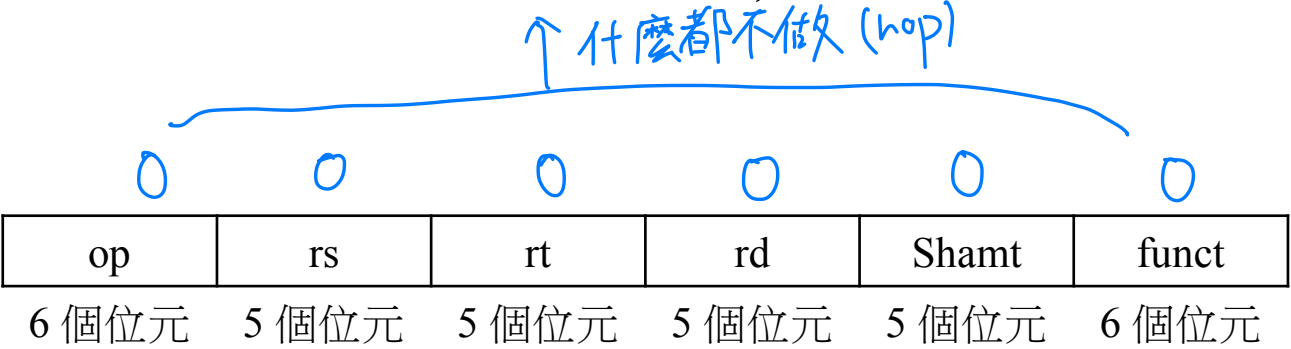
減少分支的延遲

① 將相等條件測試單元提前到ID 級中則需要新的前饋邏輯。要繞送的分支運算元可能從EX/MEM 或是MEM/WB 管道暫存器中傳來

② e.g.
add \$s1 \$s2 \$s3
beq \$s1 \$s2 L
即使 beq 提前判斷
还是要加 bubble

② 例如一個緊連在分支之前的ALU 指令會產生分支比較所需的其中一個運算元，這時候就需要一個週期的暫停。如果載入指令後方緊跟一個需要用到其值的分支，需要停滯兩個週期

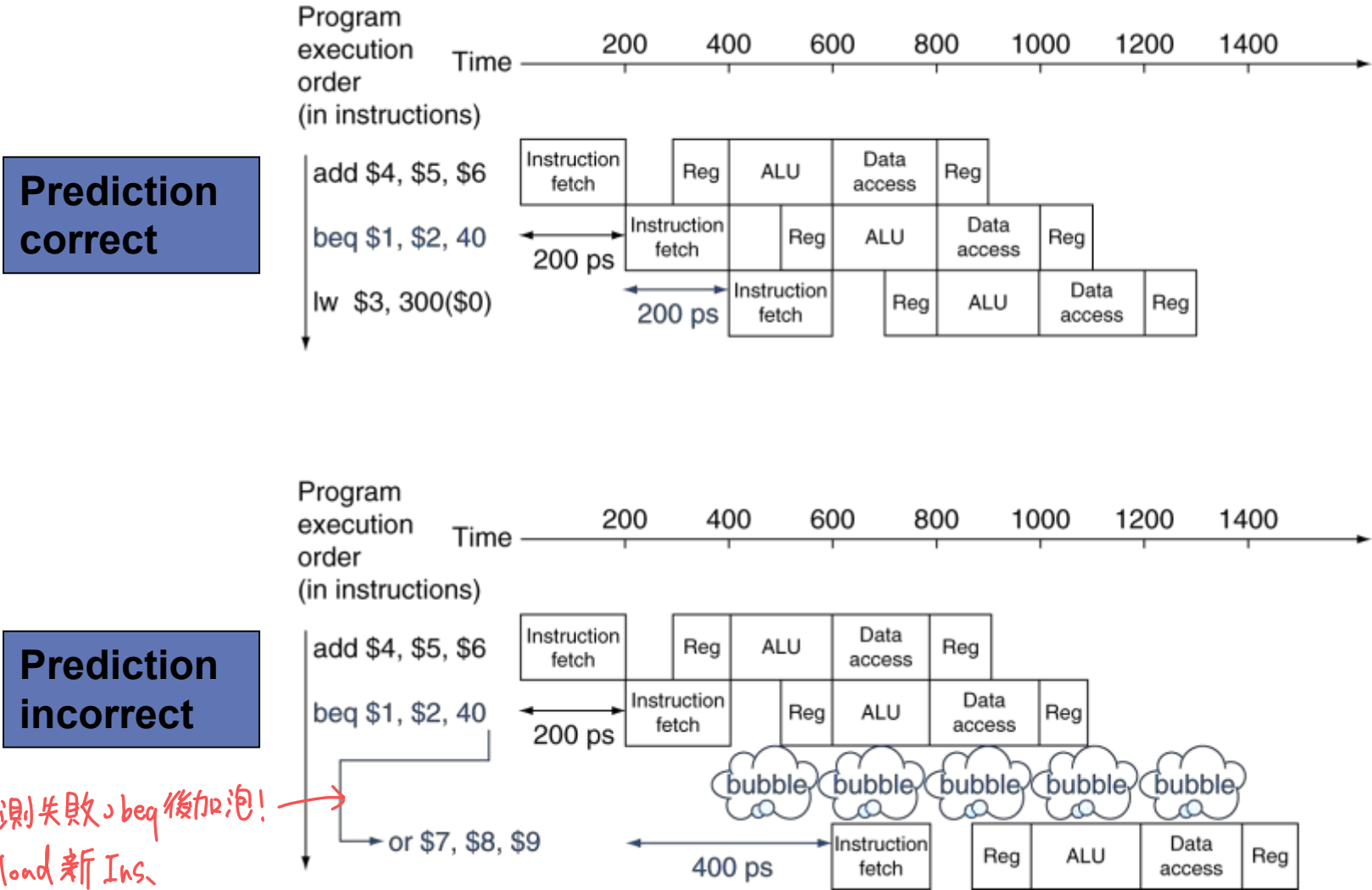
■ 為了清除IF 級中的指令，可將IF/ID 管道暫存器中的指令欄位設為0(NOP in MIPS is sll \$zero \$zero 0)



R-type指令

MIPS with Predict Not Taken

圖4.31



Reducing Branch Delay

從 MEM 移到 ID 減少延遲

▣ Move hardware to determine outcome to ID stage

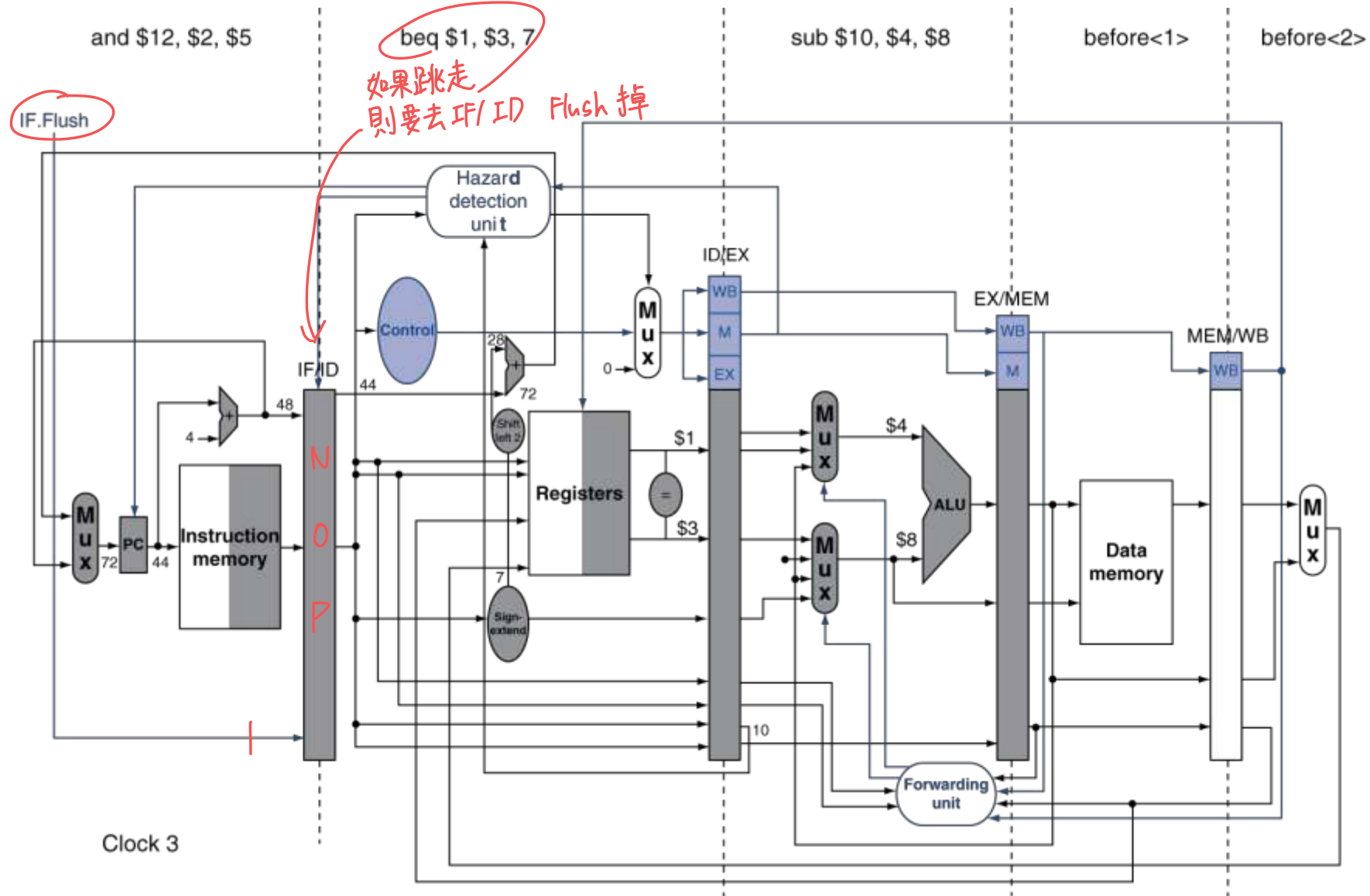
- Target address adder
- Register comparator

▣ Example: branch taken

```
36:  sub    $10, $4, $8
40:  beq    $1,  $3, 7
44:  and    $12, $2, $5
48:  or     $13, $2, $6
52:  add    $14, $4, $2
56:  slt    $15, $6, $7

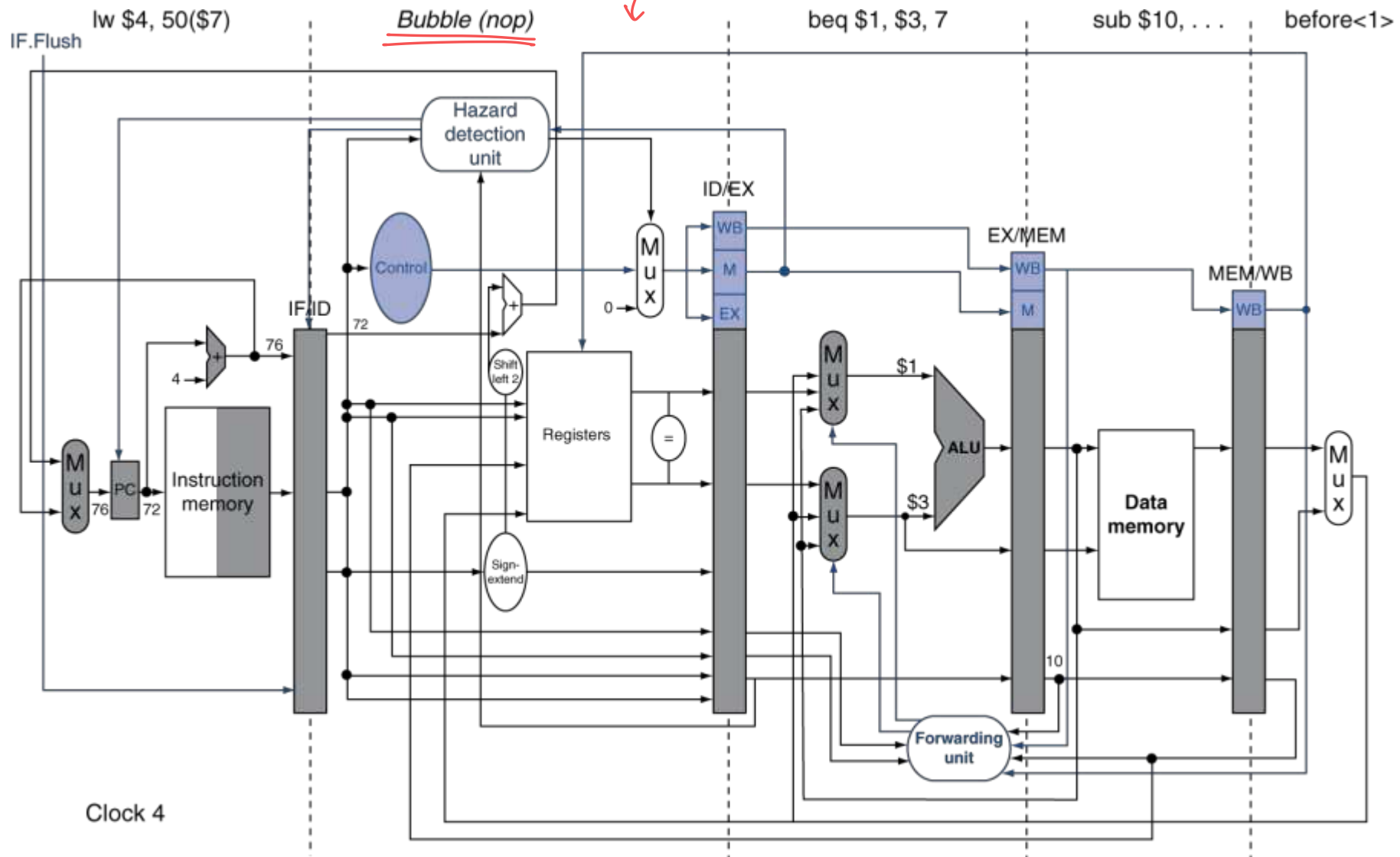
    ...
72:  lw     $4, 50($7)
```

Example: Branch Taken



Example: Branch Taken

分支指令提早判斷，也會有前饋的問題

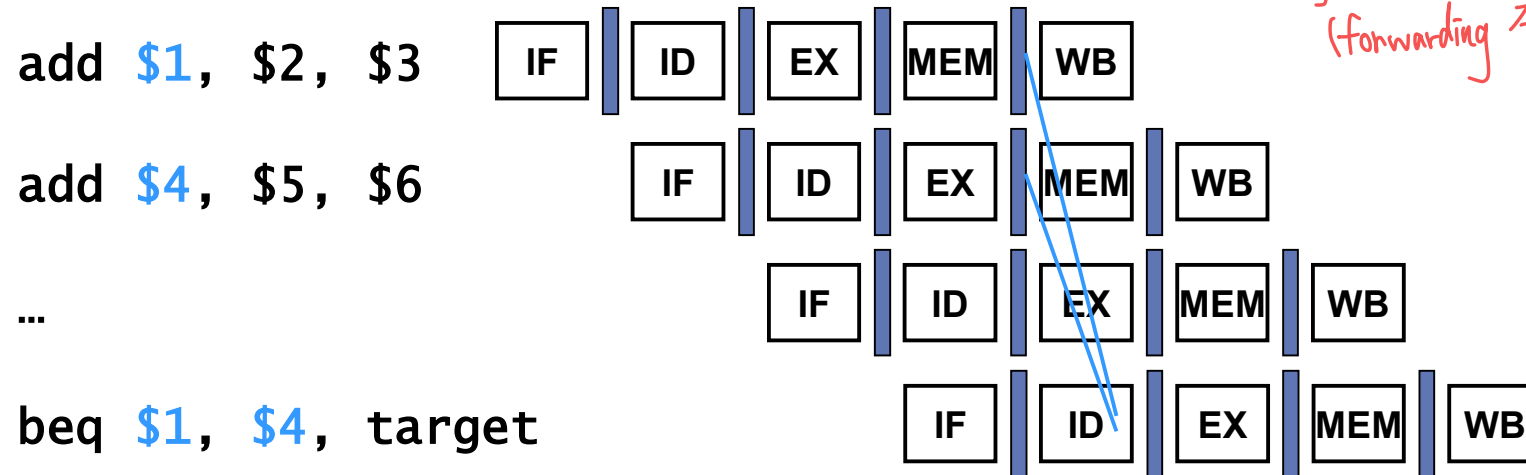


Data Hazards for Branches

- If a comparison register is a destination of 2nd or 3rd preceding ALU instruction

如果比較
前2 or 前三是ALU Ins.
直接靠 forwarding 做!

(forwarding 本來就在)

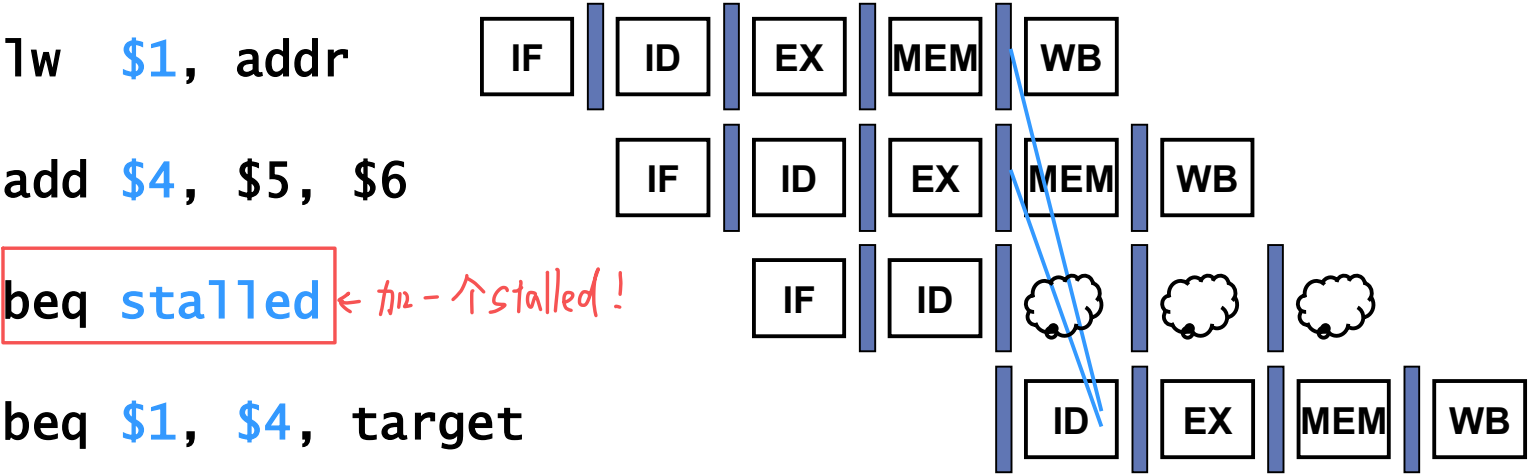


- Can resolve using forwarding

Data Hazards for Branches

- If a **comparison register** is a destination of **preceding ALU instruction** or **2nd preceding load instruction**
 - Need 1 stall cycle

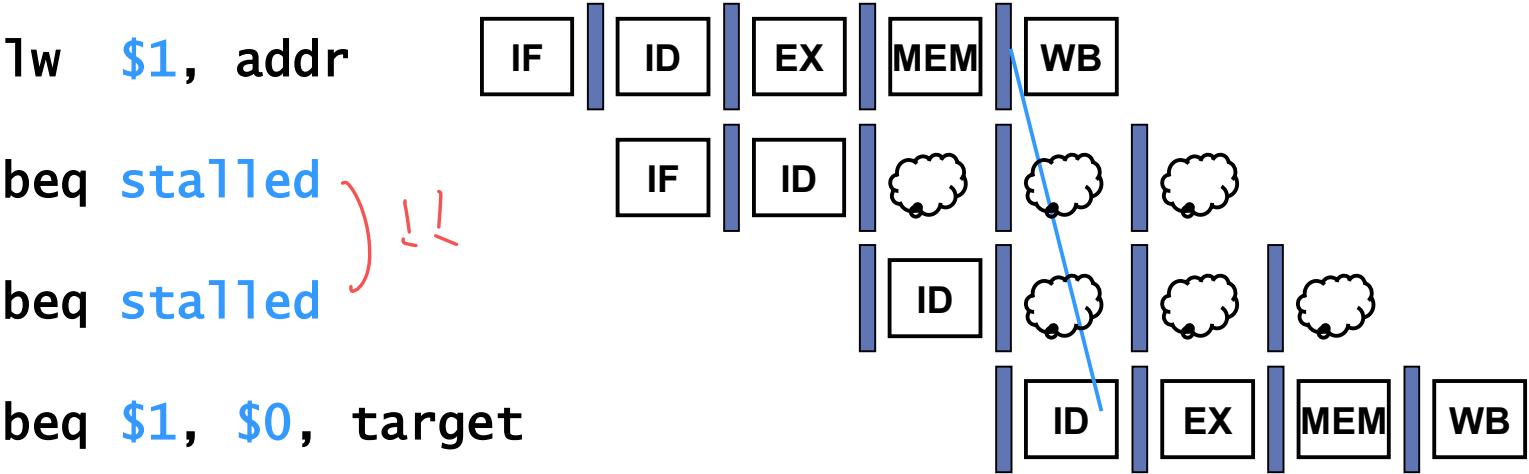
↓
如果比較
前一個是 ALU 指令，或前兩個是 lw Ins.



Data Hazards for Branches

- If a **comparison register is a destination of immediately preceding** load instruction
 - Need 2 stall cycles

如果比較前馬上跟一個lw，那就要加2個stalled！



動態條件跳躍預測

- 在更深的管道中以及在多重派發（multiple issue）的架構下，分支懲罰所造成的指令損失數也會增加
 - multiple issue：多道指令在一個時脈週期中發出的方法
- 動態分支預測(dynamic branch prediction)一種實現的形式是分支預測緩衝器(branch prediction buffer)或是分支歷程表(branch history table) 紀錄跳的比例多, 还是不跳的比例
 - 一位元預測方法(1-bit prediction scheme)有效能上的缺點
 - 二位元預測方法(2-bit prediction schemes)常用以彌補這個弱點。在二位元的預測方法中，預測的結果必須連續錯誤譬如兩次才會改變

動態條件跳躍預測

- 圖4.63 表示二位元預測方法的有限狀態機 (finite-state machine)

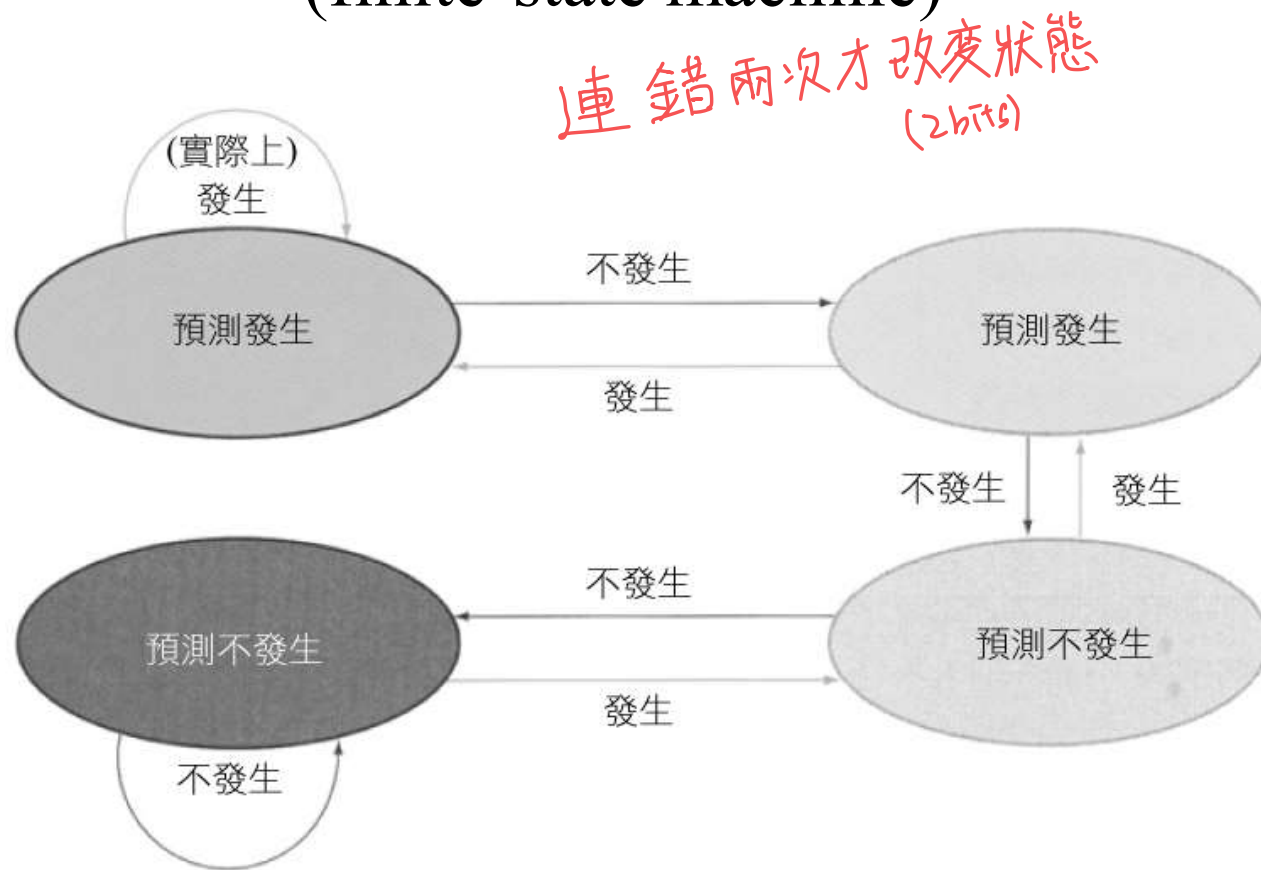


圖4.63 二位元預測方法的狀態

使用二個位元而非一位元，強烈偏向發生或是不發生的分支很多分支都有這種傾向只會猜錯一次。用兩個位元來編碼四個狀態。這種二位元預測方法是計數器型預測器的一個常見例子，其在預測準確時加一否則減一，並以中間點作為預測分支發生或不發生的分界

- 可以重新定義分支為一種特性而非控制危障：延遲的分支
 - 分支延遲槽(branch delay slot)
- 圖4.64 說明三種安插分支延遲槽的方法

圖4.64 安排分支延遲槽

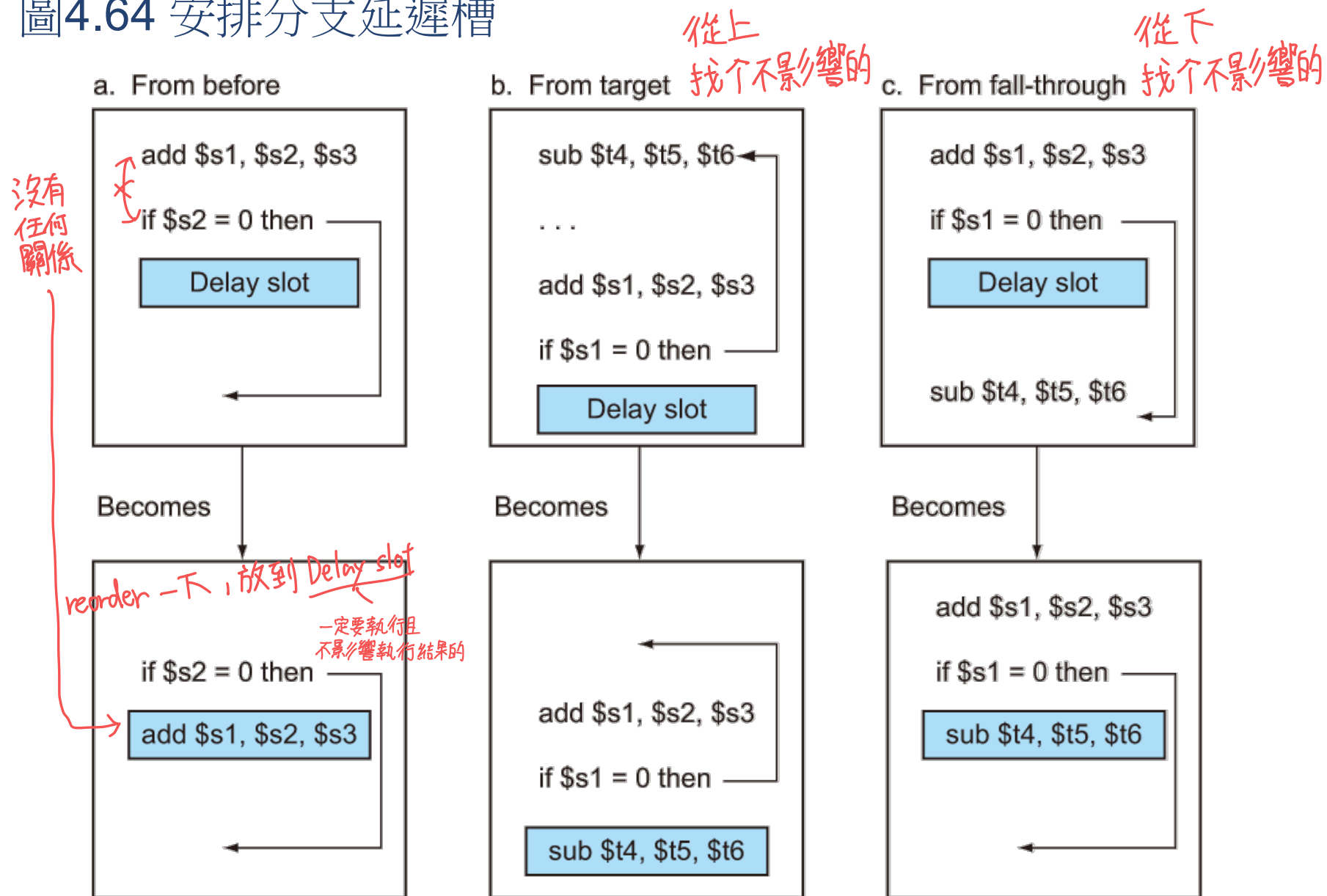


圖4.64 安排分支延遲槽

每一對方塊的上面那塊表示安排前的程式碼：下面那塊表示經過安排的程式碼。在**(a)**中，延遲槽內放進一道與分支無關的指令，這是最佳的選擇。策略**(b)**與**(c)**則是在**(a)**行不通時使用。在**(b)**與**(c)**的程式碼序列中，分支條件中使用了**\$s1**，造成**add**指令(其目的地暫存器為**\$s1**)無法被移到分支延遲槽中。在**(b)**中，分支延槽內放進分支的目標指令；通常該目標指令需要被複製過來，因為別的執行路徑可能還是會經過它。像是迴圈分支這類發生機率很高的分支適合採用策略**(b)**。最後，分支延遲槽也可以放進分支不發生的下一道指令如**(c)**所示。要使這個最佳化對**(b)**或**(c)**有效，在分支結果非如預期時，執行**sub**指令必須仍是**OK**的。**OK**的意思是做了白工，但是程式仍然會正確地執行。例如，假如分支結果非如預期時**\$t4** 會是一個用不到的暫時暫存器

□ 分支目標仍需計算。在五級管道中，這個計算需時一個週期，表示分支會有一個週期的懲罰

□ 延遲的分支是一種消除該懲罰的方法

□ 另有一法是以一個稱為分支目標緩衝器(branch target buffer)的快取來存放目標PC 或是目標指令

↓
把所有可能跳走的地方
都丟緩衝器

/*

□ 如果同時使用該分支以及最近執行過的數道分支這種更廣泛的資訊，在使用相同的預測位元數下，可以有更好的預測準確度

□ 關聯式預測器(correlating predictor)

- ▣ 競賽型預測器(tournament predictors)
 - ▣ 使用多個預測器，並對每個分支記錄哪個預測器產生最好的效果
 - ▣ 減少條件分支數量的一種方法是使用條件搬移 (conditional move)指令
 - ▣ ARMv7 指令集中大部分指令含有一個條件欄位
- */

管道總結

(还是不完整)

■ 圖4.65 表示最後得到的數據通道與控制

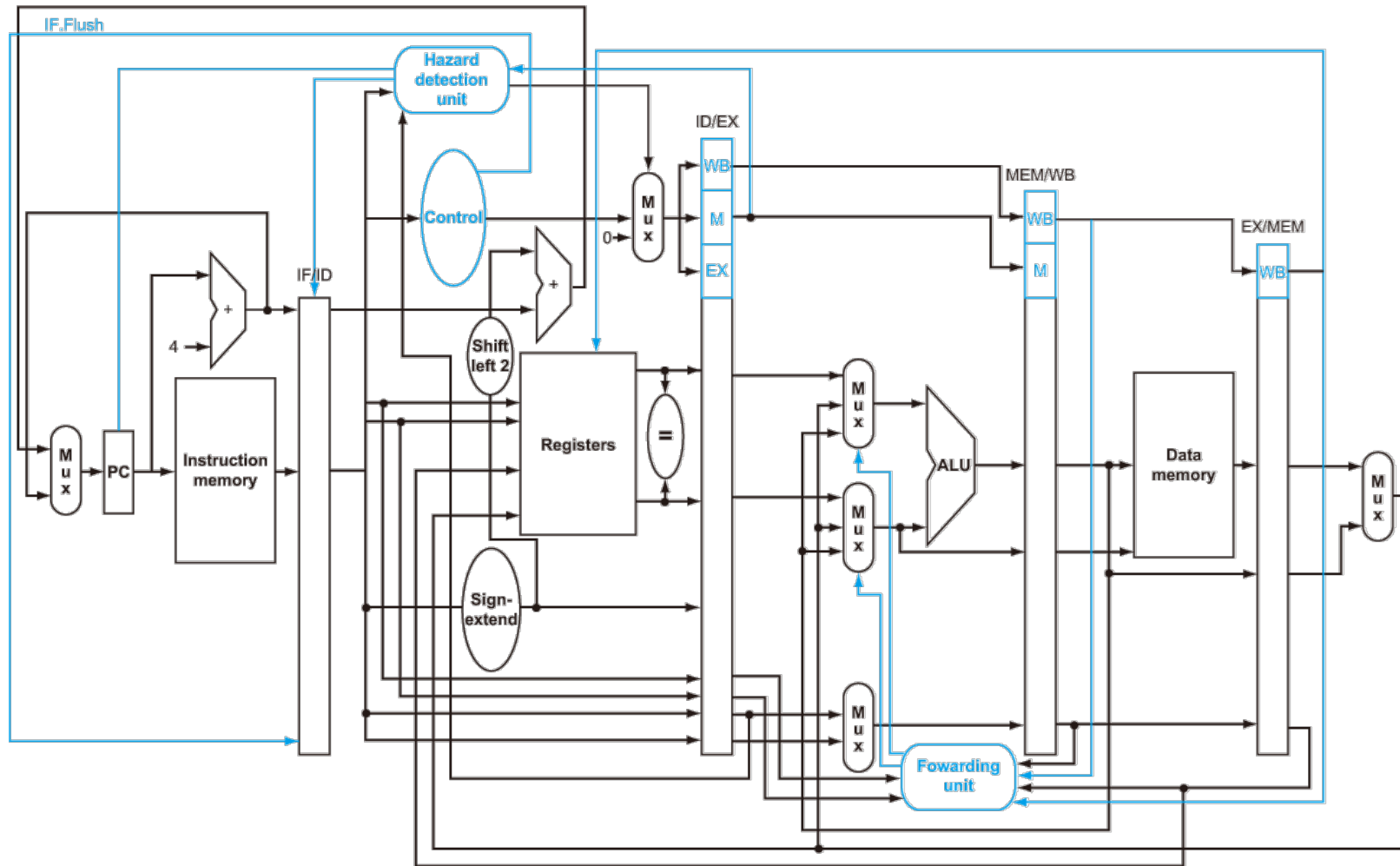


圖4.65 本章的最終數據通道與控制

注意這是一個示意圖而非詳細的數據通道，所以沒有畫出圖4.57中的ALUsrc多工器以及圖4.51中的多工器控制

4.9 例外

- ▣ 控制是處理器設計中最有挑戰性的部分：其既是最難做對的，也是最難做得快的部分
- ▣ 控制中最難的部分之一是製作例外(exceptions)和插斷(interrupts)
外部信号对CPU產生例外
中斷
- ▣ MIPS 慣例：
任何意外的控制流改變均以例外稱之；並只對外部原因造成的改變稱之為插斷

MIPS 架構中如何處理例外

- 我們目前的製作中會發生的兩種例外分別是欲執行未定義的指令以及算術滿溢 (overflow)
 - 以 `add $1, $2, $1` 指令中的算術溢位作為以下數頁中例外處理的例子
- 處理器在例外發生時必須要做的基本動作是保存引發問題的指令的地址於例外程式計數器(exception program counter, EPC)中以及之後將控制轉移給處於作業系統中的某特定位置
 - 在完成因應錯誤的相關行動後，作業系統會決定結束該程式或繼續執行該程式

MIPS 架構中如何處理例外

▣ 傳達例外原因的主要方式有二

- MIPS 架構所採用的方式是使用一個狀況暫存器(稱為原因暫存器(cause register))，其內包含一個指出例外肇因的欄位
- 第二種方式是採用向量的插斷(vectored interrupts) ← 主流

▣ 我們的製作中加入少許額外的暫存器以及控制訊號，並且稍微延伸控制單元，來提供例外的處理能力。假設：

- 以位址 8000 018016 作為唯一的進入點

↑ MIPS 中發生中斷時都跳來這

MIPS 架構中如何處理例外

- 需要在MIPS 的製作中加入兩個額外的暫存器：
 - **EPC** → 發生中斷時該行程式的位子
即使在向量的例外處理中這樣的暫存器也是需要的
 - **肇因(Cause)暫存器** ← 由 Control Unit 寫入
假設有一個5 位元的欄位用來表示上述兩種可能的例外原因，並以10代表未定義的指令以及12 代表算術滿溢
overflow

管道化製作中的例外

- ▣ 管道化的製作中視例外為另一種形式的控制危障。
 - 假設加法指令產生了算術滿溢
 - 必須清除管道中跟隨在add 指令後的指令並由新的位址開始擷取指令
 - ID.Flush與Ex.Flush與會將8000 0180₁₆ 傳給輸入到PC 的多工器，圖4.66 表示了這些改變
 - 將引發例外的指令位址存入例外程式計數器(EPC，exception program counter)中。(事實上我們保存的是位址+4，因此例外處理程序一定要先將所保存的值減4。)圖4.66 表示的是包含分支硬體以及處理例外所需線路的管道圖

圖4.66 加上例外處理控制的數據通道

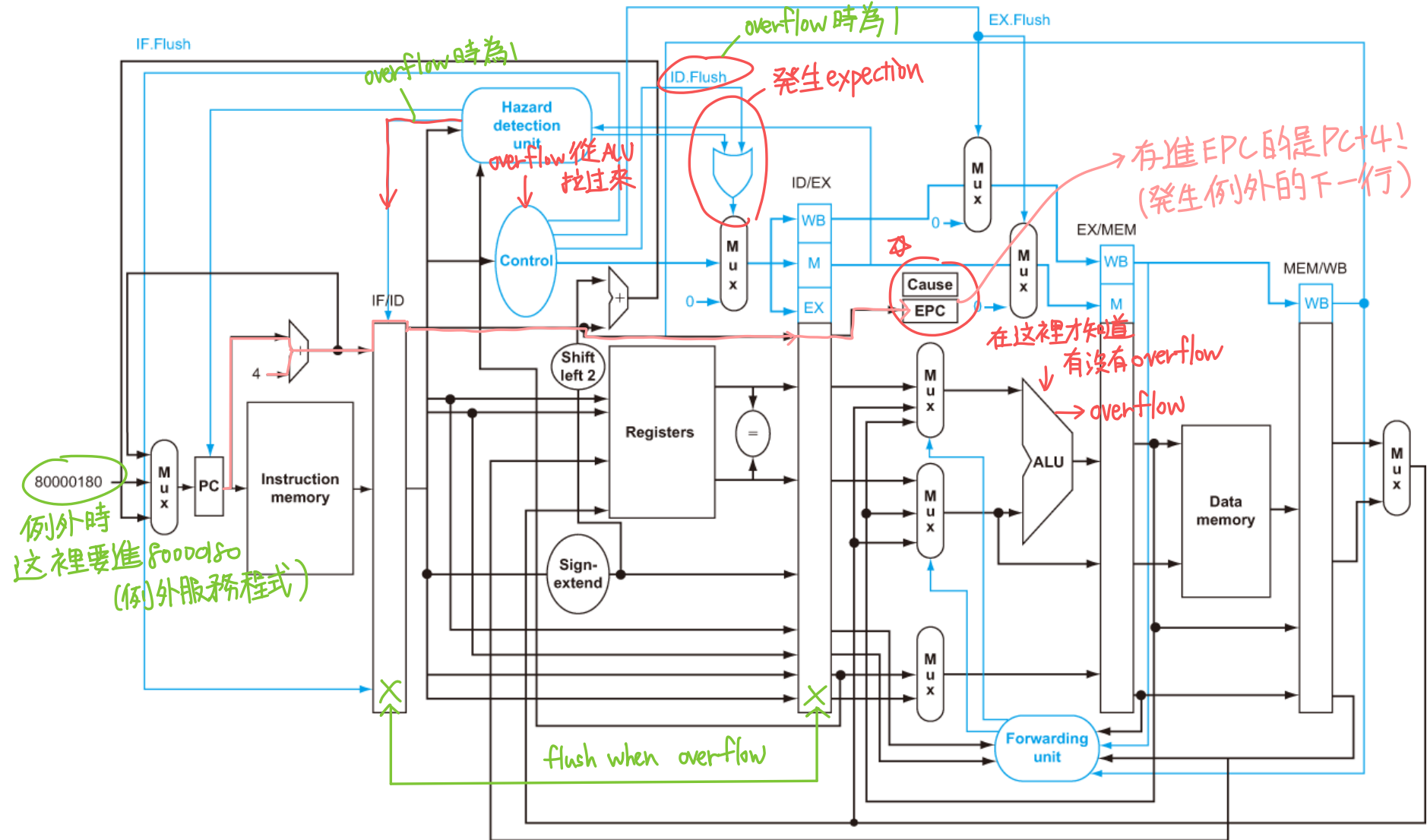


圖4.66 加上例外處理控制的數據通道

主要新增的包括：供應新**PC** 值的多工器的新輸入(輸入值為**8000 018016**)；用來記錄例外原因的**Cause** 暫存器以及用來儲存發生例外的指令位址的**EPC** 暫存器。多工器的**8000 018016** 輸入值是在發生例外時用來擷取指令的開始位址。**ALU** 的滿溢訊號雖然沒有畫出來，會是控制單元的一個輸入

管道化製作中的例外

- ▣ 面對任何時脈週期中都有五道指令正在運作，困難就在於如何將例外關聯到適當的指令上
- ▣ 甚至於在一個時脈週期中可能會發生多起例外
- ▣ 解決的方式就是將例外訂完優先次序

不同指令可能都會發生例外

硬體/軟體介面

- ▣ 硬體與作業系統必須配合，例外才能如預期般運作
 - 硬體的任務一般是使引發例外的指令在途中停止、讓之前的所有指令完成、清除之後的所有指令、設定一個暫存器來表示例外的原因、保存肇因指令的位址、之後跳到既定的位址
 - 作業系統的任務是查看例外的原因並適當地行動

→ 插斷時指令還沒做完

- 不精確的插斷(imprecise interrupts)或不精確的例外(imprecise exceptions)
- 精確的插斷(precise interrupts)或精確的例外(precise exceptions)

↓ 插斷時指令做完了

4.10 平行性與進階的指令階層平行性

- ▣ 提高可能的指令階層平行性(instruction-level parallelism, ILP)的主要方法有二
 - 第一種是增加管道深度以重疊更多道指令
 - 另一方法是複製計算機內部的元件以便其能夠在每一個管道階級中執行多道指令多重派發(multiple issue)
 - CPI 小於1。IPC，或每時脈週期指令數(instructions per clock cycle)，大於1
 - 目前的高階微處理器每個時脈週期中可以試圖發出三到六道指令

4.10 平行性與進階的指令階層平行性

- 基本上對哪些類型的指令可以在同一瞬間執行以及有相依性時應該如何處理仍有許多限制
- 靜態多重派發(static multiple issue)以及動態多重派發(dynamic multiple issue)
- 有兩個必須要面對的主要而且明顯不同的工作：
 1. 將指令包裝進派發槽(issue slots)中
 2. 處理資料以及控制危障

猜測的概念

- ▣ 尋找以及利用更多ILP的最重要方法之一就是猜測。例如，
 - 對一個分支預測其結果，以便其後的指令得以提早執行
 - 猜測一個在載入指令之前的儲存指令並不會存取到同一個位址，因此載入得以在儲存前執行
- ▣ 針對不正確猜測的回復機制差異相當大
 - 在軟體來做猜測的情況下
 - 在硬體做猜測時
- ▣ 猜測引發另一個可能的問題：預測性地執行一些指令可能會造成不應發生的例外

靜態多重派發

- ▣ 由編譯器來協助聚集指令以及處理危障
- ▣ 派發封包(issue packet)：一個含有多個運作的大指令
 - 導致了這種方法的原始名稱：很長指令字(Very Long Instruction Word, VLIW)

例子：MIPS 指令集架構的靜態多重派發

- ▣ 一個簡單的雙派發(twoissue)MIPS 處理器
- ▣ 兩道同時派發的指令中其一可以是整數ALU 運作或分支，另一則可以是載入或儲存
- ▣ 在許多靜態多重派發的，尤其是基本上所有的VLIW 處理器中，同時會派發的指令的擺放方法都會有些限制以簡化解碼以及指令派發
 - 因此，我們也要求指令必須配對並位於對齊64元邊界的位置上，而且位於前方的指令需為ALU 或分支
 - 如果指令對中有一道無法找到恰當指令，則以nop 取代之

例子：MIPS 指令集架構的靜態多重派發

■ 圖4.68

指令類別	管道階段							
ALU 或分支指令	IF	ID	EX	MEM	WB			
載入或儲存指令	IF	ID	EX	MEM	WB			
ALU 或分支指令		IF	ID	EX	MEM	WB		
載入或儲存指令		IF	ID	EX	MEM	WB		
ALU 或分支指令			IF	ID	EX	MEM	WB	
載入或儲存指令			IF	ID	EX	MEM	WB	
ALU 或分支指令				IF	ID	EX	MEM	WB
載入或儲存指令				IF	ID	EX	MEM	WB

圖4.68 運作中的靜態雙重派發管道

ALU 指令和資料傳輸指令可以同時派發。這裡我們假設與單一派發管道相同的五級管道結構。這雖非絕對必要，但的確有一些優點；特別是將寫入暫存器放在管道最後面簡化了例外處理以及精確例外模型的維護，而這工作在多重派發的處理器中將更為困難

例子：MIPS 指令集架構的靜態多重派發

- 一些設計中，編譯器完全負責將所有危障移除，排定指令序並適當插入 `nops`
 - 程式執行時完全不需作危障偵測或以硬體產生停滯
- 另有一些則由硬體偵測數據危障並於派發封包間產生停滯，而編譯器只負責避免指令對彼此間的任何相依關係
- 本例中我們將採用本方法

例子：MIPS 指令集架構的靜態多重派發

- ▣ 需要的額外硬體
 - 額外的暫存器檔案埠
 - 另一個加法器來計算數據搬移的有效位址
- ▣ 更多指令的重疊執行帶來的資料和控制危障增加了相對的效能損失
 - 需要有更有企圖心的編譯器或硬體的排程技術，而靜態多重派發要求編譯器負起全責

簡單的多重派發碼排序

以下迴圈在MIPS 的靜態雙重派發管道中應如何排序？

```
LOOP : lw $t0, 0($s1) # $t0=陣列元素  
      addu $t0,$t0, $s2 # 加上$s2中的純量  
      sw $t0, 0($s1) # 儲存結果  
      addi $s1,$s1,-4 # 遞減指標  
      bne $s1,$zero,Loop # $s1!=0 則分支
```

重排指令以儘量避免管道停滯。假設硬體會預測分支以及處理控制危障

簡單的多重派發碼排序

前三道指令間有資料相依性，因此提前執行最後兩道。圖4.70 顯示對該等指令的最佳排序。注意僅有一對指令使用掉兩個派發槽。每個迴圈反覆需時四個時脈週期；以四個時脈來執行五道指令，相對於最佳的CPI 為0.5 我們只得到令人失望的0.8，或是IPC 並非2.0 而是1.25。注意在計算CPI 或IPC 時我們不把執行過的nops 算成是有用的指令。將nops 計入可以使CPI 變好，但是無助於效能！

例子：MIPS 指令集架構的靜態多重派發

- 一個由迴圈中得到更高效能的重要編譯器技術稱為迴圈展開(loop unrolling)

	ALU 或分支指令	資料傳輸指令	時脈週期
Loop :		lw \$t0, 0(\$s1)	1
	addi \$s1,\$s1,-4		2
	addu \$t0,\$t0,\$s2		3
	bne \$s1,\$zero,Loop	sw \$t0 4(\$s1)	4

圖4.70 排程後的程式碼在雙重派發MIPS 管道中的情形
空格表示nops

為了多重派發管道的迴圈展開

看看迴圈展開與排序對上例多有用。為了簡化起見
假設迴圈索引值是4 的倍數

為了多重派發管道的迴圈展開

為了要將迴圈排序成不會有任何延遲(指停滯)，則我們需要將迴圈本體展開成四份。在展開並去除不需要的迴圈控制指令後，迴圈將包含四份的lw、add與sw，加上一道addi與一道bne。圖4.71 列出展開並排序後的碼

暫存器重命名(register renaming) 消除反相依性(antidependence)或名稱相依性(name dependence)

為了多重派發管道的迴圈展開

注意在迴圈中的14 道指令現在有12 道可以成對地執行。四個原本的迴圈反覆展開後需時8 個時脈，亦即每個反覆兩個時脈，得到CPI 為 $8/14=0.57$ 。雙派發下的迴圈展開與排序給了我們將近二倍的改善，其部分來自減少了迴圈控制的指令數以及部分來自雙派發執行。這種效能改善所付出的代價是使用了四個暫時暫存器而非僅一個，以及大大增加的程式大小

為了多重派發管道的迴圈展開

	ALU 或分支指令	資料傳輸指令	時脈週期
Loop :	addi \$s1,\$s1,-16	lw \$t0, 0(\$s1)	1
		lw \$t1,12(\$s1)	2
	addu \$t0,\$t0,\$s2	lw \$t2, 8(\$s1)	3
	addu \$t1,\$t1,\$s2	lw \$t3, 4(\$s1)	4
	addu \$t2,\$t2,\$s2	sw \$t0,16(\$s1)	5
	addu \$t3,\$t3,\$s2	sw \$t1,12(\$s1)	6
		sw \$t2, 8(\$s1)	7
	bne \$s1,\$zero,Loop	sw \$t3, 4(\$s1)	8

圖4.71 圖4.70 的程式碼經過展開以及排程以後在靜態雙重派發MIPS 管道中的情形

空格表示nops。由於迴圈中的第一道指令把\$**s1** 減16，因此載入的位址是\$**s1**的原始值，然後是原始值減4、減8、減12

動態多重派發處理器

- ▣ 動態多重派發處理器亦稱超純量(superscalar)處理器
- ▣ 最簡單的超純量處理器中，指令依序派發，處理器決定在某一時脈週期中可以派發的指令數
 - 仍需編譯器協助排序指令來拉開相依性以便提高指令派發率
- ▣ 許多超純量將動態派發判斷所用的基本架構擴充來兼作動態管道排程(dynamic pipeline scheduling)

動態管道排程

- 動態管道排程選取接下來要執行的指令，可能也會重排指令以避免停滯
- 在這類處理器中，管道可視為包含三種主要單元：
 - 一個指令擷取與派發單元
 - 多個功能單元
 - 一個認可單元(commit unit)
 - 圖4.72 表示這個模型

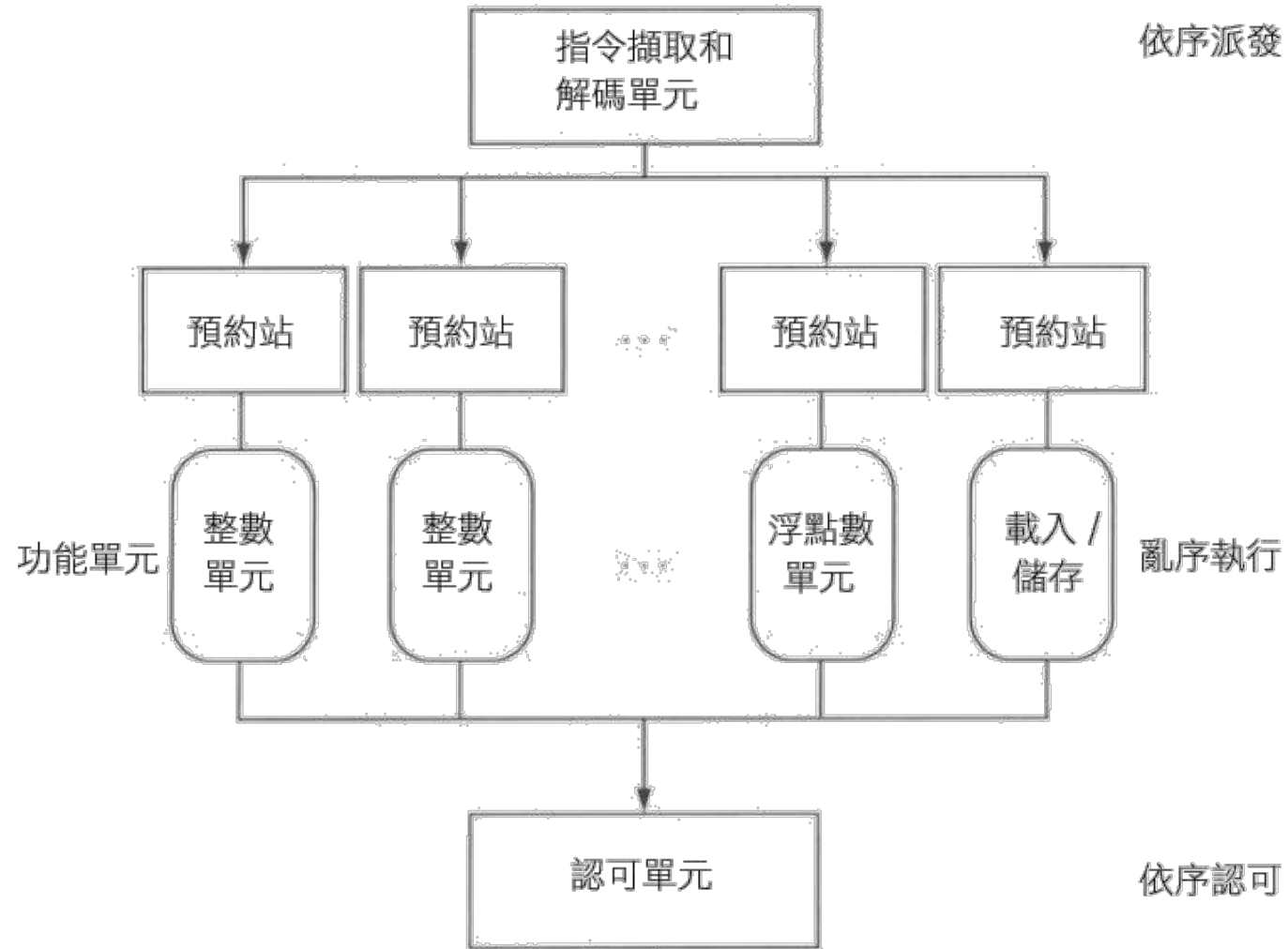


圖4.72 動態排程管道的三種主要單元

最後的更新狀態步驟又叫做退休(**retirement**)或是畢業(**graduation**)

動態管道排程

- 第一個單元擷取指令、解碼它們並將每道指令送到相關的功能單元中以便執行
- 每個功能單元中有一些稱為預約站(reservation stations)的緩衝器以保存運算元以及所需運作
- 當結果算出來後，其即被送往任何在等待該結果的預約站以及認可單元
- 認可單元暫存這筆結果，並等到可以將該結果寫入暫存器檔案，或是對儲存而言寫入記憶體

動態管道排程

- 認可單元中的緩衝器通常稱為重序緩衝器(reorder buffer)，亦可用於提供運算元，方式很像在靜態排程管道中前饋邏輯的做法
 - 使用重序緩衝器以及預約站來實現了暫存器重命名
- ▣ 這樣的執行方式稱為亂序執行(out-of-order execution)
- 因為指令可以以一個不同於擷取的次序來執行
 - 依序認可(in-order commit)
 - 若有例外發生，計算機可以指出最後一道被執行到的指令

動態管道排程

- ▣ 動態排程往往會被擴充成具有硬體的猜測能力，特別是猜測分支的結果

瞭解程式效能

■ 使用動態排程有三個主要原因

- ① 並非所有停滯都可預見。尤其是快取錯失
- ② 如果處理器以動態分支預測來猜測分支的結果，在編譯時無法得知確切的指令執行次序
- ③ 隨著管道延遲與派發寬度因不同製作而異，編譯一段碼的最佳方式也會不同。動態排程以硬體來處理掉大部分的這類細節

硬體/軟體介面

- 儘管有許多處理器可一次派發四至六道指令，極少的應用能維持每週期平均兩道指令。這是因為兩個主要的原因：
 - ① 在管道中主要的效能瓶頸來自於無法減輕的相依關係
 - ② 記憶體階層的失誤(第五章的議題)也會限制保持管道充滿的能力

功耗效率和進階的管道化處理

- 以動態多重派發和猜測來加強指令階層平行性發掘的負面效應是功耗效率
 - 轉而思索每晶片有多個處理器
 - 簡化的處理器可在每瓦(watt)中提供更好的效能
- 圖4.73 比較了許多過去以及最近的微處理器在管道級數、派發數、猜測能力、時脈速率、每晶片核數以及功耗的數據

微處理器	年度	時脈速率	管道 階數	派發 寬度	亂序 / 猜測	內核數 / 晶片	功耗	
Intel 486	1989	25 MHz	5	1	否	1	5	W
Intel Pentium	1993	66 MHz	5	2	否	1	10	W
Intel Pentium Pro	1997	200 MHz	10	3	是	1	29	W
Intel Pentium 4 Willamette	2001	2000 MHz	22	3	是	1	75	W
Intel Pentium 4 Prescott	2004	3600 MHz	31	3	是	1	103	W
Intel Core	2006	2930 MHz	14	4	是	2	75	W
Intel Core i5 Nehalem	2010	3300 MHz	14	4	是	1	87	W
Intel Core i5 Ivy Bridge	2012	3400 MHz	14	4	是	8	77	W

圖4.73 以管道複雜度、內核數、功耗表示的Intel 與Sun 微處理器歷程
Pentium 4 的管道階數未計入各認可階。如果計入它們則Pentium 4 的管道會更深

4.11 實例：ARM Cortex-A8 與Intel Core i7 的管道

處理器	ARM A8	Intel Core i7 920
市場	個人移動裝置	伺服器、雲端
溫控設計功耗	2 Watts	130 Watts
時脈速率	1 GHz	2.66 GHz
核數/晶片	1	4
浮點？	無	有
多重派發？	動態	動態
峰值指令點/時脈週期	2	4
管道階數	14	14
管道排程	靜態依序	動態亂序且有投機
分支預測	2 層	2 層
第一層快取/核	32 KiB I, 32 KiB D	32 KiB I, 32 KiB D
第二層快取/核	128 - 1024 KiB	256 KiB
第三層快取(共用)	-	2 - 8 MiB

圖4.74
ARM
Cortex-A8
與Intel
Core i7 920
的規格

ARM Cortex-A8

▣ 圖4.75 顯示整個管道的構

- 最前的三階一次擷取兩道指令來試圖保持一個12 道指令的預取緩衝儲存器是滿的
- 使用一個包含有512 個項目的分支目標緩衝器、4096 個項目的全域歷程緩衝器，以及8 個項目的返回堆疊
- 13 個時脈週期的預測錯誤懲罰
- 解碼的五階
- 指令執行區段的六階中提供載入儲存指令一條管道，以及算術運算兩條管道、其中只有第一條能執行乘法。三條管道間具有完整的資料前饋能力

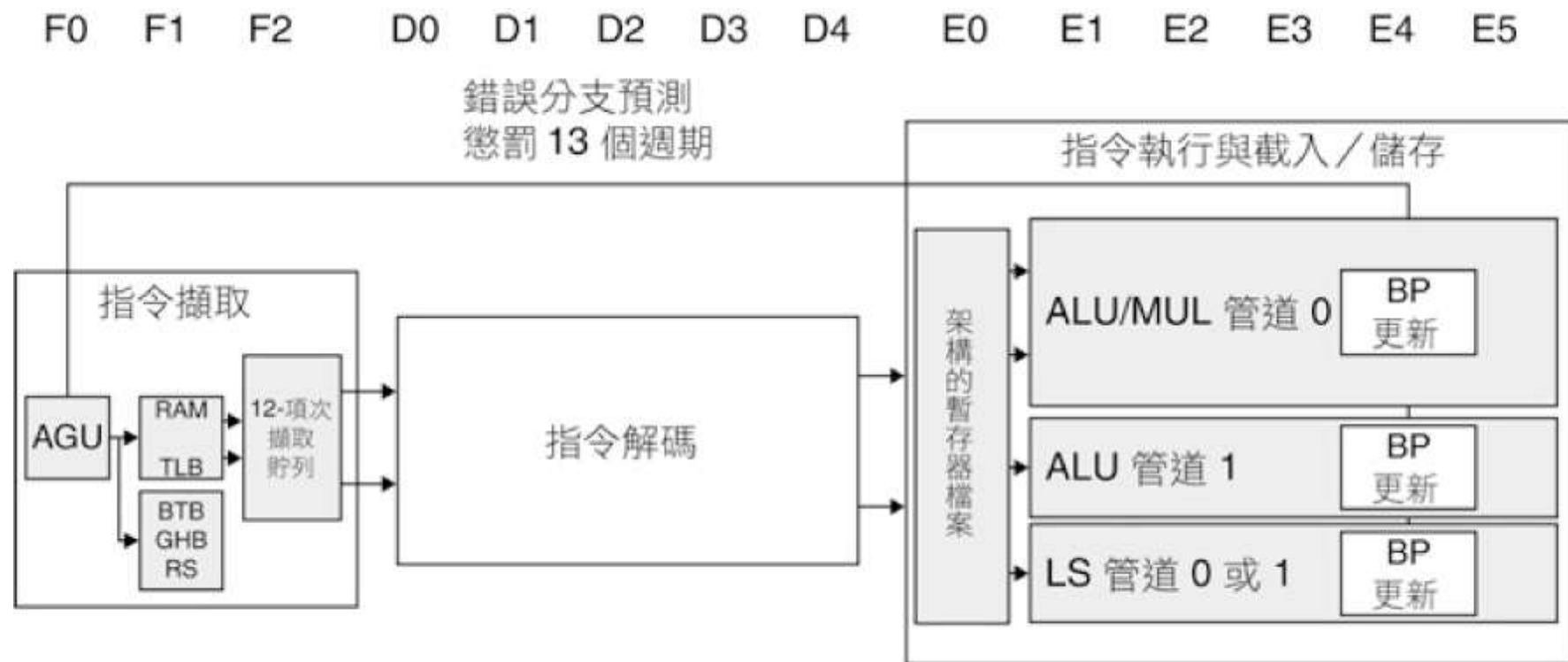


圖4.75 A8 的管道

最前的三階將指令擷取入12 個項次的指令擷取緩衝器中。一個位址產生單元(Address Generation Unit, AGU)使用分支目標緩衝器(Branch Target Buffer, BTB)、全域歷史緩衝器(Global History Buffer, GHB)以及返回堆疊(Return (Address) Stack, RS)來預測分支以求保持擷取貯列中放滿指令。指令解碼使用五階以及指令執行使用六階

ARM Cortex-A8

- ▣ 圖4.76 列出A8 在執行由SPEC2000 測試程式改寫的縮小版本時的CPI
- ▣ A8 的管道是靜態的，避免結構危障與資料相依(危障)的責任在於編譯器。為什麼如此？

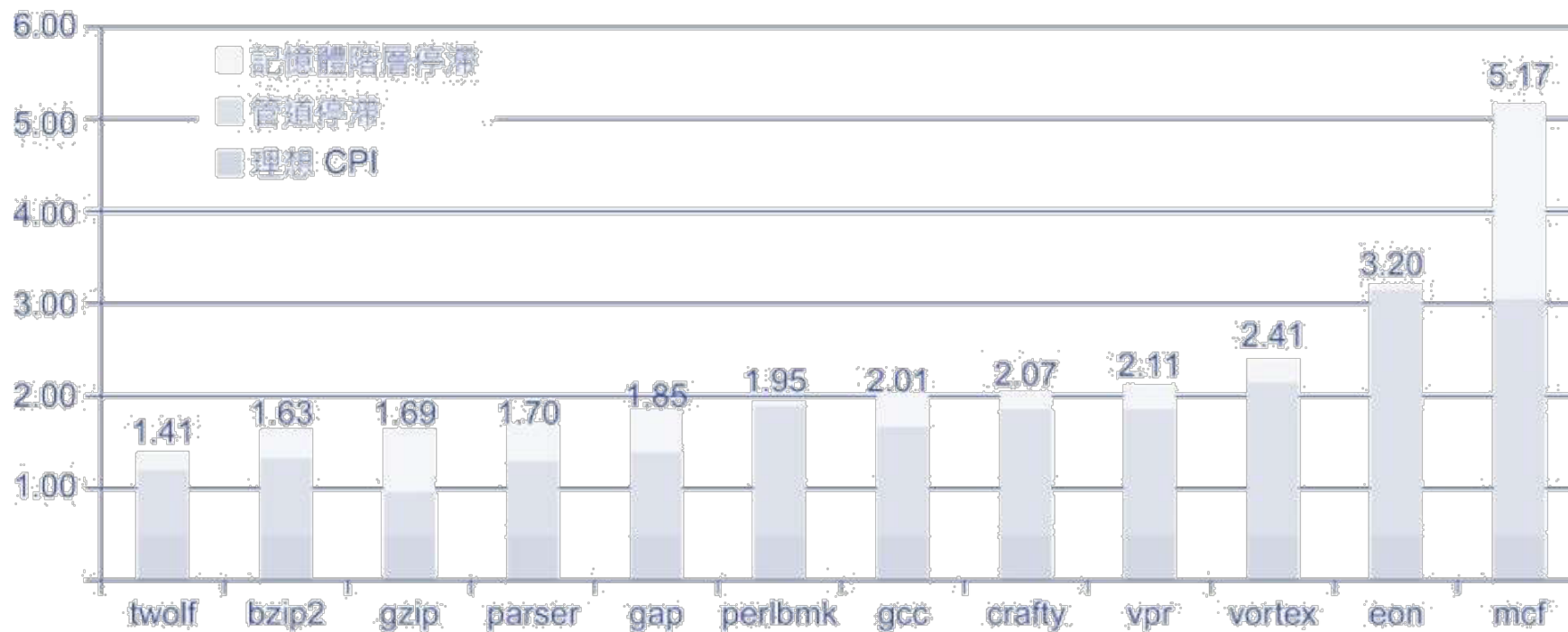


圖4.76 ARM Cortex A8 對Minnespec 測試程式——一套小型的 SPEC2000 測試程式集版本——的CPI

這些測試程式使用小很多的輸入數據來減少執行時間至十的好幾次方之一。較小的規模大大地低估了記憶體階層對CPI 造成的衝擊(見第五章)

Intel Core i7 920

- ▣ 14 階的管道中使用動態多重派發以及動態管道排程加上亂序執行和猜測(分支結果)
 - Intel 擷取x86 指令並將之轉換成其稱之為微運作(micro-operations)的(一至多道)像MIPS 指令的內部指令
 - 然後這些微指令會在每時脈週期可執行六道微指令的複雜的、可動態排程、具猜測能力的管道上執行

Intel Core i7 920

- ▣ 採用微架構(microarchitecture)這名詞來表示處理器詳細的內部架構
 - 具有暫存器重命名(register renaming)功能的重序緩衝器(reorder buffer)
 - 架構上的暫存器(architectural registers)
- ▣ 圖4.77 表示Core i7 管道的整體組織

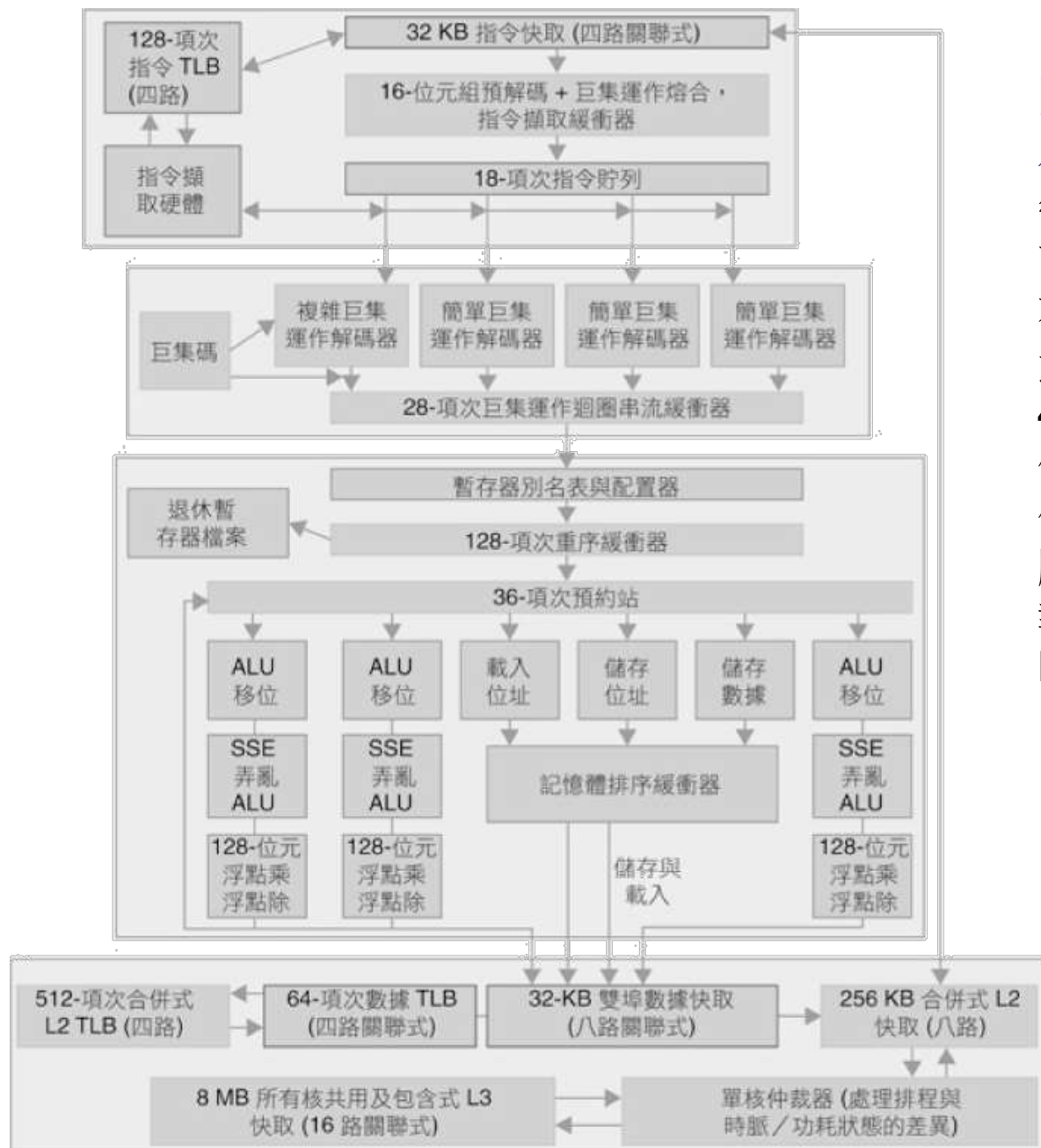


圖4.77 Core i7 管道以及各個記憶體組件。

管道的總深度有**14** 階，而錯誤的分支預測需花費**17** 個時脈週期。這個設計可以緩衝住**48** 道載入指令與**32** 道儲存指令。裡頭的六個獨立的單元每個時脈週期中都可以開始執行一道準備好了的 RISC 運作。

Intel Core i7 920

▣ x86 指令執行時經過的八個步驟：

- ① 指令擷取—使用多層的分支目標緩衝器；用以加速函數返回的返回位址堆疊；錯誤預測會導致15 個機器週期的懲罰；擷取16 個位元組
- ② 該16 個位元組置入指令預解碼緩衝器—之後個別指令將被放置入18 個項目的指令貯列中

Intel Core i7 920

- ③ 微運作解碼——一個別的x86 指令將被轉換成若干微指令運作。三個解碼器處理可直接轉換成一道微運作的x86 指令。對於那些具複雜語意的x86 指令，則有一個用以產出對應的微運作序列的微碼(microcode)引擎；其在一個週期內可產生出四道微運作並持續動作至所需序列產生完成為止
- ④ 微運作緩衝器作迴圈流偵測(loop stream detection)——長度小於28 道指令或256 個位元組則會辨識該迴圈

Intel Core i7 920

- ⑤ 執行基本指令派發—查找到相關的暫存器、對暫存器重命名、配置一個重序緩衝器項目給該微運作、並自暫存器或重序緩衝器中取出任何需要的運算元的值
- ⑥ i7 使用了一個由六個功能單元共用的36 個項目集中式預約站。其每個時脈週期至多可派發六道微運作至各功能單元中
- ⑦ 各別功能單元執行微運作，之後將結果送回
- ⑧ 當位於重序緩衝器前方一或多道指令已標記成已完成時，暫存器退休單元中相關的等候中寫即予執行，並將該等指令自重序緩衝器移除

- ▣ 第二步驟中的巨運作熔合(macroop fusion)可將某些x86 指令的組合譬如比較之後接著做分支者熔合成一道運作
- ▣ 第四步驟中的微熔合(microfusion)可合併一對例如load/ALU 運作或ALU 運作/store 來派發它們到一個預約站中(從那裡它們還是可以個別派發出來)，以提昇緩衝器的使用效率

Intel Core i7 920 的效能

■ 圖4.78 列出Intel Core i7 對每一個SPEC2006 測試程式的CPI

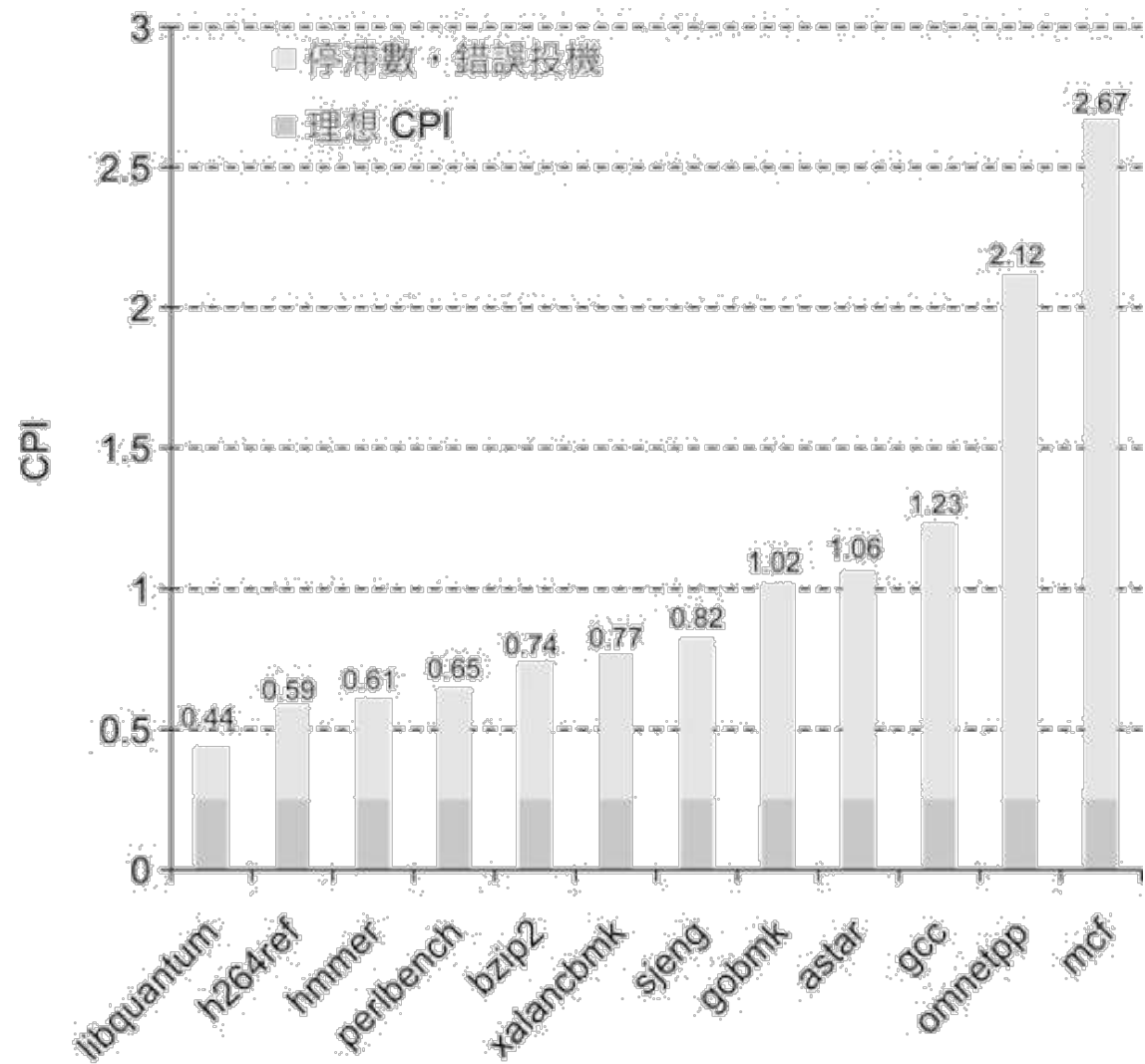
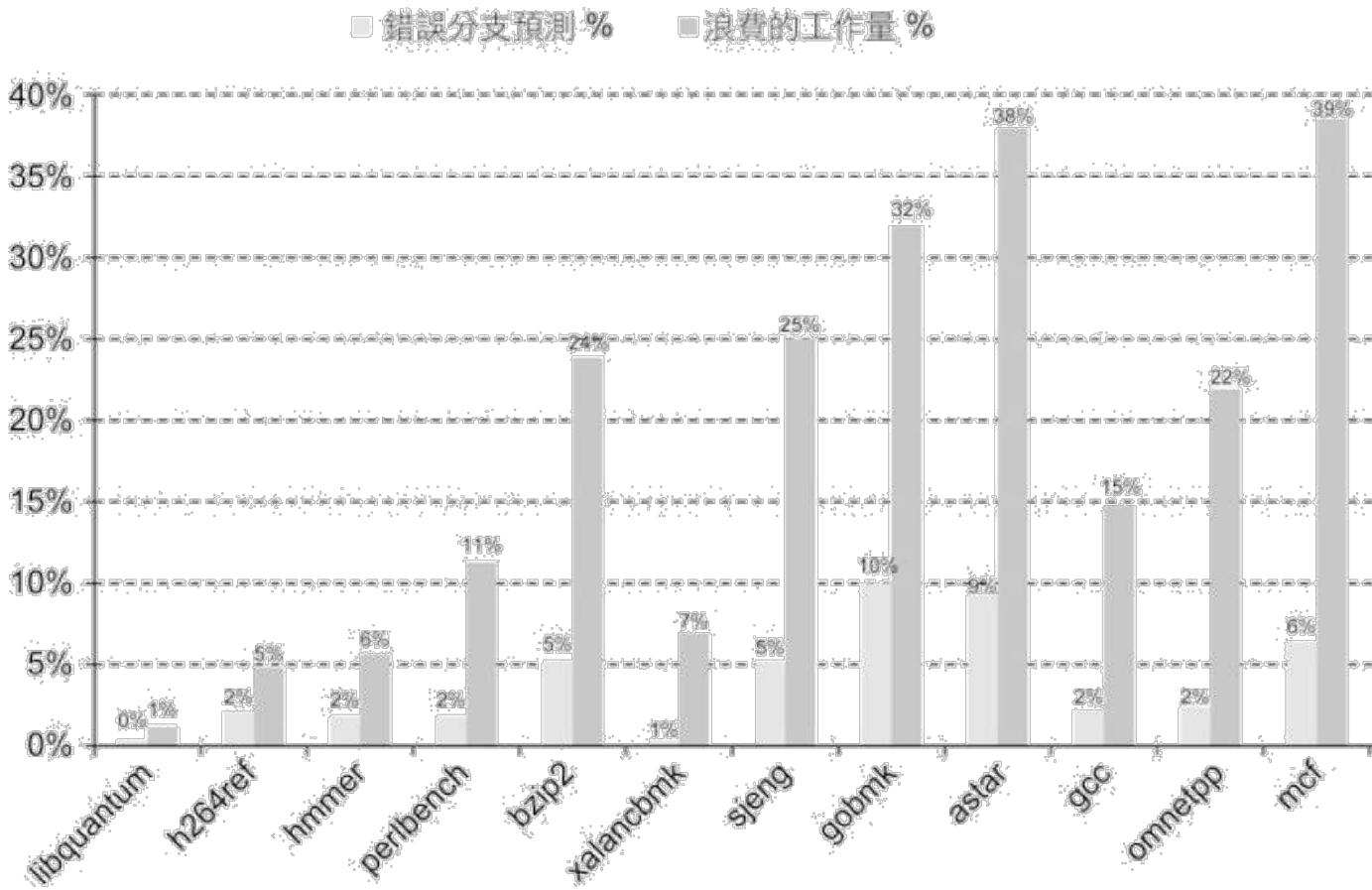


圖4.78 Intel Core i7 920 執行SPEC2006 整數測試程式的CPI

Intel Core i7 920 的效能

■ 圖4.79 列出相對於所有派發出的微運作，分支錯誤預測的百分比和不能退休的部分的百分比。

圖4.79 Intel
Core i7 920
執行
SPEC2006
整數測試程式
中，分支預測
錯誤與沒有用
處的投機執行
所浪費掉的工
作量的百分比



4.12 走得更快：指令階層平行性與矩陣乘法

- ▣ 回到第三章的DGEMM 例子
 - 展開迴圈來提供可多重派發及亂序執行的處理器更多指令來執行
- ▣ 圖4.80 顯示圖3.23 展開後的版本

圖4.80

DGEMM 經由使用C程式透過C的編譯器內建函式以便產生x86 的具部分字平行性的AVX 指令(圖3.23)、以及迴圈展開來增加更多指令階層平行性機會的最佳化後的C程式版本

圖4.81 顯示編譯器對內迴圈所產出的組合語言(程式)，其將三個for 迴圈的本體作展開來顯現指令階層的平行性

```
1 #include <x86intrin.h>
2 #define UNROLL (4)
3
4 void dgemm (int n, double* A, double* B, double* C)
5 {
6     for ( int i = 0; i < n; i+=UNROLL*4 )
7         for ( int j = 0; j < n; j++ ) {
8             __m256d c[4];
9             for ( int x = 0; x < UNROLL; x++ )
10                 c[x] = _mm256_load_pd(C+i*x*4+j*n);
11
12             for( int k = 0; k < n; k++ )
13             {
14                 __m256d b = _mm256_broadcast_sd(B+k+j*n);
15                 for (int x = 0; x < UNROLL; x++)
16                     c[x] = _mm256_add_pd(c[x],
17                                     _mm256_mul_pd(_mm256_load_pd(A+n*k+x*4+i), b));
18             }
19
20             for ( int x = 0; x < UNROLL; x++ )
21                 _mm256_store_pd(C+i*x*4+j*n, c[x]);
22         }
23 }
```

4.12 走得更快：指令階層平行性與矩陣乘法

▣ 圖4.81 列出展開後的組合語言輸出碼

- 我們僅需一份**vbroadcastsd** 指令
 - 所以圖3.24 中的五道AVX 指令在圖4.81 中成為了17 道指令
- 迴圈雖然展開了四次，迴圈本體內的指令數卻僅由12 道加倍而成為24 道

4.12 走得更快：指令階層平行性與矩陣乘法

▣ 圖4.82

- 相較於圖3.21 中的DGEMM，利用部分字的平行性以及指令階層平行性做最佳化之後可以達到8.8 倍的整體加速

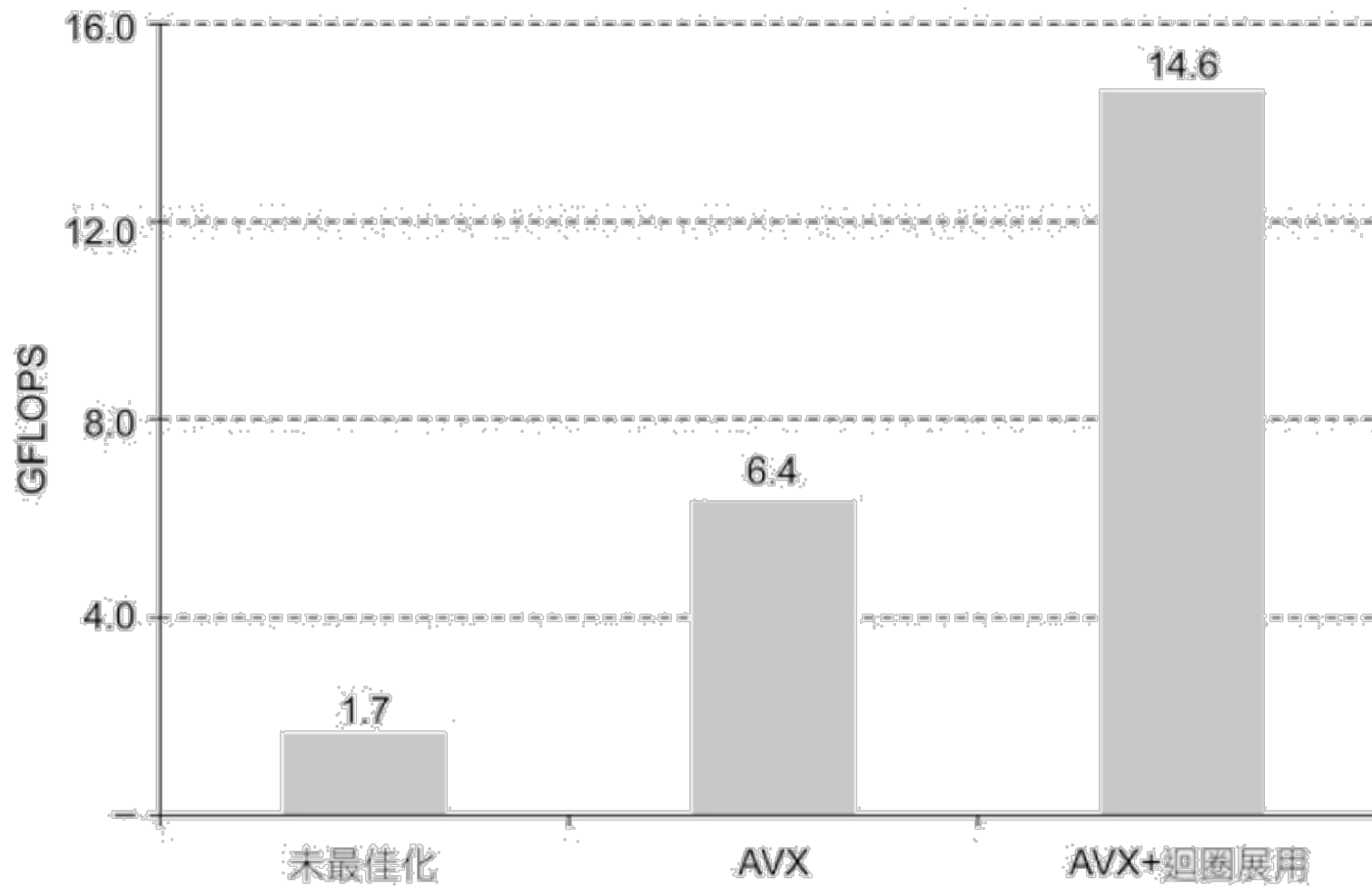


圖4.82 三種版本的DGEMM 對32×32 的矩陣的效能

部分字平行性與指令階層平行性導致相較於圖3.21 中未作最佳化的碼的將近9倍的加速

4.14 謬誤與陷阱

- ▣ 謬誤：管道化處理很容易
- ▣ 謬誤：管道處理的構想可以在不考慮半導體技術的情況下來設計
- ▣ 陷阱：不仔細思考指令集的設計將會嚴重影響管道化的處理
 - 變化很大的指令長度以及執行時間可以造成管道階段間的失衡並使得危障偵測極端困難
 - 繁複的各種定址模式可能造成各種問題

4.15 總結評論

- ▣ 處理器的數據通道與控制二者的設計都可以始於指令集架構以及對現行科技各項基本特性的瞭解
- ▣ 管道化處理改善了處理量而非指令本身的執行時間，或稱指令延遲(instruction latency)
- ▣ 相較於管道化，多重指令派發明顯地是著重於降低每道指令所需的時脈週期數(CPI)

4.15 總結評論

- ▣ 1990 年代中期由於採用了更長的管道、多重指令派發以及動態排程，使得始於1980 年代早期的處理器效能每年提昇60% 的趨勢得以維持
 - 終於都遇上了功耗障壁
 - 業界被迫嘗試多處理器的設計，其發掘的是更為大些的階層上的平行度(此為第六章的主題)
- ▣ 要經由平行處理器的使用來維持處理效能的進步時，Amdahl 定律指出系統的另一部分將會形成瓶頸
 - 那個瓶頸：記憶體階層即為下一章的主題